

烟幕干扰弹投放策略的时空协同优化

摘要：

针对无人机投放烟幕干扰弹以遮蔽圆柱形真目标的问题，本文建立导弹、无人机、烟幕弹与烟幕云团的统一三维运动学模型，并以“来袭导弹到真目标的全部视线均被烟幕阻断”作为有效遮蔽判据。区别于只考察目标中心视线的近似方法，本文将真目标视为具有实际尺寸的圆柱体：单云团情形利用烟幕球对导弹视点形成的切锥，将整圆柱遮蔽转化为上、下底圆周上的连续极值问题；多云团情形采用“对每条目标视线，至少存在一个有效云团与之相交”的联合覆盖口径，即 $\forall\text{-ray } \exists\text{-cloud}$ ，从而允许不同云团共同覆盖目标视域。有效遮蔽时长由连续时间区间的并集测度计算，重叠时段不重复计时，不连续时段可以累加。

对于问题一，在 $g = 9.8 \text{ m/s}^2$ 下求得投放点为 $(17620, 0, 1800) \text{ m}$ ，起爆点为 $(17188, 0, 1736.496) \text{ m}$ ，严格整圆柱遮蔽区间为 $[8.056445, 9.448088] \text{ s}$ ，有效时长为 1.391643 s 。目标中心视线近似给出 1.435082 s ，比严格结果多计约 3.121% 。对于问题二，采用中心视线代理筛选、差分进化广域搜索、多起点无梯度局部精修和严格几何统一复算，得到当前最佳已知时长 4.542880 s 。对于问题三，在同机三弹及投弹间隔约束下，得到联合遮蔽区间 $[2.764945, 8.278596] \text{ s}$ ，总时长 5.513651 s ；当前候选实质由前两枚弹完成，不能据此断言第三枚弹在全局最优策略中无用。对于问题四，三架无人机各投一弹形成三段接力遮蔽，联合总时长为 10.138857 s 。对于问题五，以三枚导弹遮蔽时长之和为主目标，通过单弹候选库、分配组合、逐机块精修和边际填充构成 40 维分层求解，得到 $(D_1, D_2, D_3) = (10.250038, 1.912852, 1.741134) \text{ s}$ ， $J_{\text{sum}} = 13.904024 \text{ s}$ ，公平性诊断量 $J_{\text{min}} = 1.741134 \text{ s}$ ；最终五架无人机各使用一枚有效弹。各问题均进行了可行性、网格收敛、边界探针、独立视线—球相交和内核回归检查。由于问题二至问题五的优化均受评估预算限制，本文只将其结果称为经过严格复算的最佳已知策略，不作全局最优声明。

关键词：烟幕干扰；视线遮蔽；圆柱目标；联合覆盖；区间并集；非光滑优化

AI 辅助标注：本文第 1–13 节的题面整理、模型规格检查、代码实现、数值验证与文字初稿使用了 OpenAI Codex (GPT-5) 辅助 [2]；判据选择、参数口径和最终取舍由使用者确认，数值由随文程序复算。关键交互与人工修改情况见支撑材料《AI 工具使用详情》。

1 问题重述

本题参数与任务定义依据 2025 年全国大学生数学建模竞赛 A 题 [1]。

警戒雷达发现三枚来袭导弹后，控制中心需要指挥无人机在来袭导弹与真目标之间投放烟幕干扰弹。导弹以 300 m/s 的速度直指位于坐标原点的假目标；无人机受领任务后可瞬时改变航向，随后以 $70\text{—}140\text{ m/s}$ 的速度等高度直线飞行。烟幕弹脱离无人机后作抛体运动，起爆时瞬间形成半径为 10 m 的球形有效烟幕区；云团在起爆后 20 s 内以 3 m/s 的速度竖直下沉。同一架无人机相邻两次投弹的时间间隔不得小于 1 s 。

真目标为半径 7 m 、高 10 m 的圆柱，下底面圆心为 $(0, 200, 0)$ 。需要依次解决：固定策略下单枚烟幕弹的有效遮蔽时长；单机单弹最优投放策略；单机三弹协同策略；三机各一弹协同策略；以及五机、每机至多三弹对三枚导弹的综合干扰策略。

本题的关键并非只求烟幕球与一条代表视线的距离，而是回答以下三个问题：其一，何时整座圆柱目标均被遮住；其二，多朵烟幕能否共同覆盖不同的目标视线；其三，多段遮蔽在时间上重叠或分离时如何正确计时。为此，本文先建立统一运动学和严格遮蔽几何，再将各小问表达为连续时间区间测度的计算或优化问题。

2 模型假设

1. 烟幕弹在投放瞬间继承无人机的水平速度，初始竖直速度为零；投放后忽略空气阻力，仅受重力作用。
2. 无人机接到任务后可瞬时完成转向，此后以固定航向、固定速度等高度直线飞行，不考虑转向和加速过程。
3. 烟幕弹起爆后瞬时形成半径固定为 10 m 的均匀有效球形区域；不考虑云团扩散、浓度渐变与成云时间。
4. 无风条件下，云团起爆后的水平坐标保持不变，仅以 3 m/s 竖直下沉，并按题意持续有效 20 s 。
5. 烟幕弹须在触地前起爆；导弹到达假目标原点后停止计时。
6. “有效遮蔽”定义为导弹到真目标的全部视线均在到达目标前穿过至少一朵有效烟幕，而非只遮挡圆柱中心。
7. 主计算采用 $g = 9.8\text{ m/s}^2$ ；问题一另用 $g = 9.81\text{ m/s}^2$ 检查敏感性。

上述假设与题面给定的理想烟幕模型一致，但不代表真实风场、扩散和制导反馈已被描述。数值结果的适用范围受这些假设限制。

3 符号说明

符号	含义	单位或范围
O	假目标原点	m
$\mathcal{T}$	真目标圆柱实体	—
$M_{j0}, M_j(t)$	导弹 j 的初始位置、时刻 t 的位置	m
$U_{i0}, U_i(t)$	无人机 i 的初始位置、时刻 t 的位置	m
θ_i	无人机 i 的水平航向角, 从 $+x$ 轴逆时针计	rad
v_i	无人机 i 的速度	[70, 140] m/s
t_{ik}^r	无人机 i 第 k 枚弹的投放时刻	s
δ_{ik}	投放到起爆的引信时长	s
t_{ik}^e	起爆时刻, $t_{ik}^r + \delta_{ik}$	s
P_{ik}^r, P_{ik}^e	投放点、起爆点	m
$C_{ik}(t)$	起爆后云团中心位置	m
R, T_C	有效烟幕半径、有效寿命	10 m, 20 s
$I_j(t)$	导弹 j 在时刻 t 是否被有效遮蔽	0 或 1
E_j, D_j	导弹 j 的有效遮蔽时间集合及其总长度	s
$T^{(q)}$	问题 q ($q = 1, 2, 3, 4$) 的总遮蔽时长	s
J_{sum}	三枚导弹遮蔽时长之和, 问题五主目标	s
J_{min}	三枚导弹遮蔽时长的最小值, 公平性诊断量	s

真目标集合写为

$$\mathcal{T} = \{(x, y, z) : x^2 + (y - 200)^2 \leq 7^2, 0 \leq z \leq 10\}.$$

4 统一运动学模型

4.1 导弹与无人机轨迹

导弹始终直指假目标。令

$$\hat{d}_j = -\frac{M_{j0}}{\|M_{j0}\|_2},$$

则导弹轨迹为

$$M_j(t) = M_{j0} + 300t\hat{d}_j = \left(1 - \frac{300t}{\|M_{j0}\|_2}\right) M_{j0}, \quad 0 \leq t \leq t_j^{\text{hit}},$$

其中

$$t_j^{\text{hit}} = \frac{\|M_{j0}\|_2}{300}.$$

定义无人机 i 的水平单位航向向量

$$h_i = (\cos \theta_i, \sin \theta_i, 0),$$

其轨迹为

$$U_i(t) = U_{i0} + v_i t h_i.$$

4.2 烟幕弹与云团轨迹

第 k 枚弹的投放点为

$$P_{ik}^r = U_i(t_{ik}^r).$$

在投放到起爆之间，弹体位置为

$$B_{ik}(t) = P_{ik}^r + v_i(t - t_{ik}^r)h_i - \frac{1}{2}g(t - t_{ik}^r)^2 e_z,$$

其中 $e_z = (0, 0, 1)$ 。于是

$$t_{ik}^e = t_{ik}^r + \delta_{ik},$$

$$P_{ik}^e = U_{i0} + v_i t_{ik}^e h_i - \frac{1}{2}g\delta_{ik}^2 e_z.$$

云团在有效期内的中心位置为

$$C_{ik}(t) = P_{ik}^e - 3(t - t_{ik}^e)e_z, \quad t \in [t_{ik}^e, t_{ik}^e + 20].$$

起爆可行性要求

$$t_{ik}^r \geq 0, \quad \delta_{ik} \geq 0, \quad (P_{ik}^e)_z \geq 0, \quad t_{ik}^e \leq t_j^{\text{hit}}.$$

同一无人机的相邻投弹还需满足

$$t_{i,k+1}^r - t_{ik}^r \geq 1.$$

5 严格整圆柱遮蔽模型

5.1 单条视线与烟幕球相交

对目标点 $q \in \mathcal{T}$ ，导弹 $M_j(t)$ 到 q 的视线线段为

$$L_j(q, t) = \{M_j(t) + \lambda(q - M_j(t)) : 0 \leq \lambda \leq 1\}.$$

云团中心到该线段的最短距离为

$$\lambda^* = \text{clip}_{[0,1]} \frac{[C_{ik}(t) - M_j(t)]^\top [q - M_j(t)]}{\|q - M_j(t)\|_2^2},$$

$$d_{jik}(q, t) = \|C_{ik}(t) - [M_j(t) + \lambda^*(q - M_j(t))]\|_2.$$

当 $d_{jik}(q, t) \leq R$ 时，该视线在导弹与目标之间穿过烟幕球。使用线段而非无限直线，可避免把导弹后方或目标后方的烟幕误判为有效遮蔽。

5.2 单云团的连续圆柱极值

单云团严格遮住整个圆柱要求

$$\max_{q \in \mathcal{T}} d_{jik}(q, t) \leq R.$$

为避免以少量代表点代替整座目标，本文使用烟幕球相对导弹视点形成的切锥。记同一时刻的导弹位置与云团中心为 M, C ， $D = \|C - M\|_2$ 。当 $D > R$ 时，令

$$a = \frac{C - M}{D},$$

并定义切锥裕量

$$m(q) = \|(q - M) - [(q - M)^\top a]a\|_2 - \frac{R}{\sqrt{D^2 - R^2}}(q - M)^\top a.$$

$m(q) \leq 0$ 表示点 q 位于烟幕球的遮蔽切锥内，同时还需检查目标位于球后切平面之后。由于 $m(q)$ 对 q 为凸函数，而圆柱为凸集，最大裕量可在圆柱的极点集合取得，即上、下底面的两条圆周。由此将三维连续极值化为两个周期角度上的一维最大化。若 $D \leq R$ ，导弹位于烟幕球内，所有由导弹发出的目标视线均已与烟幕相交，程序单独处理该边界情形。

5.3 多云团联合覆盖

多云团不能简单要求“存在一朵云单独遮住整个圆柱”。本文采用

$$I_j^{\text{joint}}(t) = 1 \iff \forall q \in \partial\mathcal{T}, \exists (i, k) \in \mathcal{K}(t) : d_{jik}(q, t) \leq R,$$

其中 $\mathcal{K}(t)$ 为时刻 t 仍处于有效期内的云团集合。该量词次序允许不同云团分别遮住目标视域的不同部分。若以 $m_{jik}(q, t)$ 表示第 (i, k) 朵云对视线的带符号裕量，则联合最坏裕量为

$$G_j(t) = \max_{q \in \partial \mathcal{T}} \min_{(i,k) \in \mathcal{K}(t)} m_{jik}(q, t),$$

并有

$$I_j^{\text{joint}}(t) = 1 \iff G_j(t) \leq 0.$$

由于多个切锥的并集不再保持单个凸函数的极值结构，本文对圆柱表面进行逐级加密，并利用距离函数的 Lipschitz 上界检查网格之间是否可能存在未覆盖小孔；临界时刻再进行连续局部精修。人工构造的双云互补案例中，两朵云单独均不能完整遮住目标，而联合后能够完整覆盖，验证了该实现没有退化为单云区间并集。

5.4 遮蔽区间并集

对导弹 j ，有效时刻集合为

$$E_j = \{t \in [0, t_j^{\text{hit}}] : I_j(t) = 1\},$$

总遮蔽时长为

$$D_j = \mu(E_j),$$

其中 μ 表示一维时间长度。数值上先用固定时间网格定位状态变化区间，再用 Brent 法求连续边界；最后合并相交或相邻区间并求总长度。时间网格只用于寻找根的括区，不用“有效网格点数乘步长”估计时长。多枚弹同时有效时只计算区间并集，不能把各弹时长直接相加。

6 优化方法与验证框架

遮蔽时长随决策变量变化时会出现区间生成、消失、合并和分裂，因此目标函数非凸且不保证光滑。本文采用分阶段无梯度优化：先使用较低分辨率几何代理和结构化可行解扩大候选覆盖，再用差分进化进行广域搜索；随后对多个可行候选采用 Nelder-Mead 局部精修；最终统一使用严格整圆柱判据、自适应表面细化与连续时间求根复算并排名。代理时间网格固定锚定于任务绝对时间 $t = 0$ ，同时加入起爆、失效等事件时刻，避免候选有效窗口变化导致采样相位改变。

正式验证包括：

1. 检查速度、投放时刻、引信、投弹间隔、起爆高度和命中前起爆等可行性条件；
2. 提高时间和目标表面分辨率，比较遮蔽时长的收敛性；
3. 在每个区间起点前、起点、内部、终点和终点后检查遮蔽状态；
4. 不调用主切锥公式，直接以视线线段—球相交进行高密度独立复算；
5. 检查云团标签置换不变性、单云内核回归和多云互补构造；
6. 记录随机种子、求解预算、停止状态和近优候选。

问题二至问题四的差分进化与部分局部精修均达到预设评估预算上限，问题五的分层候选与块精修同样受固定预算限制。因此，问题二至问题五的结果均称为“当前最佳

已知策略”；其可行性和目标值经过严格复算，但没有获得全局最优证明。

7 问题一：固定策略下的遮蔽时长

FY1 以 120 m/s 朝原点方向飞行，故 $h_1 = (-1, 0, 0)$ 。投放时刻为 1.5 s，引信时长为 3.6 s，起爆时刻为 5.1 s。由运动学模型得到

$$P_{11}^r = (17620, 0, 1800) \text{ m},$$

$$P_{11}^e = (17188, 0, 1736.496) \text{ m}.$$

严格整圆柱与目标中心视线两种口径的结果如下。

判据	遮蔽区间 / s	总时长 / s
严格整圆柱	[8.0564454904, 9.4480881586]	1.3916426682
中心视线近似	[8.0130060567, 9.4480881586]	1.4350821019

因此问题一的有效遮蔽时长为

$$T^{(1)} = 1.392 \text{ s}.$$

中心视线近似多计 0.043439 s，相对严格结果高约 3.121%，说明目标的有限尺寸会显著影响遮蔽开始时刻。采用 $g = 9.81 \text{ m/s}^2$ 时，严格时长为 1.405034 s，比主值增加约 0.962%。

圆周初始采样数从 256 增至 4096 时结果保持一致；时间扫描步长从 0.10 s 减至 0.01 s 时，时长最大差约 $4.1 \times 10^{-11} \text{ s}$ 。对圆柱表面独立采样 349,920 点所作的视线线段距离检查，与区间边界前后状态全部一致。导弹与云团最近距离为 4.716707 m，表明导弹在遮蔽过程中进入烟幕球，因此必须保留 $D \leq R$ 的特殊几何分支。

8 问题二：单机单弹策略优化

8.1 优化模型

决策变量为

$$x = (\theta, v, t^r, \delta),$$

目标是在可行域内最大化严格整圆柱遮蔽区间并集长度：

$$\max_{x \in \mathcal{X}_2} T^{(2)}(x).$$

其中 $0 \leq \theta < 2\pi$ ， $70 \leq v \leq 140$ ， $t^r \geq 0$ ， $\delta \geq 0$ ，且烟幕弹须在触地前、M1 到达假目标前起爆。中心视线结果仅用于生成搜索候选和结果对照，不参与最终排名。

8.2 最佳已知策略与结果

项目	数值
航向角	176.640993°
飞行速度	70.000000 m/s
投放时刻	0 s
投放点	(17800, 0, 1800) m
引信时长	2.496785640 s
起爆时刻	2.496785640 s
起爆点	(17625.525268, 10.240445, 1769.453701) m
严格遮蔽区间	[2.496787690, 7.039667637] s
严格总时长	4.542879947 s
中心视线时长	4.572673761 s

故问题二当前最佳已知严格遮蔽时长为

$$T^{(2)} = 4.543 \text{ s}.$$

该策略位于速度下界和投放时刻下界，说明在当前几何位置与单弹约束下，尽早释放并保持较低飞行速度更有利于把起爆点布置在有效视线附近。四档精度结果的最大差为 7.2×10^{-12} s；88,560 个圆柱表面点的独立视线检查通过；对航向、速度、投放时刻和引信的六项可行局部扰动均未改善当前目标值。将问题一策略输入该通用内核，时长误差仅为 1.78×10^{-15} s。

9 问题三：FY1 三弹协同策略

9.1 投弹时序参数化

三枚弹共享 FY1 的航向 θ 与速度 v 。为自动满足投放顺序和至少 1 s 的间隔，引入非负变量 s_0, s_1, s_2 ：

$$t_1^r = s_0, \quad t_2^r = s_0 + 1 + s_1, \quad t_3^r = s_0 + 2 + s_1 + s_2.$$

每枚弹配置独立引信 $\delta_1, \delta_2, \delta_3$ ，故共有八个决策变量：

$$x = (\theta, v, s_0, s_1, s_2, \delta_1, \delta_2, \delta_3).$$

目标为最大化三朵云对 M1 的 $\forall$ -ray $\exists$ -cloud 联合遮蔽时间：

$$\max_{x \in \mathcal{X}_3} T^{(3)}(x).$$

9.2 最佳已知策略

FY1 的共享航向角为 177.891490°，共享速度为 75.537408 m/s。三枚弹的策略如下。

弹号	投放时刻 / s	投放点 (x, y, z) / m	引信 / s	起爆时刻 / s	起爆点 (x, y, z) / m
1	0.0000866	(17799.9935, 0.0002, 1800)	2.7648584	2.7649450	(17591.2846, 7.6843, 1762.5422)
2	1.0001937	(17724.4991, 2.7797, 1800)	2.5816390	3.5818327	(17529.6208, 9.9546, 1767.3422)
3	2.0002754	(17649.0067, 5.5591, 1800)	9.3343535	11.3346290	(16944.3912, 31.5010, 1373.0622)

相邻投放间隔分别为 1.0001071 s 和 1.0000818 s，满足题面约束。联合遮蔽区间为

$$[2.7649450441, 8.2785964073] \text{ s},$$

总时长为

$$T^{(3)} = 5.514 \text{ s}.$$

第一、二枚弹的单独严格遮蔽区间分别为 $[2.764945, 6.913706]$ s 和 $[5.043020, 8.278596]$ s，单独时长分别为 4.148761 s 和 3.235576 s。它们重叠约 1.870686 s，合并后延长了连续遮蔽窗口。第三枚弹在当前候选中的单独严格时长为零，删除后的联合时长变化仅为 6.2×10^{-13} s；在固定前两枚的条件搜索中也未发现超过 10^{-4} s 的实质改进。因此，只能说明当前最佳已知候选没有有效利用第三枚弹，不能证明三弹问题的全局最优策略必然只需要两枚弹。

当前策略的联合时长与“单云完整遮蔽区间并集”只相差约 4.1×10^{-10} s，即该数值候选主要体现时间接力，而非目标视域互补。三档精度的时长最大差为 3.25×10^{-13} s；110,160 条独立视线检查、六种云团标签排列、问题二单云回归及人工互补覆盖案例均通过。

10 问题四：三机各一弹协同策略

10.1 优化模型

FY1、FY2、FY3 各投放一枚弹。每架无人机拥有独立的

$$x_i = (\theta_i, v_i, t_i^r, \delta_i),$$

共构成 12 维决策向量。由于每机只投一弹，同机投弹间隔约束不激活。目标仍为最大化对 M1 的联合遮蔽区间并集长度：

$$\max_{x \in \mathcal{X}_4} T^{(4)}(x).$$

10.2 最佳已知策略

无人机	航向角 / °	速度 / m/s	投放时刻 / s	投放点 (x, y, z) / m	引信 / s	起爆时刻 / s	起爆点 (x, y, z) / m
FY1	179.301490	136.942948	0.815997	(17688.263, 1.362, 1800)	3.963735	4.779732	(17145.498, 7.980, 1723.015)
FY2	290.170336	112.126363	7.471102	(12288.852, 613.668, 1400)	5.477371	12.948473	(12500.621, 37.175, 1252.992)
FY3	120.730428	138.527165	19.053573	(4651.249, - 731.190, 700)	6.818714	25.872287	(4168.570, 80.751, 472.175)

三段联合遮蔽区间为

$$[5.1449298089, 9.5689345086],$$

$$[12.9484732632, 16.4222046726],$$

$$[42.4765836426, 44.7177048596].$$

其时长分别为 4.424005 s、3.473731 s 和 2.241121 s，总时长为

$$T^{(4)} = 10.139 \text{ s}.$$

删除 FY1、FY2、FY3 所对应云团后，联合时长分别减少 4.423935 s、3.473656 s 和 2.241112 s，说明三枚弹均有实质贡献。该策略形成三段相互分离的时间接力，没有产生可报告的目标视域互补增益。三档精度计算得到 10.138972 s、10.138934 s 和 10.138857 s，最大差约 1.15×10^{-4} s，支持毫秒级结果报告。每段区间均通过 110,160 条独立视线的边界检查；云团标签置换不改变总时长，问题三几何内核回归误差为零。

当前生成的 result1.xlsx 与 result2.xlsx 仅为标注了 provisional-schema 的临时工作簿，因为题面所述官方模板未随当前资料提供。正式提交前应在不重新求解的前提下，将已计算字段映射到官方模板。

11 问题五：五机多弹对三枚导弹的综合干扰

11.1 多导弹目标函数与 40 维决策

问题五同时使用 FY1—FY5，每架无人机至多投放三枚烟幕弹，对 M1、M2、M3 实施干扰。对导弹 j ，仍按第 5.3 节的联合视线判据分别求得有效集合 E_j 和时长 $D_j = \mu(E_j)$ 。主目标取三枚导弹的总遮蔽导弹一秒数：

$$\max_{x \in \mathcal{X}_5} J_{\text{sum}}(x), \quad J_{\text{sum}}(x) = D_1(x) + D_2(x) + D_3(x).$$

该定义允许不同导弹在同一物理时刻各自贡献遮蔽时长，但同一导弹被多朵云重复遮挡时仍只计一次。为显示总和目标可能掩盖的分配不均衡，另报告

$$J_{\min}(x) = \min\{D_1(x), D_2(x), D_3(x)\}.$$

$J_{\min}$ 只作为公平性诊断量，不参与当前候选排名。若任务目标改为优先保障受遮蔽最少的导弹，可将候选按 $(J_{\min}, J_{\text{sum}})$ 的字典序重新搜索和排名。

每架无人机的三枚弹共享航向 θ_i 和速度 v_i 。沿用问题三的等待变量参数化，令

$$t_{i1}^r = s_{i0}, \quad t_{i2}^r = s_{i0} + 1 + s_{i1}, \quad t_{i3}^r = s_{i0} + 2 + s_{i1} + s_{i2},$$

并为三枚弹设置独立引信 $\delta_{i1}, \delta_{i2}, \delta_{i3}$ 。因此每架无人机有

$$x_i = (\theta_i, v_i, s_{i0}, s_{i1}, s_{i2}, \delta_{i1}, \delta_{i2}, \delta_{i3})$$

八个连续变量，五架无人机合计 40 维。由于增加一朵可行云不会缩小任何导弹的遮蔽集合，求解阶段先为每架无人机保留三个弹位；正式结果再根据删除贡献，把数值零贡献弹位标记为未使用，从而满足“每架至多三枚”的要求。

11.2 分层求解与 persistent incumbent

直接在 40 维非光滑空间中进行一次全局搜索难以兼顾覆盖范围和严格几何计算成本，因此采用以下分层策略：

1. 对 5 架无人机与 3 枚导弹的 15 个组合分别开展单弹搜索，形成候选库。每个组合使用 64 次评估；其中 13 个组合在当前预算内找到正遮蔽，FY1-M2 与 FY1-M3 仅能记为“当前预算未找到正覆盖”，不能解释为物理不可行。
2. 从候选库中组装使三枚导弹均受到干扰的分配方案，共评估 243 个有效分配子集，并始终保留问题四在 M1 上的已验证三云策略作为可行下界。
3. 固定其余无人机后，依次对每架无人机的八维变量作一次块坐标精修，每个块使用 16 次评估；代理时间网格继续锚定于绝对任务时间 $t = 0$ 。
4. 首轮严格复算显示仅五个弹位具有正删除贡献。为检验其余弹位能否填补三枚导弹的未覆盖时段，又按无人机完成 5 组、每组 16 次评估的边际填充搜索。
5. 边际填充产生的新候选经严格复算后， $J_{\text{sum}} = 13.8693042 \text{ s}$ ，低于此前已经正式验证的 13.9040240 s 。因此保留后者作为 persistent incumbent，防止后续有预算搜索使正式结果降级。

上述流程是有预算的分层启发式搜索。persistent incumbent 表示始终保留并重新严格复算的最佳已知可行解，不意味着它已被证明为 40 维全局最优。

11.3 最佳已知投放策略

删除贡献诊断表明，最终五架无人机各使用一枚有效弹，其余十个弹位的总目标删除损失为数值零，故标记为未使用。实际使用策略如下。

无人机	主要干扰导弹	航向角 / °	速度 / m/s	投放时刻 / s	投放点 (x, y, z) / m	引信 / s	起爆时刻 / s	起爆点 (x, y, z) / m
FY1	M1	179.301490	136.942948	0.815997	(17688.263, 1.362, 1800)	4.013282	4.829279	(17138.713, 8.062, 1721.078)
FY2	M1	290.170336	112.126363	7.471102	(12288.852, 613.668, 1400)	5.477371	12.948473	(12500.621, 37.175, 1252.992)
FY3	M1	121.013390	138.340570	19.098230	(4638.709, − 735.631, 700)	6.946564	26.044794	(4143.569, 87.984, 463.552)
FY4	M2	303.831378	118.628302	6.375676	(11421.090, 1371.727, 1800)	9.749407	16.125083	(12065.003, 411.000, 1334.250)
FY5	M3	135.527992	114.522224	15.757616	(11712.252, − 735.770, 1300)	4.584975	20.342591	(11337.558, − 367.919, 1196.992)

在删除贡献所用的诊断精度下, FY1、FY2、FY3 对 D_1 的删除损失分别为 4.465017 s、3.473521 s 和 2.311075 s; FY4 对 D_2 的删除损失为 1.912852 s, FY5 对 D_3 的删除损失为 1.741134 s。五枚实际使用弹均有正贡献。

11.4 三导弹遮蔽区间与目标值

各导弹的严格联合遮蔽区间为:

导弹	区间序号	联合遮蔽区间 / s	区间时长 / s
M1	1	[4.9626149841, 9.4278445555]	4.4652295714
M1	2	[12.9484732632, 16.4222046750]	3.4737314118
M1	3	[40.9072599455, 43.2183371370]	2.3110771915
M2	1	[16.1250830957, 18.0379351471]	1.9128520514
M3	1	[20.3425906024, 22.0837243332]	1.7411337307

因此

$$(D_1, D_2, D_3) = (10.2500381747, 1.9128520514, 1.7411337307) \text{ s},$$

$$J_{\text{sum}} = 13.904024 \text{ s}, \quad J_{\text{min}} = 1.741134 \text{ s}.$$

M1 获得三段接力遮蔽, M2 与 M3 各获得一段。 D_1 不低于问题四的 M1 可行下界 10.1388573 s, 但 J_{min} 仅为 1.7411337 s, 说明按总和排名的策略明显偏向 M1; 这正是同时报告 J_{min} 的必要性。

11.5 数值验证与结论边界

40 维策略的可行性违反为零; 全部 15 个参数化弹位均满足同机相邻投放间隔不小于 1 s, 且起爆高度均为正。M1、M2、M3 在三档时间—表面精度下的时长跨度分别为 2.13×10^{-4} s、 6.90×10^{-5} s 和 9.05×10^{-8} s。对五个遮蔽区间分别检查 44,280 条独立视线, 区间中点均完整遮挡, 起点前和终点后均存在未遮挡视线。

云团标签排列抽样的最大时长偏差为零; 问题四在 M1 上的几何回归误差为 9.92×10^{-10} s; J_{sum} 与三个分量之和完全一致, 且 M1 时长满足问题四单调下界。11 个事件测试、编译、格式和静态检查均通过。

上述证据支持当前策略的可行性、时长复算和几何实现一致性, 但不能证明全局最优。候选库、分配组合、块精修和边际填充均有明确预算, 40 维目标还包含区间生成与消失带来的非光滑性。若更强调三枚导弹的最低保障, 应改用 J_{min} 优先口径重新优化, 而不能把当前 J_{min} 诊断值误解为公平目标的最优值。

当前生成的 result3.xlsx 与问题三、问题四对应工作簿一样，使用带 “provisional-schema” 标记的临时字段结构，因为官方模板未随当前资料提供。获得官方模板后，只需映射已有策略和区间字段，无需重新求解。

12 模型评价

12.1 模型优点

1. **严格考虑目标尺寸。**本文没有用目标中心点替代圆柱，而是从导弹观察几何出发，判断整座圆柱是否处于烟幕球的遮蔽视域内。问题一中中心视线近似多计约 3.121%，说明该改进具有实际数值意义。
2. **联合覆盖量词符合多烟幕物理含义。**多云团判据采用 $\forall\text{-ray } \exists\text{-cloud}$ ，允许不同烟幕共同覆盖目标视域，而不是要求某一朵云独立完成全遮蔽。人工互补案例验证了这一能力。
3. **时间统计避免重复计数。**所有结果均以连续遮蔽区间的并集长度计算，既能处理不连续时段，也不会把重叠烟幕时长重复相加。
4. **求解与正式评估分离。**低分辨率代理用于扩大候选覆盖，最终结果统一通过严格几何和连续时间求根复算，减少了粗网格直接充当答案的误差。
5. **分层方法适应高维多导弹调度。**问题五将 40 维联合搜索拆成单弹候选库、分配组合、逐机块精修和边际填充，并通过 persistent incumbent 保留已严格验证的最佳已知可行解，避免后续低预算搜索造成结果退化。
6. **验证链条较完整。**模型通过网格收敛、边界前后状态、独立视线—球相交、标签置换、互补构造和前序问题回归检查，能够区分软件数值正确性与模型现实有效性。

12.2 模型局限

1. 烟幕被理想化为半径固定、内部均匀有效的球体，未考虑风场、扩散、浓度衰减、地面效应和成云过程。
2. 导弹始终直指假目标，不考虑制导系统在受干扰后的航迹变化；无人机也忽略转向和加速时间。
3. 多阶段无梯度优化只能在有限预算内搜索复杂的非光滑目标。问题二至问题五均已严格复算，但求解器没有给出全局收敛证据。
4. 问题三当前候选未有效利用第三枚弹，且问题三、四的最佳已知候选没有表现出数值可见的视域互补增益；这属于当前搜索结果的特征，不应上升为一般结论。
5. 题面所述三个官方 Excel 模板当前缺失，现有工作簿只能作为临时字段结构，提交前仍需做格式映射。
6. 问题五的 40 维分层搜索同样没有全局最优证明；15 个单弹组合中有两个在当前预算内未找到正覆盖，不能据此判定其物理不可行。
7. 问题五按 J_{sum} 排名，结果明显偏向 M1，而 $J_{\text{min}} = 1.741134 \text{ s}$ 。若任务更重视最低保障或导弹间公平性，需要更换目标函数并重新搜索，不能仅对现有候选作文字解释。

13 结论

本文围绕“烟幕何时真正遮住具有实际尺寸的圆柱目标”建立统一的三维运动学—视线几何模型。单云团情形通过切锥和圆柱极点归约实现严格连续判定，多云团情形通过 $\forall$ -ray $\exists$ -cloud 联合覆盖描述视域协同，并以连续遮蔽区间并集作为统一目标。

问题一固定策略的严格遮蔽时长为 1.392 s，目标中心视线近似会产生约 3.121% 的高估。问题二单机单弹当前最佳已知时长为 4.543 s。问题三同机三弹当前最佳已知时长为 5.514 s，其中前两枚弹形成连续接力，第三枚弹在当前候选中无实质贡献。问题四三机各一弹形成三段不连续接力，当前最佳已知总时长为 10.139 s。问题五的五枚实际使用弹分别由 FY1—FY5 投放，三枚导弹的遮蔽时长为 (10.250038, 1.912852, 1.741134) s，总遮蔽导弹一秒数为 13.904024 s，最短单导弹遮蔽时长为 1.741134 s。该结果说明总和与目标能够形成可行的多导弹分配，但会把更多资源配置给边际收益较高的 M1；若强调均衡保障，应改用 $J_{\min}$ 优先目标重新求解。

五个问题的正式结果均通过相应精度复算和独立几何检查。问题一是固定输入下的可复算结果；问题二至问题五属于有限预算搜索所得的最佳已知策略，数值验证不能替代全局最优证明，也不能消除理想球形烟幕、无风和无制导反馈等模型假设带来的现实局限。

参考文献

参考文献

- [1] 全国大学生数学建模竞赛组委会. 2025 年全国大学生数学建模竞赛 A 题：烟幕干扰弹的投放策略 [Z]. 2025.
- [2] OpenAI Codex, GPT-5, OpenAI, 使用日期：2026-08-20–2026-08-21.

A 支撑材料文件列表

本论文的支撑材料与论文电子版分开提交。主要文件如下：

文件或目录	内容说明
problem.md	经页面核对的结构化题面
model-spec.md	模型边界、变量、约束、判据与需求追踪
configs/	问题一至问题五的正式运行配置
src/q1.py–src/q5.py	五个问题的完整可运行源程序
run.py	统一运行入口
tests/	联合覆盖、事件边界与回归测试
outputs/manifest.json	正式运行种子和产物清单
outputs/logs/	优化状态、边界、收敛和独立几何验证记录
outputs/tables/	结果表、区间表及三个结果工作簿

B 完整可运行源程序

以下程序与支撑材料中的源文件一致。运行环境和命令见支撑材料中的 README.md。为满足论文附录包含完整程序的要求，正式提交稿默认展开全部源代码；若仅进行正文排版预览，可将导言区的 \fullsourceappendixtrue 改为 \fullsourceappendixfalse。

B.1 统一运行入口

```

1  """Project-level entry point for the current CUMCM 2025 Problem A checkpoint."""
2
3  import json
4  from pathlib import Path
5
6  from src.q5 import run_question_5
7
8  import mmkit
9  from mmkit.artifacts import create_run_manifest, prepare_output_paths, write_manifest
10 from mmkit.experiments import set_random_seed
11
12 PROJECT_ROOT = Path(__file__).resolve().parent
13 PROBLEM_PDF = PROJECT_ROOT / "data/raw/CUMCM-2025-problem+A-Chinese.pdf"
14 Q5_CONFIG = PROJECT_ROOT / "configs/q5.json"
15
16
17 def main() -> None:
18     """Run the currently approved Question 5 checkpoint and write its manifest."""
19     output_paths = prepare_output_paths(PROJECT_ROOT / "outputs")
20     config = json.loads(Q5_CONFIG.read_text(encoding="utf-8"))
21     seed_state = set_random_seed(int(config["seed"]))
22     artifacts = run_question_5(PROJECT_ROOT, config)
23     manifest = create_run_manifest(
24         run_name=PROJECT_ROOT.name,
25         seed=seed_state.seed,
26         config={"status": "question-5", "problem": "CUMCM-2025-A", "q5": config},
27         input_files=[PROBLEM_PDF, Q5_CONFIG],
28         artifacts=artifacts,
29     )
30     manifest_path = write_manifest(
31         manifest,
32         output_paths.root / "manifest.json",
33         overwrite=True,
34     )
35     print(f"Question 5 complete; mmkit={mmkit.__version__}; manifest={manifest_path}")
36
37
38 if __name__ == "__main__":
39     main()

```

B.2 问题一程序

```

1  """Question 1 kinematics, full-cylinder shadow test, and validation outputs."""
2
3  from __future__ import annotations
4
5  import json
6  from dataclasses import asdict, dataclass

```

```

7 from pathlib import Path
8 from typing import Any
9
10 import matplotlib.pyplot as plt
11 import numpy as np
12 import pandas as pd
13 from scipy.optimize import brentq, minimize_scalar
14
15 from mmkit.artifacts import configure_plotting, prepare_output_paths, save_figure, save_table
16
17 MISSILE_INITIAL = np.array([20000.0, 0.0, 2000.0])
18 UAV_INITIAL = np.array([17800.0, 0.0, 1800.0])
19 TARGET_CENTER = np.array([0.0, 200.0, 5.0])
20 MISSILE_SPEED = 300.0
21 UAV_SPEED = 120.0
22 RELEASE_TIME = 1.5
23 FUSE_DELAY = 3.6
24 EXPLOSION_TIME = RELEASE_TIME + FUSE_DELAY
25 CLOUD_RADIUS = 10.0
26 CLOUD_DESCENT_SPEED = 3.0
27 CLOUD_LIFETIME = 20.0
28 TARGET_RADIUS = 7.0
29 TARGET_HEIGHT = 10.0
30
31
32 @dataclass(frozen=True, slots=True)
33 class OcclusionInterval:
34     """One continuous effective-occlusion interval."""
35
36     start: float
37     end: float
38
39     @property
40     def duration(self) -> float:
41         """Return interval length in seconds."""
42         return self.end - self.start
43
44
45 @dataclass(frozen=True, slots=True)
46 class QiResult:
47     """Question 1 geometry and both requested occlusion interpretations."""
48
49     gravity: float
50     release_point: tuple[float, float, float]
51     explosion_point: tuple[float, float, float]
52     strict_intervals: tuple[OcclusionInterval, ...]
53     center_intervals: tuple[OcclusionInterval, ...]
54
55     @property
56     def strict_duration(self) -> float:
57         """Total full-cylinder occlusion time."""
58         return sum(interval.duration for interval in self.strict_intervals)
59
60     @property
61     def center_duration(self) -> float:
62         """Total center-line approximation time."""
63         return sum(interval.duration for interval in self.center_intervals)
64
65
66 def missile_position(time: float) -> np.ndarray:
67     """Return M1 position at time measured from radar detection."""
68     direction = -MISSILE_INITIAL / np.linalg.norm(MISSILE_INITIAL)

```

```

69     return MISSILE_INITIAL + MISSILE_SPEED * time * direction
70
71
72 def release_point() -> np.ndarray:
73     """Return the fixed Question 1 release point."""
74     return UAV_INITIAL + np.array([-UAV_SPEED * RELEASE_TIME, 0.0, 0.0])
75
76
77 def explosion_point(gravity: float) -> np.ndarray:
78     """Return the fixed Question 1 explosion point for a declared gravity."""
79     return release_point() + np.array(
80         [-UAV_SPEED * FUSE_DELAY, 0.0, -0.5 * gravity * FUSE_DELAY**2]
81     )
82
83
84 def cloud_position(time: float, gravity: float) -> np.ndarray:
85     """Return the cloud center during its effective lifetime."""
86     if not EXPLOSION_TIME <= time <= EXPLOSION_TIME + CLOUD_LIFETIME:
87         raise ValueError("time is outside the effective cloud lifetime")
88     return explosion_point(gravity) + np.array(
89         [0.0, 0.0, -CLOUD_DESCENT_SPEED * (time - EXPLOSION_TIME)]
90     )
91
92
93 def _cone_margin(points: np.ndarray, missile: np.ndarray, cloud: np.ndarray) -> np.ndarray:
94     """Return signed tangent-cone margins; non-positive points lie in the shadow cone."""
95     cloud_vector = cloud - missile
96     cloud_distance = float(np.linalg.norm(cloud_vector))
97     if cloud_distance <= CLOUD_RADIUS:
98         return np.full(len(points), cloud_distance - CLOUD_RADIUS)
99     axis = cloud_vector / cloud_distance
100     point_vectors = np.asarray(points, dtype=float) - missile
101     axial = point_vectors @ axis
102     perpendicular = point_vectors - np.outer(axial, axis)
103     radial = np.linalg.norm(perpendicular, axis=1)
104     tangent = CLOUD_RADIUS / np.sqrt(cloud_distance**2 - CLOUD_RADIUS**2)
105     return radial - tangent * axial
106
107
108 def _minimum_target_axial(missile: np.ndarray, cloud: np.ndarray) -> float:
109     """Return the exact minimum axial coordinate over the solid target cylinder."""
110     axis = (cloud - missile) / np.linalg.norm(cloud - missile)
111     centered = float((TARGET_CENTER - missile) @ axis)
112     horizontal_support = TARGET_RADIUS * float(np.hypot(axis[0], axis[1]))
113     vertical_support = 0.5 * TARGET_HEIGHT * abs(float(axis[2]))
114     return centered - horizontal_support - vertical_support
115
116
117 def _rim_point(angle: float, height: float) -> np.ndarray:
118     return np.array(
119         [
120             TARGET_RADIUS * np.cos(angle),
121             200.0 + TARGET_RADIUS * np.sin(angle),
122             height,
123         ]
124     )
125
126
127 def _maximum_rim_margin(
128     missile: np.ndarray,
129     cloud: np.ndarray,
130     *,

```

```

131     rim_samples: int,
132 ) -> tuple[float, np.ndarray]:
133     """Maximize cone margin over the two extreme circular rims of the cylinder."""
134     if rim_samples < 32:
135         raise ValueError("rim_samples must be at least 32")
136     angles = np.linspace(0.0, 2.0 * np.pi, rim_samples, endpoint=False)
137     step = 2.0 * np.pi / rim_samples
138     best_margin = -np.inf
139     best_point = np.zeros(3)
140
141     for height in (0.0, TARGET_HEIGHT):
142         points = np.column_stack(
143             (
144                 TARGET_RADIUS * np.cos(angles),
145                 200.0 + TARGET_RADIUS * np.sin(angles),
146                 np.full_like(angles, height),
147             )
148         )
149         margins = _cone_margin(points, missile, cloud)
150         candidate_indices = np.argmax(margins, -min(4, rim_samples))[-min(4, rim_samples) :]
151         for index in candidate_indices:
152             center_angle = float(angles[index])
153
154             def negative_margin(angle: float, fixed_height: float = height) -> float:
155                 point = _rim_point(angle, fixed_height)
156                 return -float(_cone_margin(point[None, :], missile, cloud)[0])
157
158             optimized = minimize_scalar(
159                 negative_margin,
160                 bounds=(center_angle - step, center_angle + step),
161                 method="bounded",
162                 options={"xatol": 1e-13},
163             )
164             margin = -float(optimized.fun)
165             if margin > best_margin:
166                 best_margin = margin
167                 best_point = _rim_point(float(optimized.x), height)
168     return best_margin, best_point
169
170
171 def strict_shadow_margin(time: float, gravity: float, *, rim_samples: int) -> float:
172     """Return full-cylinder shadow margin; the target is occluded iff it is non-positive."""
173     missile = missile_position(time)
174     cloud = cloud_position(time, gravity)
175     cloud_distance = float(np.linalg.norm(cloud - missile))
176     if cloud_distance <= CLOUD_RADIUS:
177         return cloud_distance - CLOUD_RADIUS
178     cone_margin, _ = _maximum_rim_margin(missile, cloud, rim_samples=rim_samples)
179     tangent_plane_distance = np.sqrt(cloud_distance**2 - CLOUD_RADIUS**2)
180     behind_margin = tangent_plane_distance - _minimum_target_axial(missile, cloud)
181     return max(cone_margin, behind_margin)
182
183
184 def center_shadow_margin(time: float, gravity: float) -> float:
185     """Return center-point shadow margin; non-positive means its sightline is occluded."""
186     missile = missile_position(time)
187     cloud = cloud_position(time, gravity)
188     cloud_distance = float(np.linalg.norm(cloud - missile))
189     if cloud_distance <= CLOUD_RADIUS:
190         return cloud_distance - CLOUD_RADIUS
191     cone_margin = float(_cone_margin(TARGET_CENTER[None, :], missile, cloud)[0])
192     axis = (cloud - missile) / cloud_distance

```

```

193     target_axial = float((TARGET_CENTER - missile) @ axis)
194     behind_margin = np.sqrt(cloud_distance**2 - CLOUD_RADIUS**2) - target_axial
195     return max(cone_margin, behind_margin)
196
197
198 def _dense_target_surface(angle_count: int, level_count: int) -> np.ndarray:
199     """Sample the complete cylinder surface for an independent segment-distance check."""
200     if angle_count < 32 or level_count < 3:
201         raise ValueError("surface validation requires at least 32 angles and 3 levels")
202     angles = np.linspace(0.0, 2.0 * np.pi, angle_count, endpoint=False)
203     heights = np.linspace(0.0, TARGET_HEIGHT, level_count)
204     angle_grid, height_grid = np.meshgrid(angles, heights, indexing="ij")
205     side = np.column_stack(
206         (
207             TARGET_RADIUS * np.cos(angle_grid).ravel(),
208             200.0 + TARGET_RADIUS * np.sin(angle_grid).ravel(),
209             height_grid.ravel(),
210         )
211     )
212
213     radii = np.linspace(0.0, TARGET_RADIUS, level_count)
214     angle_grid, radius_grid = np.meshgrid(angles, radii, indexing="ij")
215     cap_x = radius_grid.ravel() * np.cos(angle_grid).ravel()
216     cap_y = 200.0 + radius_grid.ravel() * np.sin(angle_grid).ravel()
217     bottom = np.column_stack((cap_x, cap_y, np.zeros_like(cap_x)))
218     top = np.column_stack((cap_x, cap_y, np.full_like(cap_x, TARGET_HEIGHT)))
219     return np.vstack((side, bottom, top))
220
221
222 def _maximum_segment_distance(
223     points: np.ndarray,
224     missile: np.ndarray,
225     cloud: np.ndarray,
226 ) -> tuple[float, np.ndarray]:
227     """Independently maximize distance from the cloud center to sightline segments."""
228     directions = points - missile
229     squared_lengths = np.einsum("ij,ij->i", directions, directions)
230     parameters = ((cloud - missile) @ directions.T) / squared_lengths
231     parameters = np.clip(parameters, 0.0, 1.0)
232     closest = missile + parameters[:, None] * directions
233     distances = np.linalg.norm(closest - cloud, axis=1)
234     index = int(np.argmax(distances))
235     return float(distances[index]), points[index]
236
237
238 def _closest_missile_cloud_approach(gravity: float) -> tuple[float, float]:
239     """Return closest approach time and distance during the cloud lifetime."""
240
241     def squared_distance(time: float) -> float:
242         difference = missile_position(time) - cloud_position(time, gravity)
243         return float(difference @ difference)
244
245     optimized = minimize_scalar(
246         squared_distance,
247         bounds=(EXPLOSION_TIME, EXPLOSION_TIME + CLOUD_LIFETIME),
248         method="bounded",
249         options={"xatol": 1e-13},
250     )
251     return float(optimized.x), float(np.sqrt(optimized.fun))
252
253
254 def _find_intervals(

```

```

255     margin_function: Any,
256     *,
257     start: float,
258     end: float,
259     scan_step: float,
260     root_tolerance: float,
261 ) -> tuple[OcclusionInterval, ...]:
262     """Locate non-positive intervals using a scan only for brackets and Brent roots for edges."""
263     count = int(np.ceil((end - start) / scan_step))
264     times = np.linspace(start, end, count + 1)
265     margins = np.array([margin_function(float(time)) for time in times])
266     inside = margins <= 0.0
267     boundaries: list[tuple[float, bool]] = []
268     for index in np.flatnonzero(inside[1:] != inside[:-1]):
269         left = float(times[index])
270         right = float(times[index + 1])
271         root = float(brentq(margin_function, left, right, xtol=root_tolerance, rtol=1e-14))
272         boundaries.append((root, bool(inside[index + 1])))
273
274     intervals: list[OcclusionInterval] = []
275     current_start = start if inside[0] else None
276     for boundary, enters in boundaries:
277         if enters:
278             current_start = boundary
279         elif current_start is not None:
280             intervals.append(OcclusionInterval(current_start, boundary))
281             current_start = None
282     if current_start is not None:
283         intervals.append(OcclusionInterval(current_start, end))
284     return tuple(intervals)
285
286
287 def solve_question_1(config: dict[str, Any], gravity: float) -> Q1Result:
288     """Solve Question 1 for one declared gravity value."""
289     start = EXPLOSION_TIME
290     end = EXPLOSION_TIME + CLOUD_LIFETIME
291     rim_samples = int(config["rim_samples"])
292     strict_intervals = _find_intervals(
293         lambda time: strict_shadow_margin(time, gravity, rim_samples=rim_samples),
294         start=start,
295         end=end,
296         scan_step=float(config["time_scan_step"]),
297         root_tolerance=float(config["root_tolerance"]),
298     )
299     center_intervals = _find_intervals(
300         lambda time: center_shadow_margin(time, gravity),
301         start=start,
302         end=end,
303         scan_step=float(config["time_scan_step"]),
304         root_tolerance=float(config["root_tolerance"]),
305     )
306     return Q1Result(
307         gravity=gravity,
308         release_point=tuple(float(value) for value in release_point()),
309         explosion_point=tuple(float(value) for value in explosion_point(gravity)),
310         strict_intervals=strict_intervals,
311         center_intervals=center_intervals,
312     )
313
314
315 def _interval_rows(result: Q1Result, label: str) -> list[dict[str, float | str]]:
316     rows: list[dict[str, float | str]] = []

```

```

317     for criterion, intervals in (
318         ("strict-full-cylinder", result.strict_intervals),
319         ("center-line", result.center_intervals),
320     ):
321         for index, interval in enumerate(intervals, start=1):
322             rows.append(
323                 {
324                     "scenario": label,
325                     "gravity_m_s2": result.gravity,
326                     "criterion": criterion,
327                     "interval": index,
328                     "start_s": interval.start,
329                     "end_s": interval.end,
330                     "duration_s": interval.duration,
331                 }
332             )
333     return rows
334
335
336 def _validation_record(config: dict[str, Any], result: Q1Result) -> dict[str, Any]:
337     convergence: list[dict[str, float | int]] = []
338     gravity = result.gravity
339     for rim_samples in config["validation_rim_samples"]:
340         intervals = _find_intervals(
341             lambda time, samples=int(rim_samples): strict_shadow_margin(
342                 time, gravity, rim_samples=samples
343             ),
344             start=EXPLOSION_TIME,
345             end=EXPLOSION_TIME + CLOUD_LIFETIME,
346             scan_step=float(config["time_scan_step"]),
347             root_tolerance=float(config["root_tolerance"]),
348         )
349         convergence.append(
350             {
351                 "rim_samples": int(rim_samples),
352                 "duration_s": sum(interval.duration for interval in intervals),
353                 "interval_count": len(intervals),
354             }
355         )
356
357     time_scan_convergence: list[dict[str, float | int]] = []
358     for scan_step in config["validation_time_steps"]:
359         intervals = _find_intervals(
360             lambda time: strict_shadow_margin(
361                 time, gravity, rim_samples=int(config["rim_samples"])
362             ),
363             start=EXPLOSION_TIME,
364             end=EXPLOSION_TIME + CLOUD_LIFETIME,
365             scan_step=float(scan_step),
366             root_tolerance=float(config["root_tolerance"]),
367         )
368         time_scan_convergence.append(
369             {
370                 "scan_step_s": float(scan_step),
371                 "duration_s": sum(interval.duration for interval in intervals),
372                 "interval_count": len(intervals),
373             }
374         )
375
376     boundary_checks: list[dict[str, float | bool | str]] = []
377     boundary_tolerance = float(config["boundary_margin_tolerance"])
378     for criterion, intervals, margin_function in (

```

```

379     (
380         "strict-full-cylinder",
381         result.strict_intervals,
382         lambda time: strict_shadow_margin(
383             time, gravity, rim_samples=int(config["rim_samples"])
384         ),
385     ),
386     ("center-line", result.center_intervals, lambda time: center_shadow_margin(time, gravity)),
387 ):
388     for interval in intervals:
389         midpoint = 0.5 * (interval.start + interval.end)
390         probe = float(config["boundary_probe_seconds"])
391         for location, time, expected_state in (
392             ("before-start", interval.start - probe, "outside"),
393             ("start", interval.start, "boundary"),
394             ("midpoint", midpoint, "inside"),
395             ("end", interval.end, "boundary"),
396             ("after-end", interval.end + probe, "outside"),
397         ):
398             margin = margin_function(time)
399             if expected_state == "boundary":
400                 check_passed = abs(margin) <= boundary_tolerance
401             elif expected_state == "inside":
402                 check_passed = margin < -boundary_tolerance
403             else:
404                 check_passed = margin > boundary_tolerance
405             boundary_checks.append(
406                 {
407                     "criterion": criterion,
408                     "location": location,
409                     "time_s": time,
410                     "margin_m": margin,
411                     "expected_state": expected_state,
412                     "check_passed": check_passed,
413                 }
414             )
415
416     surface_points = _dense_target_surface(
417         int(config["surface_validation_angles"]),
418         int(config["surface_validation_levels"]),
419     )
420     surface_checks: list[dict[str, Any]] = []
421     for interval in result.strict_intervals:
422         midpoint = 0.5 * (interval.start + interval.end)
423         probe = float(config["boundary_probe_seconds"])
424         for location, time in (
425             ("before-start", interval.start - probe),
426             ("start", interval.start),
427             ("midpoint", midpoint),
428             ("end", interval.end),
429             ("after-end", interval.end + probe),
430         ):
431             missile = missile_position(time)
432             cloud = cloud_position(time, gravity)
433             maximum_distance, worst_point = _maximum_segment_distance(
434                 surface_points, missile, cloud
435             )
436             surface_checks.append(
437                 {
438                     "location": location,
439                     "time_s": time,
440                     "sampled_surface_points": int(len(surface_points)),

```



```

441         "maximum_segment_distance_m": maximum_distance,
442         "distance_margin_m": maximum_distance - CLOUD_RADIUS,
443         "occluded": maximum_distance <= CLOUD_RADIUS,
444         "worst_sampled_target_point": worst_point.tolist(),
445     }
446 )
447
448 closest_time, closest_distance = _closest_missile_cloud_approach(gravity)
449 return {
450     "model": "qi-shadow-cone-v1",
451     "criterion": "full target cylinder contained in the missile-view shadow cone",
452     "target_extreme_set": "top and bottom circular rims",
453     "gravity_m_s2": gravity,
454     "time_scan_step_s": float(config["time_scan_step"]),
455     "root_tolerance_s": float(config["root_tolerance"]),
456     "boundary_margin_tolerance_m": boundary_tolerance,
457     "rim_convergence": convergence,
458     "time_scan_convergence": time_scan_convergence,
459     "boundary_checks": boundary_checks,
460     "independent_surface_segment_checks": surface_checks,
461     "derived_checks": {
462         "missile_hit_time_s": float(np.linalg.norm(MISSILE_INITIAL) / MISSILE_SPEED),
463         "release_time_s": RELEASE_TIME,
464         "explosion_time_s": EXPLOSION_TIME,
465         "cloud_effective_end_s": EXPLOSION_TIME + CLOUD_LIFETIME,
466         "explosion_above_ground": bool(result.explosion_point[2] > 0.0),
467         "closest_missile_cloud_time_s": closest_time,
468         "closest_missile_cloud_distance_m": closest_distance,
469         "missile_enters_cloud": closest_distance < CLOUD_RADIUS,
470     },
471 }
472
473
474 def run_question_1(project_root: Path, config: dict[str, Any]) -> list[Path]:
475     """Run Question 1 and write tables, plot, and a validation record."""
476     paths = prepare_output_paths(project_root / "outputs")
477     primary = solve_question_1(config, float(config["gravity"]))
478     sensitivity = solve_question_1(config, float(config["gravity_sensitivity"]))
479
480     summary = pd.DataFrame(
481         [
482             {
483                 "scenario": "primary",
484                 "gravity_m_s2": primary.gravity,
485                 "release_x_m": primary.release_point[0],
486                 "release_y_m": primary.release_point[1],
487                 "release_z_m": primary.release_point[2],
488                 "explosion_x_m": primary.explosion_point[0],
489                 "explosion_y_m": primary.explosion_point[1],
490                 "explosion_z_m": primary.explosion_point[2],
491                 "strict_duration_s": primary.strict_duration,
492                 "center_duration_s": primary.center_duration,
493             },
494             {
495                 "scenario": "gravity-sensitivity",
496                 "gravity_m_s2": sensitivity.gravity,
497                 "release_x_m": sensitivity.release_point[0],
498                 "release_y_m": sensitivity.release_point[1],
499                 "release_z_m": sensitivity.release_point[2],
500                 "explosion_x_m": sensitivity.explosion_point[0],
501                 "explosion_y_m": sensitivity.explosion_point[1],
502                 "explosion_z_m": sensitivity.explosion_point[2],

```

```

503         "strict_duration_s": sensitivity.strict_duration,
504         "center_duration_s": sensitivity.center_duration,
505     },
506 ]
507 )
508 intervals = pd.DataFrame(
509     _interval_rows(primary, "primary") + _interval_rows(sensitivity, "gravity-sensitivity")
510 )
511 summary_path = save_table(summary, paths.tables / "q1_summary.csv", overwrite=True)
512 intervals_path = save_table(intervals, paths.tables / "q1_intervals.csv", overwrite=True)
513
514 times = np.linspace(
515     EXPLOSION_TIME,
516     EXPLOSION_TIME + CLOUD_LIFETIME,
517     int(config["plot_points"]),
518 )
519 strict_margins = np.array(
520     [
521         strict_shadow_margin(
522             float(time), primary.gravity, rim_samples=int(config["rim_samples"])
523         )
524         for time in times
525     ]
526 )
527 center_margins = np.array(
528     [center_shadow_margin(float(time), primary.gravity) for time in times]
529 )
530 configure_plotting()
531 figure, axis = plt.subplots(figsize=(8.5, 4.8))
532 axis.plot(times, strict_margins, label="Full cylinder", linewidth=2)
533 axis.plot(times, center_margins, label="Center-line approximation", linestyle="--")
534 axis.axhline(0.0, color="black", linewidth=1)
535 axis.set_xlabel("Time since task assignment (s)")
536 axis.set_ylabel("Shadow-cone margin (m)")
537 axis.set_title("Question 1 occlusion margins")
538 axis.legend()
539 figure_path = save_figure(figure, paths.figures / "q1_occlusion_margin.svg", overwrite=True)
540 plt.close(figure)
541
542 validation = _validation_record(config, primary)
543 validation["primary_result"] = {
544     **asdict(primary),
545     "strict_duration": primary.strict_duration,
546     "center_duration": primary.center_duration,
547 }
548 validation["gravity_sensitivity_result"] = {
549     **asdict(sensitivity),
550     "strict_duration": sensitivity.strict_duration,
551     "center_duration": sensitivity.center_duration,
552 }
553 validation_path = paths.logs / "q1_validation.json"
554 validation_path.write_text(
555     json.dumps(validation, ensure_ascii=False, indent=2) + "\n",
556     encoding="utf-8",
557 )
558 return [summary_path, intervals_path, figure_path, validation_path]

```

B.3 问题二程序

```

1 """Question 2 optimization using the audited Question 1 occlusion geometry."""
2

```

```

3 from __future__ import annotations
4
5 import json
6 from collections.abc import Callable
7 from dataclasses import asdict, dataclass
8 from pathlib import Path
9 from typing import Any
10
11 import numpy as np
12 import pandas as pd
13 from scipy.optimize import differential_evolution, minimize
14
15 from mmkit.artifacts import prepare_output_paths, save_table
16
17 from .q1 import (
18     CLOUD_DESCENT_SPEED,
19     CLOUD_LIFETIME,
20     CLOUD_RADIUS,
21     MISSILE_INITIAL,
22     MISSILE_SPEED,
23     TARGET_CENTER,
24     TARGET_HEIGHT,
25     TARGET_RADIUS,
26     UAV_INITIAL,
27     OcclusionInterval,
28     _cone_margin,
29     _dense_target_surface,
30     _find_intervals,
31     _maximum_rim_margin,
32     _maximum_segment_distance,
33     _minimum_target_axial,
34     missile_position,
35 )
36
37 MINIMUM_SPEED = 70.0
38 MAXIMUM_SPEED = 140.0
39 UAV_ALTITUDE = float(UAV_INITIAL[2])
40
41
42 @dataclass(frozen=True, slots=True)
43 class Q2Strategy:
44     """One feasible FY1 release strategy."""
45
46     theta_rad: float
47     speed_m_s: float
48     release_time_s: float
49     fuse_delay_s: float
50
51     @property
52     def explosion_time_s(self) -> float:
53         """Return explosion time from task assignment."""
54         return self.release_time_s + self.fuse_delay_s
55
56
57 @dataclass(frozen=True, slots=True)
58 class Q2Evaluation:
59     """Formal strict and comparison evaluation for one feasible strategy."""
60
61     strategy: Q2Strategy
62     release_point: tuple[float, float, float]
63     explosion_point: tuple[float, float, float]
64     strict_intervals: tuple[OcclusionInterval, ...]

```

```

65     center_intervals: tuple[OcclusionInterval, ...]
66     minimum_cloud_center_height_m: float
67     cloud_center_below_ground: bool
68     cloud_center_ground_crossing_time_s: float | None
69
70     @property
71     def strict_duration_s(self) -> float:
72         """Return union length of strict full-cylinder intervals."""
73         return sum(interval.duration for interval in self.strict_intervals)
74
75     @property
76     def center_duration_s(self) -> float:
77         """Return union length of center-line comparison intervals."""
78         return sum(interval.duration for interval in self.center_intervals)
79
80
81 def missile_hit_time() -> float:
82     """Return time at which M1 reaches the false target at the origin."""
83     return float(np.linalg.norm(MISSILE_INITIAL) / MISSILE_SPEED)
84
85
86 def _heading(theta_rad: float) -> np.ndarray:
87     return np.array([np.cos(theta_rad), np.sin(theta_rad), 0.0])
88
89
90 def release_point(strategy: Q2Strategy) -> np.ndarray:
91     """Return the strategy's release point."""
92     return UAV_INITIAL + strategy.speed_m_s * strategy.release_time_s * _heading(strategy.theta_rad)
93
94
95 def explosion_point(strategy: Q2Strategy, gravity: float) -> np.ndarray:
96     """Return the ballistic explosion point."""
97     horizontal = strategy.speed_m_s * strategy.explosion_time_s * _heading(strategy.theta_rad)
98     return (
99         UAV_INITIAL + horizontal + np.array([0.0, 0.0, -0.5 * gravity * strategy.fuse_delay_s**2])
100     )
101
102
103 def cloud_position(time: float, strategy: Q2Strategy, gravity: float) -> np.ndarray:
104     """Return cloud center during the declared 20-second effective window."""
105     if not strategy.explosion_time_s <= time <= strategy.explosion_time_s + CLOUD_LIFETIME:
106         raise ValueError("time is outside the effective cloud lifetime")
107     return explosion_point(strategy, gravity) + np.array(
108         [0.0, 0.0, -CLOUD_DESCENT_SPEED * (time - strategy.explosion_time_s)]
109     )
110
111
112 def _strategy_from_array(values: np.ndarray) -> Q2Strategy:
113     return Q2Strategy(*(float(value) for value in values))
114
115
116 def _feasibility_violation(strategy: Q2Strategy, gravity: float) -> float:
117     """Return a scaled nonnegative violation measure."""
118     hit_time = missile_hit_time()
119     explosion_height = float(explosion_point(strategy, gravity)[2])
120     violation = max(0.0, strategy.explosion_time_s - hit_time) / hit_time
121     violation += max(0.0, -explosion_height) / UAV_ALTITUDE
122     return violation
123
124
125 def _strict_margin(
126     time: float,

```

```

127     strategy: Q2Strategy,
128     gravity: float,
129     *,
130     rim_samples: int,
131     refine_rim: bool,
132 ) -> float:
133     """Return the generalized full-cylinder margin for one strategy."""
134     missile = missile_position(time)
135     cloud = cloud_position(time, strategy, gravity)
136     cloud_distance = float(np.linalg.norm(cloud - missile))
137     if cloud_distance <= CLOUD_RADIUS:
138         return cloud_distance - CLOUD_RADIUS
139     if refine_rim:
140         cone_margin, _ = _maximum_rim_margin(missile, cloud, rim_samples=rim_samples)
141     else:
142         angles = np.linspace(0.0, 2.0 * np.pi, rim_samples, endpoint=False)
143         x = TARGET_RADIUS * np.cos(angles)
144         y = 200.0 + TARGET_RADIUS * np.sin(angles)
145         points = np.vstack(
146             (
147                 np.column_stack((x, y, np.zeros_like(x))),
148                 np.column_stack((x, y, np.full_like(x, TARGET_HEIGHT))),
149             )
150         )
151         cone_margin = float(np.max(_cone_margin(points, missile, cloud)))
152     tangent_plane_distance = np.sqrt(cloud_distance**2 - CLOUD_RADIUS**2)
153     behind_margin = tangent_plane_distance - _minimum_target_axial(missile, cloud)
154     return max(cone_margin, behind_margin)
155
156
157 def _center_margin(time: float, strategy: Q2Strategy, gravity: float) -> float:
158     """Return the center-line comparison margin for one strategy."""
159     missile = missile_position(time)
160     cloud = cloud_position(time, strategy, gravity)
161     cloud_distance = float(np.linalg.norm(cloud - missile))
162     if cloud_distance <= CLOUD_RADIUS:
163         return cloud_distance - CLOUD_RADIUS
164     cone_margin = float(_cone_margin(TARGET_CENTER[None, :], missile, cloud)[0])
165     axis = (cloud - missile) / cloud_distance
166     target_axial = float((TARGET_CENTER - missile) @ axis)
167     behind_margin = np.sqrt(cloud_distance**2 - CLOUD_RADIUS**2) - target_axial
168     return max(cone_margin, behind_margin)
169
170
171 def _effective_window(strategy: Q2Strategy) -> tuple[float, float]:
172     start = strategy.explosion_time_s
173     return start, min(start + CLOUD_LIFETIME, missile_hit_time())
174
175
176 def _sampled_metrics(
177     strategy: Q2Strategy,
178     margin_function: Callable[[float], float],
179     *,
180     time_step: float,
181 ) -> tuple[float, float]:
182     start, end = _effective_window(strategy)
183     if end <= start:
184         return 0.0, float("inf")
185     count = max(1, int(np.ceil((end - start) / time_step)))
186     times = np.linspace(start, end, count + 1)
187     margins = np.array([margin_function(float(time)) for time in times])
188     inside = (margins <= 0.0).astype(float)

```

```

189     return float(np.trapezoid(inside, times)), float(np.min(margins))
190
191
192 def _objective(
193     values: np.ndarray,
194     gravity: float,
195     *,
196     criterion: str,
197     time_step: float,
198     rim_samples: int = 96,
199 ) -> float:
200     strategy = _strategy_from_array(values)
201     violation = _feasibility_violation(strategy, gravity)
202     if violation > 0.0:
203         return 100.0 + 100.0 * violation
204     if criterion == "center":
205
206         def margin(time: float) -> float:
207             return _center_margin(time, strategy, gravity)
208     else:
209
210         def margin(time: float) -> float:
211             return _strict_margin(
212                 time,
213                 strategy,
214                 gravity,
215                 rim_samples=rim_samples,
216                 refine_rim=False,
217             )
218
219     duration, minimum_margin = _sampled_metrics(strategy, margin, time_step=time_step)
220     if duration > 0.0:
221         return -duration
222     return max(minimum_margin, 0.0) / 100.0
223
224
225 def _root_refined_objective(
226     values: np.ndarray,
227     gravity: float,
228     *,
229     time_step: float,
230     rim_samples: int,
231     root_tolerance: float,
232 ) -> float:
233     """Return a locally useful duration objective with root-refined boundaries."""
234     strategy = _strategy_from_array(values)
235     violation = _feasibility_violation(strategy, gravity)
236     if violation > 0.0:
237         return 100.0 + 100.0 * violation
238     start, end = _effective_window(strategy)
239
240     def margin(time: float) -> float:
241         return _strict_margin(
242             time,
243             strategy,
244             gravity,
245             rim_samples=rim_samples,
246             refine_rim=False,
247         )
248
249     sampled_duration, minimum_margin = _sampled_metrics(strategy, margin, time_step=time_step)
250     if sampled_duration <= 0.0:

```

```

251         return max(minimum_margin, 0.0) / 100.0
252     intervals = _find_intervals(
253         margin,
254         start=start,
255         end=end,
256         scan_step=time_step,
257         root_tolerance=root_tolerance,
258     )
259     return -sum(interval.duration for interval in intervals)
260
261
262 def _formal_evaluation(
263     strategy: Q2Strategy,
264     gravity: float,
265     config: dict[str, Any],
266 ) -> Q2Evaluation:
267     settings = config["formal_evaluation"]
268     start, end = _effective_window(strategy)
269     strict_intervals = _find_intervals(
270         lambda time: _strict_margin(
271             time,
272             strategy,
273             gravity,
274             rim_samples=int(settings["rim_samples"]),
275             refine_rim=True,
276         ),
277         start=start,
278         end=end,
279         scan_step=float(settings["time_scan_step_s"]),
280         root_tolerance=float(settings["root_tolerance_s"]),
281     )
282     center_intervals = _find_intervals(
283         lambda time: _center_margin(time, strategy, gravity),
284         start=start,
285         end=end,
286         scan_step=float(settings["time_scan_step_s"]),
287         root_tolerance=float(settings["root_tolerance_s"]),
288     )
289     release = release_point(strategy)
290     explosion = explosion_point(strategy, gravity)
291     minimum_height = float(explosion[2] - CLOUD_DESCENT_SPEED * CLOUD_LIFETIME)
292     ground_crossing_time = (
293         strategy.explosion_time_s + float(explosion[2]) / CLOUD_DESCENT_SPEED
294         if minimum_height < 0.0
295         else None
296     )
297     return Q2Evaluation(
298         strategy=strategy,
299         release_point=tuple(float(value) for value in release),
300         explosion_point=tuple(float(value) for value in explosion),
301         strict_intervals=strict_intervals,
302         center_intervals=center_intervals,
303         minimum_cloud_center_height_m=minimum_height,
304         cloud_center_below_ground=minimum_height < 0.0,
305         cloud_center_ground_crossing_time_s=ground_crossing_time,
306     )
307
308
309 def _bounds(config: dict[str, Any]) -> list[tuple[float, float]]:
310     configured = config["bounds"]
311     return [
312         tuple(configured["theta_rad"]),

```

```

313         tuple(configured["speed_m_s"]),
314         tuple(configured["release_time_s"]),
315         tuple(configured["fuse_delay_s"]),
316     ]
317
318
319 def _initial_population(bounds: list[tuple[float, float]], popsize: int, seed: int) -> np.ndarray:
320     """Build a seeded population containing a known positive-duration baseline."""
321     rng = np.random.default_rng(seed)
322     population_count = max(5, popsize * len(bounds))
323     lower = np.array([bound[0] for bound in bounds])
324     upper = np.array([bound[1] for bound in bounds])
325     population = rng.uniform(lower, upper, size=(population_count, len(bounds)))
326     population[0] = np.array([np.pi, 120.0, 1.5, 3.6])
327     population[1] = np.array([np.pi, 140.0, 0.0, 3.6])
328     population[2] = np.array([np.pi, 100.0, 0.0, 5.0])
329     return population
330
331
332 def _deduplicate(candidates: list[np.ndarray]) -> list[np.ndarray]:
333     unique: list[np.ndarray] = []
334     for candidate in candidates:
335         if not any(np.linalg.norm(candidate - existing) < 1e-7 for existing in unique):
336             unique.append(np.asarray(candidate, dtype=float))
337     return unique
338
339
340 def optimize_question_2(config: dict[str, Any]) -> tuple[list[Q2Evaluation], dict[str, Any]]:
341     """Run center-screened and strict derivative-free optimization."""
342     gravity = float(config["gravity"])
343     seed = int(config["seed"])
344     bounds = _bounds(config)
345     center_settings = config["center_search"]
346     strict_settings = config["strict_search"]
347     center_init = _initial_population(bounds, int(center_settings["popsize"]), seed)
348
349     center_result = differential_evolution(
350         lambda values: _objective(
351             values,
352             gravity,
353             criterion="center",
354             time_step=float(center_settings["time_step_s"]),
355         ),
356         bounds,
357         seed=seed,
358         maxiter=int(center_settings["maxiter"]),
359         popsize=int(center_settings["popsize"]),
360         tol=float(center_settings["tol"]),
361         polish=bool(center_settings["polish"]),
362         init=center_init,
363         workers=1,
364     )
365     center_population = np.asarray(center_result.population, dtype=float)
366     center_order = np.argsort(center_result.population_energies)
367     strict_init = center_population[center_order]
368
369     strict_result = differential_evolution(
370         lambda values: _objective(
371             values,
372             gravity,
373             criterion="strict",
374             time_step=float(strict_settings["time_step_s"]),

```



```

375         rim_samples=int(strict_settings["rim_samples"]),
376     ),
377     bounds,
378     seed=seed + 1,
379     maxiter=int(strict_settings["maxiter"]),
380     popsize=int(strict_settings["popsize"]),
381     tol=float(strict_settings["tol"]),
382     polish=bool(strict_settings["polish"]),
383     init=strict_init,
384     workers=1,
385 )
386
387 boundary_settings = config["boundary_refinement"]
388
389 def boundary_objective(values: np.ndarray) -> float:
390     full_values = np.array(
391         [
392             values[0],
393             float(boundary_settings["speed_m_s"]),
394             float(boundary_settings["release_time_s"]),
395             values[1],
396         ]
397     )
398     return _root_refined_objective(
399         full_values,
400         gravity,
401         time_step=float(boundary_settings["time_step_s"]),
402         rim_samples=int(boundary_settings["rim_samples"]),
403         root_tolerance=float(config["formal_evaluation"]["root_tolerance_s"]),
404     )
405
406 boundary_result = differential_evolution(
407     boundary_objective,
408     [
409         tuple(boundary_settings["theta_rad"]),
410         tuple(boundary_settings["fuse_delay_s"]),
411     ],
412     seed=int(boundary_settings["seed"]),
413     maxiter=int(boundary_settings["maxiter"]),
414     popsize=int(boundary_settings["popsize"]),
415     tol=float(boundary_settings["tol"]),
416     polish=True,
417     workers=1,
418 )
419 boundary_candidate = np.array(
420     [
421         boundary_result.x[0],
422         float(boundary_settings["speed_m_s"]),
423         float(boundary_settings["release_time_s"]),
424         boundary_result.x[1],
425     ]
426 )
427 warm_start = boundary_settings["independent_warm_start"]
428 independent_warm_start = np.array(
429     [
430         float(warm_start["theta_rad"]),
431         float(boundary_settings["speed_m_s"]),
432         float(boundary_settings["release_time_s"]),
433         float(warm_start["fuse_delay_s"]),
434     ]
435 )
436

```

```

437 raw_candidates = [
438     center_result.x,
439     strict_result.x,
440     boundary_candidate,
441     independent_warm_start,
442 ]
443 strict_population = np.asarray(strict_result.population, dtype=float)
444 strict_order = np.argsort(strict_result.population_energies)
445 retained_population = max(8, int(config["local_refinement"]["starts"]))
446 raw_candidates.extend(strict_population[strict_order[:retained_population]])
447
448 local_results = []
449 local_settings = config["local_refinement"]
450 for start_values in _deduplicate(raw_candidates)[: int(local_settings["starts"])]:
451     result = minimize(
452         lambda values: _root_refined_objective(
453             values,
454             gravity,
455             time_step=float(strict_settings["time_step_s"]),
456             rim_samples=int(strict_settings["rim_samples"]),
457             root_tolerance=float(config["formal_evaluation"]["root_tolerance_s"]),
458         ),
459         start_values,
460         method="Nelder-Mead",
461         bounds=bounds,
462         options={
463             "maxfev": int(local_settings["maxfev"]),
464             "xatol": float(local_settings["xtol"]),
465             "fatol": float(local_settings["ftol"]),
466         },
467     )
468     local_results.append(result)
469     raw_candidates.append(result.x)
470
471 feasible_candidates = [
472     candidate
473     for candidate in _deduplicate(raw_candidates)
474     if _feasibility_violation(_strategy_from_array(candidate), gravity) <= 1e-12
475 ]
476 ranked_coarse = sorted(
477     feasible_candidates,
478     key=lambda values: _root_refined_objective(
479         values,
480         gravity,
481         time_step=float(strict_settings["time_step_s"]),
482         rim_samples=int(strict_settings["rim_samples"]),
483         root_tolerance=float(config["formal_evaluation"]["root_tolerance_s"]),
484     ),
485 )
486 formal_count = min(int(config["formal_evaluation"]["candidate_count"]), len(ranked_coarse))
487 formal = [
488     _formal_evaluation(_strategy_from_array(values), gravity, config)
489     for values in ranked_coarse[:formal_count]
490 ]
491 formal.sort(key=lambda evaluation: evaluation.strict_duration_s, reverse=True)
492
493 solver_record = {
494     "seed": seed,
495     "variable_order": ["theta_rad", "speed_m_s", "release_time_s", "fuse_delay_s"],
496     "bounds": config["bounds"],
497     "candidate_initialization": {
498         "mechanism": "seeded uniform population with Q1 and two heading baselines injected",

```

```

499         "reason": "positive-duration strategies occupy a sparse subset of the box bounds",
500         "baseline_q1": [float(np.pi), 120.0, 1.5, 3.6],
501     },
502     "center_differential_evolution": {
503         "success": bool(center_result.success),
504         "message": str(center_result.message),
505         "iterations": int(center_result.nit),
506         "evaluations": int(center_result.nfev),
507         "best_sampled_duration_s": -float(center_result.fun),
508         "settings": center_settings,
509     },
510     "strict_differential_evolution": {
511         "success": bool(strict_result.success),
512         "message": str(strict_result.message),
513         "iterations": int(strict_result.nit),
514         "evaluations": int(strict_result.nfev),
515         "best_sampled_duration_s": -float(strict_result.fun),
516         "settings": strict_settings,
517     },
518     "independent_boundary_refinement": {
519         "success": bool(boundary_result.success),
520         "message": str(boundary_result.message),
521         "iterations": int(boundary_result.nit),
522         "evaluations": int(boundary_result.nfev),
523         "root_refined_duration_s": -float(boundary_result.fun),
524         "theta_rad": float(boundary_result.x[0]),
525         "fuse_delay_s": float(boundary_result.x[1]),
526         "settings": boundary_settings,
527     },
528     "local_refinements": [
529         {
530             "success": bool(result.success),
531             "message": str(result.message),
532             "evaluations": int(result.nfev),
533             "feasible": _feasibility_violation(_strategy_from_array(result.x), gravity)
534             <= 1e-12,
535             "root_refined_duration_s": (
536                 -float(result.fun) if float(result.fun) <= 0.0 else None
537             ),
538         }
539         for result in local_results
540     ],
541     "formal_evaluation": config["formal_evaluation"],
542     "feasible_candidate_count": len(feasible_candidates),
543     "formal_candidate_count": len(formal),
544     "convergence_claimed": bool(center_result.success and strict_result.success),
545     "status": (
546         "converged"
547         if center_result.success and strict_result.success
548         else "budget-exhausted-with-feasible-candidates"
549     ),
550 }
551 return formal, solver_record
552
553
554 def _summary_row(evaluation: Q2Evaluation, rank: int) -> dict[str, Any]:
555     strategy = evaluation.strategy
556     return {
557         "rank": rank,
558         "theta_rad": strategy.theta_rad,
559         "theta_deg": float(np.degrees(strategy.theta_rad) % 360.0),
560         "speed_m_s": strategy.speed_m_s,

```

```

561         "release_time_s": strategy.release_time_s,
562         "release_x_m": evaluation.release_point[0],
563         "release_y_m": evaluation.release_point[1],
564         "release_z_m": evaluation.release_point[2],
565         "fuse_delay_s": strategy.fuse_delay_s,
566         "explosion_time_s": strategy.explosion_time_s,
567         "explosion_x_m": evaluation.explosion_point[0],
568         "explosion_y_m": evaluation.explosion_point[1],
569         "explosion_z_m": evaluation.explosion_point[2],
570         "strict_duration_s": evaluation.strict_duration_s,
571         "center_duration_s": evaluation.center_duration_s,
572         "minimum_cloud_center_height_m": evaluation.minimum_cloud_center_height_m,
573         "cloud_center_below_ground": evaluation.cloud_center_below_ground,
574         "cloud_center_ground_crossing_time_s": (evaluation.cloud_center_ground_crossing_time_s),
575     }
576
577
578 def _strict_intervals_with_settings(
579     strategy: Q2Strategy,
580     gravity: float,
581     *,
582     time_scan_step: float,
583     rim_samples: int,
584     root_tolerance: float,
585 ) -> tuple[OcclusionInterval, ...]:
586     start, end = _effective_window(strategy)
587     return _find_intervals(
588         lambda time: _strict_margin(
589             time,
590             strategy,
591             gravity,
592             rim_samples=rim_samples,
593             refine_rim=True,
594         ),
595         start=start,
596         end=end,
597         scan_step=time_scan_step,
598         root_tolerance=root_tolerance,
599     )
600
601
602 def _q2_validation_record(best: Q2Evaluation, config: dict[str, Any]) -> dict[str, Any]:
603     """Build numerical, boundary, independent-geometry, and perturbation checks."""
604     gravity = float(config["gravity"])
605     validation = config["validation"]
606     root_tolerance = float(config["formal_evaluation"]["root_tolerance_s"])
607     convergence = []
608     for settings in validation["convergence_settings"]:
609         intervals = _strict_intervals_with_settings(
610             best.strategy,
611             gravity,
612             time_scan_step=float(settings["time_scan_step_s"]),
613             rim_samples=int(settings["rim_samples"]),
614             root_tolerance=root_tolerance,
615         )
616         convergence.append(
617             {
618                 **settings,
619                 "interval_count": len(intervals),
620                 "strict_duration_s": sum(interval.duration for interval in intervals),
621             }
622         )

```

```

623
624 boundary_checks = []
625 probe = float(validation["boundary_probe_s"])
626 for interval in best.strict_intervals:
627     midpoint = 0.5 * (interval.start + interval.end)
628     explosion_time = best.strategy.explosion_time_s
629     locations = [
630         ("pre-explosion", explosion_time - probe, "inactive"),
631         ("start", interval.start, "boundary"),
632         ("midpoint", midpoint, "inside"),
633         ("end", interval.end, "boundary"),
634         ("after-end", interval.end + probe, "outside"),
635     ]
636     if interval.start > explosion_time:
637         active_left_probe = interval.start - min(probe, 0.5 * (interval.start - explosion_time))
638         locations.insert(1, ("before-start", active_left_probe, "outside"))
639     for location, time, expected in locations:
640         active = (
641             best.strategy.explosion_time_s
642             <= time
643             <= (best.strategy.explosion_time_s + CLOUD_LIFETIME)
644         )
645         margin = (
646             _strict_margin(
647                 time,
648                 best.strategy,
649                 gravity,
650                 rim_samples=int(config["formal_evaluation"]["rim_samples"]),
651                 refine_rim=True,
652             )
653             if active
654             else None
655         )
656         boundary_checks.append(
657             {
658                 "location": location,
659                 "time_s": time,
660                 "expected_state": expected,
661                 "strict_margin_m": margin,
662                 "cloud_active": active,
663             }
664         )
665
666 surface_points = _dense_target_surface(
667     int(validation["surface_angles"]), int(validation["surface_levels"])
668 )
669 independent_checks = []
670 for boundary in boundary_checks:
671     time = float(boundary["time_s"])
672     if not boundary["cloud_active"]:
673         independent_checks.append(
674             {
675                 "location": boundary["location"],
676                 "time_s": time,
677                 "cloud_active": False,
678                 "occluded": False,
679             }
680         )
681     continue
682 maximum_distance, worst_point = _maximum_segment_distance(
683     surface_points,
684     missile_position(time),

```

```

685         cloud_position(time, best.strategy, gravity),
686     )
687     independent_checks.append(
688         {
689             "location": boundary["location"],
690             "time_s": time,
691             "cloud_active": True,
692             "sampled_surface_points": len(surface_points),
693             "maximum_segment_distance_m": maximum_distance,
694             "distance_margin_m": maximum_distance - CLOUD_RADIUS,
695             "occluded": maximum_distance <= CLOUD_RADIUS,
696             "worst_sampled_target_point": worst_point.tolist(),
697         }
698     )
699
700     perturbations = {
701         "theta-minus": (-float(validation["perturbation_theta_rad"]), 0.0, 0.0, 0.0),
702         "theta-plus": (float(validation["perturbation_theta_rad"]), 0.0, 0.0, 0.0),
703         "speed-plus": (0.0, float(validation["perturbation_speed_m_s"]), 0.0, 0.0),
704         "release-plus": (0.0, 0.0, float(validation["perturbation_release_time_s"]), 0.0),
705         "fuse-minus": (0.0, 0.0, 0.0, -float(validation["perturbation_fuse_delay_s"])),
706         "fuse-plus": (0.0, 0.0, 0.0, float(validation["perturbation_fuse_delay_s"])),
707     }
708     baseline = np.array(list(asdict(best.strategy).values()), dtype=float)
709     perturbation_checks = []
710     for label, offset in perturbations.items():
711         candidate = baseline + np.asarray(offset)
712         objective = _root_refined_objective(
713             candidate,
714             gravity,
715             time_step=float(validation["perturbation_time_step_s"]),
716             rim_samples=int(validation["perturbation_rim_samples"]),
717             root_tolerance=root_tolerance,
718         )
719         perturbation_checks.append(
720             {
721                 "perturbation": label,
722                 "strategy": asdict(_strategy_from_array(candidate)),
723                 "root_refined_strict_duration_s": -objective if objective <= 0.0 else 0.0,
724                 "feasible": _feasibility_violation(_strategy_from_array(candidate), gravity)
725                 <= 1e-12,
726             }
727         )
728
729     q1_baseline = _formal_evaluation(Q2Strategy(np.pi, 120.0, 1.5, 3.6), gravity, config)
730     return {
731         "convergence": convergence,
732         "boundary_checks": boundary_checks,
733         "independent_full_surface_segment_checks": independent_checks,
734         "local_perturbation_checks": perturbation_checks,
735         "q1_kernel_regression": {
736             "strategy": asdict(q1_baseline.strategy),
737             "strict_duration_s": q1_baseline.strict_duration_s,
738             "expected_q1_strict_duration_s": 1.3916426681980933,
739             "absolute_difference_s": abs(q1_baseline.strict_duration_s - 1.3916426681980933),
740         },
741     }
742
743
744 def run_question_2(project_root: Path, config: dict[str, Any]) -> list[Path]:
745     """Optimize Question 2 and write auditable tables and solver metadata."""
746     paths = prepare_output_paths(project_root / "outputs")

```

```

747 evaluations, solver_record = optimize_question_2(config)
748 if not evaluations:
749     raise RuntimeError("Question 2 optimization produced no feasible formal candidate")
750
751 near_count = min(int(config["near_optimal_count"]), len(evaluations))
752 candidate_rows = [
753     _summary_row(item, rank) for rank, item in enumerate(evaluations[:near_count], 1)
754 ]
755 summary_path = save_table(
756     pd.DataFrame(candidate_rows[:1]), paths.tables / "q2_summary.csv", overwrite=True
757 )
758 candidates_path = save_table(
759     pd.DataFrame(candidate_rows), paths.tables / "q2_candidates.csv", overwrite=True
760 )
761
762 interval_rows: list[dict[str, Any]] = []
763 for rank, evaluation in enumerate(evaluations[:near_count], 1):
764     for criterion, intervals in (
765         ("strict-full-cylinder", evaluation.strict_intervals),
766         ("center-line", evaluation.center_intervals),
767     ):
768         for number, interval in enumerate(intervals, 1):
769             interval_rows.append(
770                 {
771                     "rank": rank,
772                     "criterion": criterion,
773                     "interval": number,
774                     "start_s": interval.start,
775                     "end_s": interval.end,
776                     "duration_s": interval.duration,
777                 }
778             )
779 intervals_path = save_table(
780     pd.DataFrame(interval_rows), paths.tables / "q2_intervals.csv", overwrite=True
781 )
782
783 best = evaluations[0]
784 record = {
785     "model": "q2-two-stage-derivative-free-v1",
786     "objective": "maximize union duration of strict full-cylinder occlusion",
787     "gravity_m_s2": float(config["gravity"]),
788     "best": {
789         **_summary_row(best, 1),
790         "strict_intervals": [asdict(interval) for interval in best.strict_intervals],
791         "center_intervals": [asdict(interval) for interval in best.center_intervals],
792     },
793     "near_optimal_candidates": candidate_rows,
794     "solver": solver_record,
795     "ground_rule": (
796         "Cloud continues descending for 20 seconds as stated; no additional ground cutoff."
797     ),
798     "feasibility_checks": {
799         "speed_within_bounds": MINIMUM_SPEED <= best.strategy.speed_m_s <= MAXIMUM_SPEED,
800         "release_time_nonnegative": best.strategy.release_time_s >= 0.0,
801         "fuse_delay_nonnegative": best.strategy.fuse_delay_s >= 0.0,
802         "explosion_before_missile_hit": best.strategy.explosion_time_s <= missile_hit_time(),
803         "explosion_above_ground": best.explosion_point[2] >= 0.0,
804         "cloud_center_below_ground_during_effective_window": best.cloud_center_below_ground,
805     },
806 }
807 log_path = paths.logs / "q2_optimization.json"
808 log_path.write_text(json.dumps(record, ensure_ascii=False, indent=2) + "\n", encoding="utf-8")

```

```

809     validation_record = _q2_validation_record(best, config)
810     validation_path = paths.logs / "q2-validation.json"
811     validation_path.write_text(
812         json.dumps(validation_record, ensure_ascii=False, indent=2) + "\n",
813         encoding="utf-8",
814     )
815     return [summary_path, candidates_path, intervals_path, log_path, validation_path]

```

B.4 问题三程序

```

1  """Question 3 joint line-of-sight coverage by three smoke clouds."""
2
3  from __future__ import annotations
4
5  import json
6  from collections.abc import Callable, Sequence
7  from dataclasses import asdict, dataclass
8  from functools import lru_cache
9  from itertools import permutations
10 from pathlib import Path
11 from typing import Any
12
13 import numpy as np
14 import pandas as pd
15 from scipy.optimize import brentq, differential_evolution, minimize
16
17 from mmkit.artifacts import prepare_output_paths, save_table
18
19 from .q1 import (
20     CLOUD_DESCENT_SPEED,
21     CLOUD_LIFETIME,
22     CLOUD_RADIUS,
23     TARGET_HEIGHT,
24     TARGET_RADIUS,
25     UAV_INITIAL,
26     OcclusionInterval,
27     _dense_target_surface,
28     missile_position,
29 )
30 from .q2 import Q2Strategy, missile_hit_time
31 from .q2 import _formal_evaluation as q2_formal_evaluation
32 from .q2 import explosion_point as single_explosion_point
33 from .q2 import release_point as single_release_point
34
35
36 @dataclass(frozen=True, slots=True)
37 class Q3Strategy:
38     """Shared FY1 flight controls and three release/fuse controls."""
39
40     theta_rad: float
41     speed_m_s: float
42     s0_s: float
43     s1_s: float
44     s2_s: float
45     fuse1_s: float
46     fuse2_s: float
47     fuse3_s: float
48
49     @property
50     def release_times_s(self) -> tuple[float, float, float]:
51         """Return release times under the automatic one-second-gap parameterization."""

```



```

52         return (
53             self.s0_s,
54             self.s0_s + 1.0 + self.s1_s,
55             self.s0_s + 2.0 + self.s1_s + self.s2_s,
56         )
57
58     @property
59     def fuse_delays_s(self) -> tuple[float, float, float]:
60         """Return the three independently selected fuse delays."""
61         return (self.fuse1_s, self.fuse2_s, self.fuse3_s)
62
63     @property
64     def explosion_times_s(self) -> tuple[float, float, float]:
65         """Return explosion times; their ordering is deliberately unconstrained."""
66         return tuple(
67             release + fuse
68             for release, fuse in zip(self.release_times_s, self.fuse_delays_s, strict=True)
69         )
70
71
72 @dataclass(frozen=True, slots=True)
73 class BombGeometry:
74     """Derived geometry for one bomb."""
75
76     bomb: int
77     release_time_s: float
78     fuse_delay_s: float
79     explosion_time_s: float
80     release_point: tuple[float, float, float]
81     explosion_point: tuple[float, float, float]
82
83
84 @dataclass(frozen=True, slots=True)
85 class Q3Evaluation:
86     """Formal joint and conservative evaluation of one three-bomb strategy."""
87
88     strategy: Q3Strategy
89     bombs: tuple[BombGeometry, ...]
90     joint_intervals: tuple[OcclusionInterval, ...]
91     individual_strict_intervals: tuple[tuple[OcclusionInterval, ...], ...]
92     conservative_union_intervals: tuple[OcclusionInterval, ...]
93     deletion_durations_s: tuple[float, float, float]
94     ordered_incremental_durations_s: tuple[float, float, float]
95
96     @property
97     def joint_duration_s(self) -> float:
98         return _duration(self.joint_intervals)
99
100     @property
101     def conservative_duration_s(self) -> float:
102         return _duration(self.conservative_union_intervals)
103
104
105 @dataclass(frozen=True, slots=True)
106 class CoverageAssessment:
107     """Adaptive surface certificate or boundary-level unresolved assessment."""
108
109     status: str
110     signed_margin_m: float
111     sampled_lower_bound_m: float
112     lipschitz_upper_bound_m: float
113     surface_angles: int

```

```

114     surface_levels: int
115     refinement_trace: tuple[dict[str, float | int], ...]
116
117
118 def _strategy_from_array(values: np.ndarray) -> Q3Strategy:
119     return Q3Strategy(*(float(value) for value in values))
120
121
122 def _bomb_strategies(strategy: Q3Strategy) -> tuple[Q2Strategy, ...]:
123     return tuple(
124         Q2Strategy(
125             strategy.theta_rad,
126             strategy.speed_m_s,
127             release_time,
128             fuse,
129         )
130         for release_time, fuse in zip(strategy.release_times_s, strategy.fuse_delays_s, strict=True)
131     )
132
133
134 def _bomb_geometries(strategy: Q3Strategy, gravity: float) -> tuple[BombGeometry, ...]:
135     rows = []
136     for index, bomb in enumerate(_bomb_strategies(strategy), 1):
137         release = single_release_point(bomb)
138         explosion = single_explosion_point(bomb, gravity)
139         rows.append(
140             BombGeometry(
141                 bomb=index,
142                 release_time_s=bomb.release_time_s,
143                 fuse_delay_s=bomb.fuse_delay_s,
144                 explosion_time_s=bomb.explosion_time_s,
145                 release_point=tuple(float(value) for value in release),
146                 explosion_point=tuple(float(value) for value in explosion),
147             )
148         )
149     return tuple(rows)
150
151
152 def _feasibility_violation(strategy: Q3Strategy, gravity: float) -> float:
153     hit_time = missile_hit_time()
154     violation = 0.0
155     values = asdict(strategy).values()
156     if any(value < 0.0 for value in values):
157         violation += sum(max(0.0, -value) for value in values)
158     for bomb in _bomb_strategies(strategy):
159         violation += max(0.0, bomb.explosion_time_s - hit_time) / hit_time
160         height = float(single_explosion_point(bomb, gravity)[2])
161         violation += max(0.0, -height) / float(UAV_INITIAL[2])
162     return violation
163
164
165 def _cloud_center(time: float, bomb: BombGeometry) -> np.ndarray:
166     return np.asarray(bomb.explosion_point) + np.array(
167         [0.0, 0.0, -CLOUD_DESCENT_SPEED * (time - bomb.explosion_time_s)]
168     )
169
170
171 def _active_bombs(time: float, bombs: Sequence[BombGeometry]) -> tuple[BombGeometry, ...]:
172     return tuple(
173         bomb
174         for bomb in bombs
175         if bomb.explosion_time_s <= time <= bomb.explosion_time_s + CLOUD_LIFETIME

```

```

176     )
177
178
179 @lru_cache(maxsize=24)
180 def _surface_points(angle_count: int, level_count: int) -> np.ndarray:
181     points = _dense_target_surface(angle_count, level_count)
182     points.setflags(write=False)
183     return points
184
185
186 def _point_cloud_distances(
187     points: np.ndarray, missile: np.ndarray, cloud: np.ndarray
188 ) -> np.ndarray:
189     """Return cloud-center distances to missile-to-target-point segments."""
190     directions = points - missile
191     squared_lengths = np.einsum("ij,ij->i", directions, directions)
192     parameters = ((cloud - missile) @ directions.T) / squared_lengths
193     parameters = np.clip(parameters, 0.0, 1.0)
194     closest = missile + parameters[:, None] * directions
195     return np.linalg.norm(closest - cloud, axis=1)
196
197
198 def _coverage_margins(
199     points: np.ndarray,
200     missile: np.ndarray,
201     active_bombs: Sequence[BombGeometry],
202     time: float,
203 ) -> np.ndarray:
204     """Return pointwise best-cloud margins; nonpositive means jointly covered."""
205     if not active_bombs:
206         return np.full(len(points), np.inf)
207     distances = np.vstack(
208         [
209             _point_cloud_distances(points, missile, _cloud_center(time, bomb))
210             for bomb in active_bombs
211         ]
212     )
213     return np.min(distances, axis=0) - CLOUD_RADIUS
214
215
216 def _target_point(surface: str, first: float, second: float) -> np.ndarray:
217     if surface == "side":
218         angle, height = first, second
219         radius = TARGET_RADIUS
220         z = height
221     else:
222         angle, radius = first, second
223         z = 0.0 if surface == "bottom" else TARGET_HEIGHT
224     return np.array([radius * np.cos(angle), 200.0 + radius * np.sin(angle), z], dtype=float)
225
226
227 def _continuous_worst_margin(
228     missile: np.ndarray,
229     active: Sequence[BombGeometry],
230     time: float,
231     sampled_points: np.ndarray,
232     sampled_margins: np.ndarray,
233     *,
234     starts: int,
235     maxfev: int,
236 ) -> float:
237     """Refine the worst sampled ray continuously on side and both caps."""

```

```

238     block_size = len(sampled_points) // 3
239     order_parts = [
240         offset + np.argsort(sampled_margins[offset : offset + block_size])[-starts:]
241         for offset in (0, block_size, 2 * block_size)
242     ]
243     order = np.concatenate(order_parts)
244     seeds: list[tuple[str, np.ndarray]] = []
245     for index in order:
246         point = sampled_points[int(index)]
247         angle = float(np.arctan2(point[1] - 200.0, point[0]) % (2.0 * np.pi))
248         radial = float(np.clip(np.hypot(point[0], point[1] - 200.0), 0.0, TARGET_RADIUS))
249         seeds.append(("side", np.array([angle, float(np.clip(point[2], 0, TARGET_HEIGHT))])))
250         if abs(point[2]) < 1e-9:
251             seeds.append(("bottom", np.array([angle, radial])))
252         if abs(point[2] - TARGET_HEIGHT) < 1e-9:
253             seeds.append(("top", np.array([angle, radial])))
254
255     best = float(np.max(sampled_margins))
256     for surface, seed in seeds:
257         upper_second = TARGET_HEIGHT if surface == "side" else TARGET_RADIUS
258
259         def negative_margin(parameters: np.ndarray, fixed_surface: str = surface) -> float:
260             point = _target_point(fixed_surface, float(parameters[0]), float(parameters[1]))
261             margin = _coverage_margins(point[None, :], missile, active, time)[0]
262             return -float(margin)
263
264         result = minimize(
265             negative_margin,
266             seed,
267             method="Nelder-Mead",
268             bounds=[(0.0, 2.0 * np.pi), (0.0, upper_second)],
269             options={"maxfev": maxfev, "xatol": 1e-9, "fatol": 1e-9},
270         )
271         best = max(best, -float(result.fun))
272     return best
273
274
275 def _surface_covering_radius(angle_count: int, level_count: int) -> float:
276     """Return a Euclidean covering-radius bound for the sampled cylinder surface."""
277     angular_half_chord = 2.0 * TARGET_RADIUS * np.sin(np.pi / (2.0 * angle_count))
278     linear_half_step = max(TARGET_HEIGHT, TARGET_RADIUS) / (2.0 * (level_count - 1))
279     return float(np.hypot(angular_half_chord, linear_half_step))
280
281
282 def joint_coverage_margin(
283     time: float,
284     bombs: Sequence[BombGeometry],
285     *,
286     surface_angles: int,
287     surface_levels: int,
288     continuous_refinement: bool,
289     refinement_starts: int = 0,
290     refinement_maxfev: int = 0,
291     adaptive_max_surface_angles: int | None = None,
292     adaptive_max_surface_levels: int | None = None,
293 ) -> float:
294     """Return max-over-rays min-over-active-clouds coverage margin."""
295     return joint_coverage_assessment(
296         time,
297         bombs,
298         surface_angles=surface_angles,
299         surface_levels=surface_levels,

```

```

300         continuous_refinement=continuous_refinement,
301         refinement_starts=refinement_starts,
302         refinement_maxfev=refinement_maxfev,
303         adaptive_max_surface_angles=adaptive_max_surface_angles,
304         adaptive_max_surface_levels=adaptive_max_surface_levels,
305     ).signed_margin_m
306
307
308 def joint_coverage_assessment(
309     time: float,
310     bombs: Sequence[BombGeometry],
311     *,
312     surface_angles: int,
313     surface_levels: int,
314     continuous_refinement: bool,
315     refinement_starts: int = 0,
316     refinement_maxfev: int = 0,
317     adaptive_max_surface_angles: int | None = None,
318     adaptive_max_surface_levels: int | None = None,
319 ) -> CoverageAssessment:
320     """Adaptively certify covered/uncovered rays using a Lipschitz grid bound."""
321     active = _active_bombs(time, bombs)
322     if not active:
323         return CoverageAssessment(
324             "uncovered-no-active-cloud",
325             float("inf"),
326             float("inf"),
327             float("inf"),
328             surface_angles,
329             surface_levels,
330             (),
331         )
332     missile = missile_position(time)
333     max_angles = adaptive_max_surface_angles or surface_angles
334     max_levels = adaptive_max_surface_levels or surface_levels
335     current_angles = surface_angles
336     current_levels = surface_levels
337     trace: list[dict[str, float | int]] = []
338     while True:
339         points = _surface_points(current_angles, current_levels)
340         margins = _coverage_margins(points, missile, active, time)
341         sampled_maximum = float(np.max(margins))
342         covering_radius = _surface_covering_radius(current_angles, current_levels)
343         upper_bound = sampled_maximum + covering_radius
344         trace.append(
345             {
346                 "surface_angles": current_angles,
347                 "surface_levels": current_levels,
348                 "covering_radius_m": covering_radius,
349                 "sampled_lower_bound_m": sampled_maximum,
350                 "lipschitz_upper_bound_m": upper_bound,
351             }
352         )
353     if not continuous_refinement or sampled_maximum > 0.0:
354         status = "uncovered-certified" if sampled_maximum > 0.0 else "sampled-covered"
355         return CoverageAssessment(
356             status,
357             sampled_maximum,
358             sampled_maximum,
359             upper_bound,
360             current_angles,
361             current_levels,

```

```

362         tuple(trace),
363     )
364     if upper_bound <= 0.0:
365         return CoverageAssessment(
366             "covered-certified",
367             upper_bound,
368             sampled_maximum,
369             upper_bound,
370             current_angles,
371             current_levels,
372             tuple(trace),
373         )
374     if current_angles >= max_angles and current_levels >= max_levels:
375         break
376     current_angles = min(max_angles, current_angles * 2)
377     current_levels = min(max_levels, 2 * (current_levels - 1) + 1)
378
379     local_margin = _continuous_worst_margin(
380         missile,
381         active,
382         time,
383         points,
384         margins,
385         starts=refinement_starts,
386         maxfev=refinement_maxfev,
387     )
388     status = "uncovered-certified-local-witness" if local_margin > 0.0 else "boundary-uncertain"
389     return CoverageAssessment(
390         status,
391         local_margin,
392         sampled_maximum,
393         upper_bound,
394         current_angles,
395         current_levels,
396         tuple(trace),
397     )
398
399
400 def _merge_intervals(intervals: Sequence[OcclusionInterval]) -> tuple[OcclusionInterval, ...]:
401     if not intervals:
402         return ()
403     ordered = sorted(intervals, key=lambda item: item.start)
404     merged = [ordered[0]]
405     for interval in ordered[1:]:
406         previous = merged[-1]
407         if interval.start <= previous.end + 1e-9:
408             merged[-1] = OcclusionInterval(previous.start, max(previous.end, interval.end))
409         else:
410             merged.append(interval)
411     return tuple(merged)
412
413
414 def _duration(intervals: Sequence[OcclusionInterval]) -> float:
415     return sum(interval.duration for interval in intervals)
416
417
418 def _find_joint_intervals(
419     margin: Callable[[float], float],
420     bombs: Sequence[BombGeometry],
421     *,
422     scan_step: float,
423     root_tolerance: float,

```

```

424 ) -> tuple[OcclusionInterval, ...]:
425     """Find union intervals, preserving cloud activation/deactivation breakpoints."""
426     start = min(bomb.explosion_time_s for bomb in bombs)
427     end = min(missile_hit_time(), max(bomb.explosion_time_s + CLOUD_LIFETIME for bomb in bombs))
428     regular = np.linspace(start, end, int(np.ceil((end - start) / scan_step)) + 1)
429     events = [
430         event
431         for bomb in bombs
432         for event in (bomb.explosion_time_s, bomb.explosion_time_s + CLOUD_LIFETIME)
433         if start <= event <= end
434     ]
435     event_probes = []
436     probe = max(root_tolerance * 10.0, scan_step * 1e-5)
437     for event in events:
438         event_probes.extend((max(start, event - probe), min(end, event + probe)))
439     times = np.unique(np.concatenate((regular, np.asarray(events), np.asarray(event_probes))))
440     margins = np.array([margin(float(time)) for time in times])
441     inside = margins <= 0.0
442     intervals: list[OcclusionInterval] = []
443     current = float(times[0]) if inside[0] else None
444     for index in np.flatnonzero(inside[1:] != inside[:-1]):
445         left, right = float(times[index]), float(times[index + 1])
446         midpoint = 0.5 * (left + right)
447         left_active = tuple(bomb.bomb for bomb in _active_bombs(left, bombs))
448         middle_active = tuple(bomb.bomb for bomb in _active_bombs(midpoint, bombs))
449         right_active = tuple(bomb.bomb for bomb in _active_bombs(right, bombs))
450         if left_active != middle_active and middle_active == right_active:
451             boundary = left
452         elif left_active == middle_active and middle_active != right_active:
453             boundary = right
454         else:
455             boundary = float(brentq(margin, left, right, xtol=root_tolerance, rtol=1e-14))
456         if inside[index + 1]:
457             current = boundary
458         elif current is not None:
459             intervals.append(OcclusionInterval(current, boundary))
460             current = None
461     if current is not None:
462         intervals.append(OcclusionInterval(current, float(times[-1])))
463     return _merge_intervals(intervals)
464
465
466 def _settings_margin(
467     bombs: Sequence[BombGeometry], settings: dict[str, Any], *, refine: bool
468 ) -> Callable[[float], float]:
469     return lambda time: joint_coverage_margin(
470         time,
471         bombs,
472         surface_angles=int(settings["surface_angles"]),
473         surface_levels=int(settings["surface_levels"]),
474         continuous_refinement=refine,
475         refinement_starts=int(settings.get("continuous_refinement_starts", 0)),
476         refinement_maxfev=int(settings.get("continuous_refinement_maxfev", 0)),
477         adaptive_max_surface_angles=int(
478             settings.get("adaptive_max_surface_angles", settings["surface_angles"])
479         ),
480         adaptive_max_surface_levels=int(
481             settings.get("adaptive_max_surface_levels", settings["surface_levels"])
482         ),
483     )
484
485

```

```

486 def _sampled_metrics(
487     bombs: Sequence[BombGeometry], margin: Callable[[float], float], time_step: float
488 ) -> tuple[float, float]:
489     start = min(bomb.explosion_time_s for bomb in bombs)
490     end = min(missile_hit_time(), max(bomb.explosion_time_s + CLOUD_LIFETIME for bomb in bombs))
491     anchored = np.arange(0.0, missile_hit_time() + 0.5 * time_step, time_step)
492     anchored = anchored[(anchored >= start) & (anchored <= end)]
493     events = np.array(
494         [
495             event
496             for bomb in bombs
497             for event in (bomb.explosion_time_s, bomb.explosion_time_s + CLOUD_LIFETIME)
498             if start <= event <= end
499         ]
500     )
501     times = np.unique(np.concatenate([start, end], anchored, events))
502     margins = np.array([margin(float(time)) for time in times])
503     inside = (margins <= 0.0).astype(float)
504     return float(np.trapezoid(inside, times)), float(np.min(margins))
505
506
507 def _objective(
508     values: np.ndarray, gravity: float, settings: dict[str, Any], *, roots: bool
509 ) -> float:
510     strategy = _strategy_from_array(values)
511     violation = _feasibility_violation(strategy, gravity)
512     if violation > 0.0:
513         return 100.0 + 100.0 * violation
514     bombs = _bomb_geometries(strategy, gravity)
515     margin = _settings_margin(bombs, settings, refine=False)
516     duration, minimum = _sampled_metrics(bombs, margin, float(settings["time_step_s"]))
517     if duration <= 0.0:
518         return max(minimum, 0.0) / 100.0
519     if not roots:
520         return -duration
521     intervals = _find_joint_intervals(
522         margin,
523         bombs,
524         scan_step=float(settings["time_step_s"]),
525         root_tolerance=1e-8,
526     )
527     return -_duration(intervals)
528
529
530 def _bounds(config: dict[str, Any]) -> list[tuple[float, float]]:
531     bounds = config["bounds"]
532     fuse = tuple(bounds["fuse_delay_s"])
533     return [
534         tuple(bounds["theta_rad"]),
535         tuple(bounds["speed_m_s"]),
536         tuple(bounds["s0_s"]),
537         tuple(bounds["s1_s"]),
538         tuple(bounds["s2_s"]),
539         fuse,
540         fuse,
541         fuse,
542     ]
543
544
545 def _warm_vector(config: dict[str, Any]) -> np.ndarray:
546     warm = config["q2_warm_start"]
547     return np.array(

```



```

548     [
549         warm["theta_rad"],
550         warm["speed_m_s"],
551         warm["s0_s"],
552         warm["s1_s"],
553         warm["s2_s"],
554         *warm["fuse_delays_s"],
555     ],
556     dtype=float,
557 )
558
559
560 def _initial_population(config: dict[str, Any]) -> np.ndarray:
561     bounds = _bounds(config)
562     settings = config["global_search"]
563     rng = np.random.default_rng(int(config["seed"]))
564     count = int(settings["popsize"]) * len(bounds)
565     lower = np.array([item[0] for item in bounds])
566     upper = np.array([item[1] for item in bounds])
567     population = rng.uniform(lower, upper, size=(count, len(bounds)))
568     warm = _warm_vector(config)
569     structured = []
570     fuse_families = (
571         (2.4968, 2.7, 0.5),
572         (2.4968, 1.5, 0.5),
573         (2.4968, 3.5, 4.5),
574         (2.4968, 2.7, 2.0),
575     )
576     for speed in (70.0, 90.0, 110.0, 140.0):
577         for theta_offset, fuses in zip((-0.03, -0.01, 0.01, 0.03), fuse_families, strict=True):
578             structured.append(
579                 np.array([warm[0] + theta_offset, speed, 0.0, 0.0, 0.0, *fuses], dtype=float)
580             )
581     structured.insert(0, warm)
582     for index, candidate in enumerate(structured[: len(population)]):
583         population[index] = np.clip(candidate, lower, upper)
584     return population
585
586
587 def _deduplicate(values: Sequence[np.ndarray]) -> list[np.ndarray]:
588     unique: list[np.ndarray] = []
589     for value in values:
590         if not any(np.linalg.norm(value - existing) < 1e-7 for existing in unique):
591             unique.append(np.asarray(value, dtype=float))
592     return unique
593
594
595 def _individual_strict_intervals(
596     strategy: Q3Strategy, gravity: float, config: dict[str, Any]
597 ) -> tuple[tuple[OcclusionInterval, ...], ...]:
598     q2_config = {
599         "formal_evaluation": {
600             "time_scan_step_s": config["formal_evaluation"]["time_scan_step_s"],
601             "root_tolerance_s": config["formal_evaluation"]["root_tolerance_s"],
602             "rim_samples": 2048,
603         }
604     }
605     return tuple(
606         q2_formal_evaluation(bomb, gravity, q2_config).strict_intervals
607         for bomb in _bomb_strategies(strategy)
608     )
609

```

```

610
611 def _formal_evaluation(strategy: Q3Strategy, config: dict[str, Any]) -> Q3Evaluation:
612     gravity = float(config["gravity"])
613     bombs = _bomb_geometries(strategy, gravity)
614     settings = config["formal_evaluation"]
615     margin = _settings_margin(bombs, settings, refine=True)
616     joint = _find_joint_intervals(
617         margin,
618         bombs,
619         scan_step=float(settings["time_scan_step_s"]),
620         root_tolerance=float(settings["root_tolerance_s"]),
621     )
622     individual = _individual_strict_intervals(strategy, gravity, config)
623     conservative = _merge_intervals([interval for group in individual for interval in group])
624     deletion = []
625     for removed in range(3):
626         remaining = tuple(bomb for index, bomb in enumerate(bombs) if index != removed)
627         remaining_margin = _settings_margin(remaining, settings, refine=False)
628         intervals = _find_joint_intervals(
629             remaining_margin,
630             remaining,
631             scan_step=float(settings["time_scan_step_s"]),
632             root_tolerance=float(settings["root_tolerance_s"]),
633         )
634         deletion.append(min(_duration(intervals), _duration(joint)))
635     first_duration = _duration(individual[0])
636     first_two = bombs[:2]
637     first_two_intervals = _find_joint_intervals(
638         _settings_margin(first_two, settings, refine=True),
639         first_two,
640         scan_step=float(settings["time_scan_step_s"]),
641         root_tolerance=float(settings["root_tolerance_s"]),
642     )
643     first_two_duration = _duration(first_two_intervals)
644     incremental = (
645         first_duration,
646         first_two_duration - first_duration,
647         _duration(joint) - first_two_duration,
648     )
649     return Q3Evaluation(
650         strategy=strategy,
651         bombs=bombs,
652         joint_intervals=joint,
653         individual_strict_intervals=individual,
654         conservative_union_intervals=conservative,
655         deletion_durations_s=tuple(deletion),
656         ordered_incremental_durations_s=incremental,
657     )
658
659
660 def optimize_question_3(
661     config: dict[str, Any], *, stage_log_path: Path | None = None
662 ) -> tuple[list[Q3Evaluation], dict[str, Any]]:
663     """Run seeded eight-dimensional global search and multi-start local refinement."""
664     gravity = float(config["gravity"])
665     global_settings = config["global_search"]
666     result = differential_evolution(
667         lambda values: _objective(values, gravity, global_settings, roots=False),
668         _bounds(config),
669         seed=int(config["seed"]),
670         init=_initial_population(config),
671         maxiter=int(global_settings["maxiter"]),

```

```

672     popsize=int(global_settings["popsize"]),
673     tol=float(global_settings["tol"]),
674     polish=bool(global_settings["polish"]),
675     workers=1,
676 )
677 print(
678     "Q3 global search complete; "
679     f"evaluations={result.nfev}; sampled_duration={max(0.0, -float(result.fun)):.6f}s"
680 )
681 order = np.argsort(result.population_energies)
682 persistent_incumbent = np.asarray(config["persistent_incumbent"]["vector"], dtype=float)
683 candidates = [result.x, _warm_vector(config), persistent_incumbent]
684 candidates.extend(np.asarray(result.population)[order[:10]])
685 if stage_log_path is not None:
686     stage_log_path.write_text(
687         json.dumps(
688             {
689                 "stage": "global-search-complete",
690                 "seed": int(config["seed"]),
691                 "success": bool(result.success),
692                 "message": str(result.message),
693                 "evaluations": int(result.nfev),
694                 "sampled_duration_s": max(0.0, -float(result.fun)),
695                 "ranked_candidate_vectors": [
696                     np.asarray(item, dtype=float).tolist() for item in candidates
697                 ],
698             },
699             ensure_ascii=False,
700             indent=2,
701         )
702         + "\n",
703         encoding="utf-8",
704     )
705 local_results = []
706 local_settings = config["local_refinement"]
707 for start in _deduplicate(candidates)[:int(local_settings["starts"])]:
708     local = minimize(
709         lambda values: _objective(values, gravity, local_settings, roots=True),
710         start,
711         method="Nelder-Mead",
712         bounds=_bounds(config),
713         options={
714             "maxfev": int(local_settings["maxfev"]),
715             "xatol": float(local_settings["xatol"]),
716             "fatol": float(local_settings["fatol"]),
717         },
718     )
719 local_results.append(local)
720 candidates.append(local.x)
721 print(
722     "Q3 local refinement complete; "
723     f"start={len(local_results)}; evaluations={local.nfev}; "
724     f"duration={max(0.0, -float(local.fun)):.6f}s"
725 )
726 if stage_log_path is not None:
727     stage_log_path.write_text(
728         json.dumps(
729             {
730                 "stage": "local-refinement-in-progress",
731                 "completed_starts": len(local_results),
732                 "planned_starts": int(local_settings["starts"]),
733                 "candidate_vectors": [

```

```

734         np.asarray(item, dtype=float).tolist() for item in candidates
735     ],
736     "local_results": [
737         {
738             "success": bool(item.success),
739             "message": str(item.message),
740             "evaluations": int(item.nfev),
741             "duration_s": max(0.0, -float(item.fun)),
742             "vector": np.asarray(item.x, dtype=float).tolist(),
743         }
744         for item in local_results
745     ],
746 },
747 ensure_ascii=False,
748 indent=2,
749 )
750 + "\n",
751 encoding="utf-8",
752 )
753 feasible = [
754     item
755     for item in _deduplicate(candidates)
756     if _feasibility_violation(_strategy_from_array(item), gravity) <= 1e-12
757 ]
758 ranked = sorted(
759     feasible,
760     key=lambda values: _objective(values, gravity, local_settings, roots=True),
761 )
762 count = min(int(config["formal_evaluation"]["candidate_count"]), len(ranked))
763 formal = []
764 for index, item in enumerate(ranked[:count], 1):
765     formal.append(_formal_evaluation(_strategy_from_array(item), config))
766     print(f"Q3 formal candidate complete; candidate={index}/{count}")
767 formal.sort(key=lambda item: item.joint_duration_s, reverse=True)
768 if stage_log_path is not None:
769     stage_log_path.write_text(
770         json.dumps(
771             {
772                 "stage": "search-complete",
773                 "seed": int(config["seed"]),
774                 "global_success": bool(result.success),
775                 "global_message": str(result.message),
776                 "global_evaluations": int(result.nfev),
777                 "completed_local_starts": len(local_results),
778                 "formal_candidate_count": len(formal),
779                 "formal_candidates": [
780                     _strategy_summary(item, rank) for rank, item in enumerate(formal, 1)
781                 ],
782                 "best_formal_joint_duration_s": (
783                     formal[0].joint_duration_s if formal else None
784                 ),
785                 "sampled_time_grid": "absolute task time t=0",
786             },
787             ensure_ascii=False,
788             indent=2,
789         )
790         + "\n",
791         encoding="utf-8",
792     )
793 log = {
794     "seed": int(config["seed"]),
795     "variable_order": [

```

```

796         "theta_rad",
797         "speed_m_s",
798         "s0_s",
799         "s1_s",
800         "s2_s",
801         "fuse1_s",
802         "fuse2_s",
803         "fuse3_s",
804     ],
805     "global_search": {
806         "success": bool(result.success),
807         "message": str(result.message),
808         "iterations": int(result.nit),
809         "evaluations": int(result.nfev),
810         "sampled_joint_duration_s": -float(result.fun),
811         "settings": global_settings,
812     },
813     "candidate_initialization": {
814         "mechanism": (
815             "Q2 anchor plus structured partial-edge-complement seeds over four speeds, "
816             "four heading offsets, and simultaneous/overlapping/staggered fuse families"
817         ),
818         "interpretation": (
819             "later bombs target partial view-domain gaps; no single-cloud relay is assumed"
820         ),
821         "structured_seed_count": 17,
822         "persistent_incumbent": config["persistent_incumbent"],
823     },
824     "sampled_time_grid": {
825         "anchor": "absolute task time t=0",
826         "included_points": "fixed-step grid plus window endpoints and cloud events",
827         "invariance": (
828             "inactive late clouds cannot shift the sampling phase of earlier coverage"
829         ),
830     },
831     "corrected_full_rerun": True,
832     "local_refinements": [
833         {
834             "success": bool(item.success),
835             "message": str(item.message),
836             "evaluations": int(item.nfev),
837             "root_refined_duration_s": -float(item.fun) if item.fun <= 0 else None,
838         }
839         for item in local_results
840     ],
841     "formal_settings": config["formal_evaluation"],
842     "status": "converged" if result.success else "budget-exhausted-with-feasible-candidates",
843     "convergence_claimed": bool(result.success),
844     "feasible_candidate_count": len(feasible),
845 }
846 return formal, log
847
848
849 def _strategy_summary(evaluation: Q3Evaluation, rank: int) -> dict[str, Any]:
850     strategy = evaluation.strategy
851     individual_total = sum(
852         _duration(intervals) for intervals in evaluation.individual_strict_intervals
853     )
854     return {
855         "rank": rank,
856         "theta_rad": strategy.theta_rad,
857         "theta_deg": float(np.degrees(strategy.theta_rad) % 360.0),

```

```

858     "speed_m_s": strategy.speed_m_s,
859     "release_times_s": json.dumps(strategy.release_times_s),
860     "fuse_delays_s": json.dumps(strategy.fuse_delays_s),
861     "explosion_times_s": json.dumps(strategy.explosion_times_s),
862     "joint_duration_s": evaluation.joint_duration_s,
863     "conservative_duration_s": evaluation.conservative_duration_s,
864     "joint_synergy_s": evaluation.joint_duration_s - evaluation.conservative_duration_s,
865     "sum_individual_strict_duration_s": individual_total,
866     "individual_strict_overlap_s": individual_total - evaluation.conservative_duration_s,
867 }
868
869
870 def _bomb_rows(evaluation: Q3Evaluation) -> list[dict[str, Any]]:
871     rows = []
872     individual_durations = [_duration(group) for group in evaluation.individual_strict_intervals]
873     for bomb, individual, deleted, incremental in zip(
874         evaluation.bombs,
875         individual_durations,
876         evaluation.deletion_durations_s,
877         evaluation.ordered_incremental_durations_s,
878         strict=True,
879     ):
880         rows.append(
881             {
882                 "schema_status": "provisional-schema; official result1.xlsx unavailable",
883                 "drone": "FY1",
884                 "bomb": bomb.bomb,
885                 "theta_rad": evaluation.strategy.theta_rad,
886                 "theta_deg": float(np.degrees(evaluation.strategy.theta_rad) % 360.0),
887                 "speed_m_s": evaluation.strategy.speed_m_s,
888                 "release_time_s": bomb.release_time_s,
889                 "release_x_m": bomb.release_point[0],
890                 "release_y_m": bomb.release_point[1],
891                 "release_z_m": bomb.release_point[2],
892                 "fuse_delay_s": bomb.fuse_delay_s,
893                 "explosion_time_s": bomb.explosion_time_s,
894                 "explosion_x_m": bomb.explosion_point[0],
895                 "explosion_y_m": bomb.explosion_point[1],
896                 "explosion_z_m": bomb.explosion_point[2],
897                 "individual_strict_duration_s": individual,
898                 "joint_deletion_loss_s": evaluation.joint_duration_s - deleted,
899                 "ordered_incremental_contribution_s": incremental,
900                 "minimum_cloud_center_height_m": bomb.explosion_point[2]
901                 - CLOUD_DESCENT_SPEED * CLOUD_LIFETIME,
902             }
903         )
904     return rows
905
906
907 def _independent_ray_intersections(
908     points: np.ndarray, missile: np.ndarray, clouds: Sequence[np.ndarray]
909 ) -> np.ndarray:
910     """Independently test segment-sphere intersections through quadratic roots."""
911     directions = points - missile
912     lengths = np.linalg.norm(directions, axis=1)
913     unit_directions = directions / lengths[:, None]
914     covered = np.zeros(len(points), dtype=bool)
915     for cloud in clouds:
916         offset = missile - cloud
917         b = 2.0 * (unit_directions @ offset)
918         c = float(offset @ offset) - CLOUD_RADIUS**2
919         discriminant = b**2 - 4.0 * c

```

```

920     real = discriminant >= 0.0
921     root = np.sqrt(np.maximum(discriminant, 0.0))
922     lower = (-b - root) / 2.0
923     upper = (-b + root) / 2.0
924     covered |= real & (upper >= 0.0) & (lower <= lengths)
925 return covered
926
927
928 def _validation_record(evaluation: Q3Evaluation, config: dict[str, Any]) -> dict[str, Any]:
929     gravity = float(config["gravity"])
930     validation = config["validation"]
931     convergence = []
932     convergence_settings = validation["convergence_settings"]
933     for index, settings in enumerate(convergence_settings):
934         use_refinement = index == len(convergence_settings) - 1
935         merged = {
936             **settings,
937             "continuous_refinement_starts": 1,
938             "continuous_refinement_maxfev": 50,
939             "adaptive_max_surface_angles": (
940                 int(config["formal_evaluation"]["adaptive_max_surface_angles"])
941                 if use_refinement
942                 else int(settings["surface_angles"])
943             ),
944             "adaptive_max_surface_levels": (
945                 int(config["formal_evaluation"]["adaptive_max_surface_levels"])
946                 if use_refinement
947                 else int(settings["surface_levels"])
948             ),
949         }
950         margin = _settings_margin(evaluation.bombs, merged, refine=use_refinement)
951         intervals = _find_joint_intervals(
952             margin,
953             evaluation.bombs,
954             scan_step=float(settings["time_scan_step_s"]),
955             root_tolerance=float(config["formal_evaluation"]["root_tolerance_s"]),
956         )
957         convergence.append(
958             {
959                 **settings,
960                 "continuous_surface_refinement": use_refinement,
961                 "joint_duration_s": _duration(intervals),
962             }
963         )
964
965     boundary = []
966     direct = []
967     probe = float(validation["boundary_probe_s"])
968     points = _surface_points(
969         int(validation["direct_surface_angles"]), int(validation["direct_surface_levels"])
970     )
971     for interval in evaluation.joint_intervals:
972         for location, time in (
973             ("before-start", interval.start - probe),
974             ("start", interval.start),
975             ("midpoint", 0.5 * (interval.start + interval.end)),
976             ("end", interval.end),
977             ("after-end", interval.end + probe),
978         ):
979             assessment = joint_coverage_assessment(
980                 time,
981                 evaluation.bombs,

```

```

982         surface_angles=int(config["formal_evaluation"]["surface_angles"]),
983         surface_levels=int(config["formal_evaluation"]["surface_levels"]),
984         continuous_refinement=True,
985         refinement_starts=int(config["formal_evaluation"]["continuous_refinement_starts"]),
986         refinement_maxfev=int(config["formal_evaluation"]["continuous_refinement_maxfev"]),
987         adaptive_max_surface_angles=int(
988             config["formal_evaluation"]["adaptive_max_surface_angles"]
989         ),
990         adaptive_max_surface_levels=int(
991             config["formal_evaluation"]["adaptive_max_surface_levels"]
992         ),
993     )
994     active = _active_bombs(time, evaluation.bombs)
995     covered = _independent_ray_intersections(
996         points,
997         missile_position(time),
998         [_cloud_center(time, bomb) for bomb in active],
999     )
1000     boundary.append(
1001         {
1002             "location": location,
1003             "time_s": time,
1004             "coverage_status": assessment.status,
1005             "worst_margin_m": assessment.signed_margin_m,
1006             "sampled_lower_bound_m": assessment.sampled_lower_bound_m,
1007             "lipschitz_upper_bound_m": assessment.lipschitz_upper_bound_m,
1008             "surface_angles": assessment.surface_angles,
1009             "surface_levels": assessment.surface_levels,
1010             "adaptive_trace": assessment.refinement_trace,
1011         }
1012     )
1013     direct.append(
1014         {
1015             "location": location,
1016             "time_s": time,
1017             "sampled_rays": len(points),
1018             "uncovered_rays": int(np.count_nonzero(~covered)),
1019             "all_covered": bool(np.all(covered)),
1020         }
1021     )
1022
1023     q2 = config["q2_warm_start"]
1024     q2_strategy = Q2Strategy(q2["theta_rad"], q2["speed_m_s"], 0.0, q2["fuse_delays_s"][0])
1025     q2_config = {
1026         "formal_evaluation": {
1027             "time_scan_step_s": config["formal_evaluation"]["time_scan_step_s"],
1028             "root_tolerance_s": config["formal_evaluation"]["root_tolerance_s"],
1029             "rim_samples": 2048,
1030         }
1031     }
1032
1033     q2_reference = q2_formal_evaluation(q2_strategy, gravity, q2_config)
1034     single_bomb = _bomb_geometries(
1035         Q3Strategy(
1036             q2_strategy.theta_rad, q2_strategy.speed_m_s, 0, 0, 0, q2_strategy.fuse_delay_s, 19, 19
1037         ),
1038         gravity,
1039     )[0]
1040     single_settings = config["formal_evaluation"]
1041     single_intervals = _find_joint_intervals(
1042         _settings_margin((single_bomb,), single_settings, refine=True),
1043         (single_bomb,),

```



```

1044     scan_step=float(single_settings["time_scan_step_s"]),
1045     root_tolerance=float(single_settings["root_tolerance_s"]),
1046 )
1047 permutation_durations = []
1048 permutation_settings = {
1049     **validation["convergence_settings"][1],
1050     "continuous_refinement_starts": 0,
1051     "continuous_refinement_maxfev": 0,
1052 }
1053 for order in permutations(evaluation.bombs):
1054     intervals = _find_joint_intervals(
1055         _settings_margin(order, permutation_settings, refine=False),
1056         order,
1057         scan_step=float(permutation_settings["time_scan_step_s"]),
1058         root_tolerance=float(config["formal_evaluation"]["root_tolerance_s"]),
1059     )
1060     permutation_durations.append(_duration(intervals))
1061
1062 complement_time = 4.768
1063 reference_cloud = np.array([17625.525268056128, 10.240444866222791, 1762.640057120904])
1064 complement_bombs = tuple(
1065     BombGeometry(
1066         bomb=index,
1067         release_time_s=complement_time,
1068         fuse_delay_s=0.0,
1069         explosion_time_s=complement_time,
1070         release_point=tuple(reference_cloud + np.array([0.0, sign * 9.65, 0.0])),
1071         explosion_point=tuple(reference_cloud + np.array([0.0, sign * 9.65, 0.0])),
1072     )
1073     for index, sign in enumerate((-1.0, 1.0), 1)
1074 )
1075 complement_settings = {
1076     "surface_angles": 360,
1077     "surface_levels": 41,
1078     "continuous_refinement_starts": 8,
1079     "continuous_refinement_maxfev": 100,
1080 }
1081 complement_individual = [
1082     joint_coverage_margin(
1083         complement_time,
1084         (bomb,),
1085         surface_angles=360,
1086         surface_levels=41,
1087         continuous_refinement=True,
1088         refinement_starts=8,
1089         refinement_maxfev=100,
1090     )
1091     for bomb in complement_bombs
1092 ]
1093 complement_joint = _settings_margin(complement_bombs, complement_settings, refine=True)(
1094     complement_time
1095 )
1096 return {
1097     "feasibility": {
1098         "violation": _feasibility_violation(evaluation.strategy, gravity),
1099         "release_gaps_s": np.diff(evaluation.strategy.release_times_s).tolist(),
1100         "explosion_times_s": list(evaluation.strategy.explosion_times_s),
1101         "explosion_heights_m": [bomb.explosion_point[2] for bomb in evaluation.bombs],
1102         "cloud_minimum_heights_m": [
1103             bomb.explosion_point[2] - CLOUD_DESCENT_SPEED * CLOUD_LIFETIME
1104             for bomb in evaluation.bombs
1105         ],
1106     },

```

```

1106     },
1107     "surface_time_convergence": convergence,
1108     "boundary_checks": boundary,
1109     "independent_quadratic_ray_checks": direct,
1110     "label_permutation_durations_s": permutation_durations,
1111     "complementarity_construct": {
1112         "time_s": complement_time,
1113         "cloud_y_offsets_m": [-9.65, 9.65],
1114         "individual_worst_margins_m": complement_individual,
1115         "joint_worst_margin_m": complement_joint,
1116         "each_individual_incomplete": all(value > 0.0 for value in complement_individual),
1117         "joint_complete": complement_joint <= 0.0,
1118     },
1119     "single_cloud_q2_regression": {
1120         "q3_joint_duration_s": _duration(single_intervals),
1121         "q2_strict_duration_s": q2_reference.strict_duration_s,
1122         "absolute_difference_s": abs(
1123             _duration(single_intervals) - q2_reference.strict_duration_s
1124         ),
1125         "tolerance_s": float(validation["single_cloud_tolerance_s"]),
1126     },
1127     "joint_vs_conservative": {
1128         "joint_duration_s": evaluation.joint_duration_s,
1129         "conservative_duration_s": evaluation.conservative_duration_s,
1130         "joint_not_less": evaluation.joint_duration_s + 1e-9
1131         >= evaluation.conservative_duration_s,
1132     },
1133     "deletion": [
1134         {
1135             "bomb": index,
1136             "remaining_duration_s": duration,
1137             "marginal_loss_s": evaluation.joint_duration_s - duration,
1138         }
1139         for index, duration in enumerate(evaluation.deletion_durations_s, 1)
1140     ],
1141 }
1142
1143
1144 def _conditional_third_bomb_search(
1145     baseline: Q3Evaluation, config: dict[str, Any]
1146 ) -> tuple[dict[str, Any], Q3Evaluation | None]:
1147     """Optimize only the third bomb conditional on the first two formal controls."""
1148     settings = config["conditional_third_bomb_search"]
1149     gravity = float(config["gravity"])
1150     baseline_vector = np.array(list(asdict(baseline.strategy).values()), dtype=float)
1151
1152     def full_vector(values: np.ndarray) -> np.ndarray:
1153         candidate = baseline_vector.copy()
1154         candidate[4] = values[0]
1155         candidate[7] = values[1]
1156         return candidate
1157
1158     search_settings = {
1159         "time_step_s": settings["time_step_s"],
1160         "surface_angles": settings["surface_angles"],
1161         "surface_levels": settings["surface_levels"],
1162     }
1163     result = differential_evolution(
1164         lambda values: _objective(full_vector(values), gravity, search_settings, roots=False),
1165         [tuple(config["bounds"]["s2_s"]), tuple(config["bounds"]["fuse_delay_s"])],
1166         seed=int(settings["seed"]),
1167         maxiter=int(settings["maxiter"]),

```

```

1168     popsize=int(settings["popsize"]),
1169     tol=float(settings["tol"]),
1170     polish=False,
1171     workers=1,
1172 )
1173 population = np.asarray(result.population)
1174 order = np.argsort(result.population_energies)
1175 candidate_values = [result.x]
1176 candidate_values.extend(population[order[: int(settings["root_candidate_count"])]])
1177 candidate_values.append(np.array([baseline.strategy.s2_s, baseline.strategy.fuse3_s]))
1178 root_settings = {
1179     "time_step_s": config["formal_evaluation"]["time_scan_step_s"],
1180     "surface_angles": config["formal_evaluation"]["surface_angles"],
1181     "surface_levels": config["formal_evaluation"]["surface_levels"],
1182 }
1183 root_ranked = []
1184 for values in _deduplicate(candidate_values):
1185     objective = _objective(full_vector(values), gravity, root_settings, roots=True)
1186     root_ranked.append(
1187         {
1188             "s2_s": float(values[0]),
1189             "fuse3_s": float(values[1]),
1190             "root_refined_duration_s": max(0.0, -objective),
1191             "vector": full_vector(values),
1192         }
1193     )
1194 root_ranked.sort(key=lambda item: item["root_refined_duration_s"], reverse=True)
1195 formal_candidate = _formal_evaluation(
1196     _strategy_from_array(np.asarray(root_ranked[0]["vector"])), config
1197 )
1198 improvement = formal_candidate.joint_duration_s - baseline.joint_duration_s
1199 threshold = float(settings["improvement_threshold_s"])
1200 record = {
1201     "scope": "theta, speed, and first two bombs fixed at the formal baseline",
1202     "variable_order": ["s2_s", "fuse3_s"],
1203     "seed": int(settings["seed"]),
1204     "search_success": bool(result.success),
1205     "search_message": str(result.message),
1206     "search_evaluations": int(result.nfev),
1207     "settings": settings,
1208     "root_refined_candidates": [
1209         {key: value for key, value in item.items() if key != "vector"} for item in root_ranked
1210     ],
1211     "formal_candidate": _strategy_summary(formal_candidate, 1),
1212     "baseline_joint_duration_s": baseline.joint_duration_s,
1213     "formal_improvement_s": improvement,
1214     "improvement_threshold_s": threshold,
1215     "material_improvement_found": improvement > threshold,
1216     "conclusion": (
1217         "conditional search found a material third-bomb contribution"
1218         if improvement > threshold
1219         else "conditional on the first two bombs, no material third-bomb contribution was found"
1220     ),
1221 }
1222 return record, formal_candidate if improvement > threshold else None
1223
1224
1225 def run_question_3(project_root: Path, config: dict[str, Any]) -> list[Path]:
1226     """Optimize Question 3 and write tables, provisional workbook, and audit logs."""
1227     paths = prepare_output_paths(project_root / "outputs")
1228     stage_log_path = paths.logs / "q3_search_stage.json"
1229     evaluations, solver = optimize_question_3(config, stage_log_path=stage_log_path)

```

```

1230     if not evaluations:
1231         raise RuntimeError("Question 3 optimization produced no feasible formal candidate")
1232     best = evaluations[0]
1233     conditional_record, conditional_improvement = _conditional_third_bomb_search(best, config)
1234     if conditional_improvement is not None:
1235         evaluations = [conditional_improvement, *evaluations]
1236         evaluations.sort(key=lambda item: item.joint_duration_s, reverse=True)
1237         best = evaluations[0]
1238     near_count = min(int(config["near_optimal_count"]), len(evaluations))
1239     summaries = [
1240         _strategy_summary(item, rank) for rank, item in enumerate(evaluations[:near_count], 1)
1241     ]
1242     summary_path = save_table(
1243         pd.DataFrame(summaries[:1]), paths.tables / "q3_summary.csv", overwrite=True
1244     )
1245     candidates_path = save_table(
1246         pd.DataFrame(summaries), paths.tables / "q3_candidates.csv", overwrite=True
1247     )
1248     bombs = pd.DataFrame(_bomb_rows(best))
1249     bombs_path = save_table(bombs, paths.tables / "q3_bombs.csv", overwrite=True)
1250     interval_rows = []
1251     for criterion, groups in (
1252         ("joint-union-cover", (best.joint_intervals,)),
1253         ("conservative-single-cloud-union", (best.conservative_union_intervals,)),
1254         ("bomb-1-strict", (best.individual_strict_intervals[0],)),
1255         ("bomb-2-strict", (best.individual_strict_intervals[1],)),
1256         ("bomb-3-strict", (best.individual_strict_intervals[2],)),
1257     ):
1258         for group in groups:
1259             for number, interval in enumerate(group, 1):
1260                 interval_rows.append(
1261                     {
1262                         "criterion": criterion,
1263                         "interval": number,
1264                         "start_s": interval.start,
1265                         "end_s": interval.end,
1266                         "duration_s": interval.duration,
1267                     }
1268                 )
1269     intervals_path = save_table(
1270         pd.DataFrame(interval_rows), paths.tables / "q3_intervals.csv", overwrite=True
1271     )
1272     workbook_path = paths.tables / "result1.xlsx"
1273     with pd.ExcelWriter(workbook_path, engine="openpyxl") as writer:
1274         bombs.to_excel(writer, sheet_name="provisional_result1", index=False)
1275         pd.DataFrame(
1276             [
1277                 {
1278                     "schema_status": "provisional",
1279                     "reason": "official result1.xlsx template is missing",
1280                     "mapping_action": (
1281                         "map these explicit fields when the official template is supplied"
1282                     ),
1283                 }
1284             ]
1285         ).to_excel(writer, sheet_name="schema_note", index=False)
1286
1287     optimization_path = paths.logs / "q3_optimization.json"
1288     optimization_path.write_text(
1289         json.dumps(
1290             {
1291                 "model": "q3-joint-ray-cover-v1",

```

```

1292         "quantifiers": (
1293             "for every target ray, at least one active cloud intersects before target"
1294         ),
1295         "best": {**_strategy_summary(best, 1), "bombs": _bomb_rows(best)},
1296         "near_optimal_candidates": summaries,
1297         "solver": solver,
1298     },
1299     ensure_ascii=False,
1300     indent=2,
1301 )
1302 + "\n",
1303     encoding="utf-8",
1304 )
1305 validation_path = paths.logs / "q3_validation.json"
1306 validation_path.write_text(
1307     json.dumps(_validation_record(best, config), ensure_ascii=False, indent=2) + "\n",
1308     encoding="utf-8",
1309 )
1310 conditional_path = paths.logs / "q3_third_bomb_incremental.json"
1311 conditional_path.write_text(
1312     json.dumps(conditional_record, ensure_ascii=False, indent=2) + "\n",
1313     encoding="utf-8",
1314 )
1315 return [
1316     summary_path,
1317     candidates_path,
1318     bombs_path,
1319     intervals_path,
1320     workbook_path,
1321     stage_log_path,
1322     optimization_path,
1323     validation_path,
1324     conditional_path,
1325 ]

```

B.5 问题四程序

```

1  """Question 4 optimization for one smoke bomb from each of FY1, FY2, and FY3."""
2
3  from __future__ import annotations
4
5  import json
6  from collections.abc import Sequence
7  from dataclasses import asdict, dataclass
8  from itertools import permutations
9  from pathlib import Path
10 from typing import Any
11
12 import numpy as np
13 import pandas as pd
14 from scipy.optimize import differential_evolution, minimize
15
16 from mmkit.artifacts import prepare_output_paths, save_table
17
18 from .q1 import CLOUD_DESCENT_SPEED, CLOUD_LIFETIME, OcclusionInterval, missile_position
19 from .q2 import missile_hit_time
20 from .q3 import (
21     BombGeometry,
22     Q3Strategy,
23     _active_bombs,
24     _cloud_center,

```

```

25     _duration,
26     _find_joint_intervals,
27     _independent_ray_intersections,
28     _merge_intervals,
29     _sampled_metrics,
30     _settings_margin,
31     _surface_points,
32     joint_coverage_assessment,
33     joint_coverage_margin,
34 )
35 from .q3 import (
36     _bomb_geometries as q3_bomb_geometries,
37 )
38
39 UAV_NAMES = ("FY1", "FY2", "FY3")
40
41
42 @dataclass(frozen=True, slots=True)
43 class UAVControl:
44     """Flight and bomb controls for one UAV."""
45
46     theta_rad: float
47     speed_m_s: float
48     release_time_s: float
49     fuse_delay_s: float
50
51     @property
52     def explosion_time_s(self) -> float:
53         return self.release_time_s + self.fuse_delay_s
54
55
56 @dataclass(frozen=True, slots=True)
57 class Q4Strategy:
58     """Independent four-variable controls for FY1, FY2, and FY3."""
59
60     controls: tuple[UAVControl, UAVControl, UAVControl]
61
62
63 @dataclass(frozen=True, slots=True)
64 class Q4Evaluation:
65     """Formal joint evaluation for one Question 4 strategy."""
66
67     strategy: Q4Strategy
68     bombs: tuple[BombGeometry, BombGeometry, BombGeometry]
69     joint_intervals: tuple[OcclusionInterval, ...]
70     individual_intervals: tuple[tuple[OcclusionInterval, ...], ...]
71     conservative_intervals: tuple[OcclusionInterval, ...]
72     deletion_durations_s: tuple[float, float, float]
73
74     @property
75     def joint_duration_s(self) -> float:
76         return _duration(self.joint_intervals)
77
78     @property
79     def conservative_duration_s(self) -> float:
80         return _duration(self.conservative_intervals)
81
82
83 def _strategy_from_array(values: np.ndarray) -> Q4Strategy:
84     array = np.asarray(values, dtype=float).reshape(3, 4)
85     return Q4Strategy(tuple(UAVControl(*(float(value) for value in row)) for row in array))
86

```

```

87
88 def _strategy_vector(strategy: Q4Strategy) -> np.ndarray:
89     return np.array(
90         [value for control in strategy.controls for value in asdict(control).values()], dtype=float
91     )
92
93
94 def _uav_initials(config: dict[str, Any]) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
95     return tuple(np.asarray(config["uavs"][name], dtype=float) for name in UAV_NAMES)
96
97
98 def _bomb_geometries(
99     strategy: Q4Strategy, gravity: float, config: dict[str, Any]
100 ) -> tuple[BombGeometry, BombGeometry, BombGeometry]:
101     bombs = []
102     for index, (control, initial) in enumerate(
103         zip(strategy.controls, _uav_initials(config), strict=True), 1
104     ):
105         heading = np.array([np.cos(control.theta_rad), np.sin(control.theta_rad), 0.0])
106         release = initial + control.speed_m_s * control.release_time_s * heading
107         explosion = initial + control.speed_m_s * control.explosion_time_s * heading
108         explosion = explosion + np.array([0.0, 0.0, -0.5 * gravity * control.fuse_delay_s**2])
109         bombs.append(
110             BombGeometry(
111                 bomb=index,
112                 release_time_s=control.release_time_s,
113                 fuse_delay_s=control.fuse_delay_s,
114                 explosion_time_s=control.explosion_time_s,
115                 release_point=tuple(float(value) for value in release),
116                 explosion_point=tuple(float(value) for value in explosion),
117             )
118         )
119     return tuple(bombs)
120
121
122 def _fuse_maxima(config: dict[str, Any]) -> tuple[float, float, float]:
123     gravity = float(config["gravity"])
124     return tuple(float(np.sqrt(2.0 * initial[2] / gravity)) for initial in _uav_initials(config))
125
126
127 def _bounds(config: dict[str, Any]) -> list[tuple[float, float]]:
128     common = config["bounds"]
129     bounds = []
130     for fuse_maximum in _fuse_maxima(config):
131         bounds.extend(
132             (
133                 tuple(common["theta_rad"]),
134                 tuple(common["speed_m_s"]),
135                 tuple(common["release_time_s"]),
136                 (0.0, fuse_maximum),
137             )
138         )
139     return bounds
140
141
142 def _feasibility_violation(strategy: Q4Strategy, gravity: float, config: dict[str, Any]) -> float:
143     violation = 0.0
144     hit_time = missile_hit_time()
145     for control, bomb, fuse_maximum, initial in zip(
146         strategy.controls,
147         _bomb_geometries(strategy, gravity, config),
148         _fuse_maxima(config),

```

```

149     _uav_initials(config),
150     strict=True,
151 ):
152     violation += max(0.0, 70.0 - control.speed_m_s) / 70.0
153     violation += max(0.0, control.speed_m_s - 140.0) / 70.0
154     violation += max(0.0, -control.release_time_s) / hit_time
155     violation += max(0.0, -control.fuse_delay_s) / fuse_maximum
156     violation += max(0.0, control.fuse_delay_s - fuse_maximum) / fuse_maximum
157     violation += max(0.0, control.explosion_time_s - hit_time) / hit_time
158     violation += max(0.0, -bomb.explosion_point[2]) / float(initial[2])
159     return violation
160
161
162 def _objective(
163     values: np.ndarray,
164     gravity: float,
165     config: dict[str, Any],
166     settings: dict[str, Any],
167     *,
168     roots: bool,
169 ) -> float:
170     strategy = _strategy_from_array(values)
171     violation = _feasibility_violation(strategy, gravity, config)
172     if violation > 0.0:
173         return 100.0 + 100.0 * violation
174     bombs = _bomb_geometries(strategy, gravity, config)
175     margin = _settings_margin(bombs, settings, refine=False)
176     duration, minimum = _sampled_metrics(bombs, margin, float(settings["time_step_s"]))
177     if duration <= 0.0:
178         return max(minimum, 0.0) / 100.0
179     if not roots:
180         return -duration
181     intervals = _find_joint_intervals(
182         margin,
183         bombs,
184         scan_step=float(settings["time_step_s"]),
185         root_tolerance=1e-8,
186     )
187     return -_duration(intervals)
188
189
190 def _single_initial_population(
191     initial: np.ndarray, fuse_maximum: float, config: dict[str, Any], seed: int
192 ) -> np.ndarray:
193     settings = config["single_search"]
194     count = 4 * int(settings["popsize"])
195     rng = np.random.default_rng(seed)
196     lower = np.array([0.0, 70.0, 0.0, 0.0])
197     upper = np.array([2.0 * np.pi, 140.0, 30.0, fuse_maximum])
198     population = rng.uniform(lower, upper, size=(count, 4))
199     toward_origin = float(np.arctan2(-initial[1], -initial[0]) % (2.0 * np.pi))
200     structured = []
201     for speed in (70.0, 100.0, 140.0):
202         for release, fraction in ((0.0, 0.15), (0.0, 0.35), (2.0, 0.25)):
203             structured.append(np.array([toward_origin, speed, release, fraction * fuse_maximum]))
204     for index, candidate in enumerate(structured):
205         population[index] = candidate
206     return population
207
208
209 def _single_searches(config: dict[str, Any]) -> tuple[list[np.ndarray], list[dict[str, Any]]]:
210     gravity = float(config["gravity"])

```



```

211 settings = config["single_search"]
212 best_vectors = []
213 records = []
214 for index, (name, initial, fuse_maximum) in enumerate(
215     zip(UAV_NAMES, _uav_initials(config), _fuse_maxima(config), strict=True)
216 ):
217
218     def objective(values: np.ndarray, fixed_index: int = index) -> float:
219         controls = [UAVControl(0.0, 70.0, 0.0, 0.0) for _ in UAV_NAMES]
220         controls[fixed_index] = UAVControl(*(float(value) for value in values))
221         strategy = Q4Strategy(tuple(controls))
222         bomb = (_bomb_geometries(strategy, gravity, config)[fixed_index],)
223         control = strategy.controls[fixed_index]
224         if control.explosion_time_s > missile_hit_time():
225             return 100.0 + control.explosion_time_s - missile_hit_time()
226         margin = _settings_margin(bomb, settings, refine=False)
227         duration, minimum = _sampled_metrics(bomb, margin, float(settings["time_step_s"]))
228         return -duration if duration > 0.0 else max(minimum, 0.0) / 100.0
229
230     result = differential_evolution(
231         objective,
232         [
233             tuple(config["bounds"]["theta_rad"]),
234             tuple(config["bounds"]["speed_m_s"]),
235             tuple(config["bounds"]["release_time_s"]),
236             (0.0, fuse_maximum),
237         ],
238         seed=int(config["seed"]) + index,
239         init=_single_initial_population(
240             initial, fuse_maximum, config, int(config["seed"]) + index
241         ),
242         maxiter=int(settings["maxiter"]),
243         popsize=int(settings["popsize"]),
244         tol=float(settings["tol"]),
245         polish=False,
246         workers=1,
247     )
248     best_vectors.append(np.asarray(result.x, dtype=float))
249     records.append(
250         {
251             "uav": name,
252             "success": bool(result.success),
253             "message": str(result.message),
254             "evaluations": int(result.nfev),
255             "sampled_duration_s": max(0.0, -float(result.fun)),
256             "vector": np.asarray(result.x, dtype=float).tolist(),
257         }
258     )
259     return best_vectors, records
260
261
262 def _joint_initial_population(
263     single_vectors: Sequence[np.ndarray], config: dict[str, Any]
264 ) -> np.ndarray:
265     bounds = _bounds(config)
266     count = len(bounds) * int(config["joint_search"]["popsize"])
267     rng = np.random.default_rng(int(config["seed"]) + 100)
268     lower = np.array([bound[0] for bound in bounds])
269     upper = np.array([bound[1] for bound in bounds])
270     population = rng.uniform(lower, upper, size=(count, len(bounds)))
271     base = np.concatenate(single_vectors)
272     population[0] = base

```

```

273     variants = (
274         np.zeros(12),
275         np.array([0, 0, 0, 0, 0, 0, 2, 0, 0, 0, 4, 0], dtype=float),
276         np.array([0, 0, 0, 0, 0.03, 10, 0, 0, -0.03, -5, 0, 0], dtype=float),
277         np.array([0, 0, 0, 0.3, 0, 0, 0, -0.3, 0, 0, 0, 0.2], dtype=float),
278     )
279     for index, offset in enumerate(variants, 1):
280         population[index] = np.clip(base + offset, lower, upper)
281     for index in range(5, min(16, count)):
282         population[index] = np.clip(base + rng.normal(0.0, 0.2, 12), lower, upper)
283     return population
284
285
286 def _deduplicate(candidates: Sequence[np.ndarray]) -> list[np.ndarray]:
287     unique = []
288     for candidate in candidates:
289         if not any(np.linalg.norm(candidate - existing) < 1e-7 for existing in unique):
290             unique.append(np.asarray(candidate, dtype=float))
291     return unique
292
293
294 def _formal_evaluation(strategy: Q4Strategy, config: dict[str, Any]) -> Q4Evaluation:
295     gravity = float(config["gravity"])
296     bombs = _bomb_geometries(strategy, gravity, config)
297     settings = config["formal_evaluation"]
298     joint = _find_joint_intervals(
299         _settings_margin(bombs, settings, refine=True),
300         bombs,
301         scan_step=float(settings["time_scan_step_s"]),
302         root_tolerance=float(settings["root_tolerance_s"]),
303     )
304     individual = []
305     for bomb in bombs:
306         intervals = _find_joint_intervals(
307             _settings_margin((bomb,), settings, refine=True),
308             (bomb,),
309             scan_step=float(settings["time_scan_step_s"]),
310             root_tolerance=float(settings["root_tolerance_s"]),
311         )
312         individual.append(intervals)
313     conservative = _merge_intervals([interval for group in individual for interval in group])
314     deletion = []
315     for removed in range(3):
316         remaining = tuple(bomb for index, bomb in enumerate(bombs) if index != removed)
317         intervals = _find_joint_intervals(
318             _settings_margin(remaining, settings, refine=False),
319             remaining,
320             scan_step=float(settings["time_scan_step_s"]),
321             root_tolerance=float(settings["root_tolerance_s"]),
322         )
323         deletion.append(min(_duration(intervals), _duration(joint)))
324     return Q4Evaluation(
325         strategy=strategy,
326         bombs=bombs,
327         joint_intervals=joint,
328         individual_intervals=tuple(individual),
329         conservative_intervals=conservative,
330         deletion_durations_s=tuple(deletion),
331     )
332
333
334 def optimize_question_4(

```

```

335     config: dict[str, Any], stage_log_path: Path
336 ) -> tuple[list[Q4Evaluation], dict[str, Any]]:
337     """Run single-UAV searches, joint global search, local refinement, and formal ranking."""
338     gravity = float(config["gravity"])
339     single_vectors, single_records = _single_searches(config)
340     stage_log_path.write_text(
341         json.dumps(
342             {"stage": "single-search-complete", "single_candidates": single_records},
343             ensure_ascii=False,
344             indent=2,
345         )
346         + "\n",
347         encoding="utf-8",
348     )
349     print("Q4 single-UAV searches complete", flush=True)
350     settings = config["joint_search"]
351     result = differential_evolution(
352         lambda values: _objective(values, gravity, config, settings, roots=False),
353         _bounds(config),
354         seed=int(config["seed"]) + 10,
355         init=_joint_initial_population(single_vectors, config),
356         maxiter=int(settings["maxiter"]),
357         popsize=int(settings["popsize"]),
358         tol=float(settings["tol"]),
359         polish=False,
360         workers=1,
361     )
362     order = np.argsort(result.population_energies)
363     candidates = [result.x, np.concatenate(single_vectors)]
364     candidates.extend(np.asarray(result.population)[order[:10]])
365     stage_log_path.write_text(
366         json.dumps(
367             {
368                 "stage": "joint-global-complete",
369                 "single_candidates": single_records,
370                 "success": bool(result.success),
371                 "message": str(result.message),
372                 "evaluations": int(result.nfev),
373                 "sampled_duration_s": max(0.0, -float(result.fun)),
374                 "candidate_vectors": [np.asarray(item).tolist() for item in candidates],
375                 "sampled_time_grid": "absolute task time t=0",
376             },
377             ensure_ascii=False,
378             indent=2,
379         )
380         + "\n",
381         encoding="utf-8",
382     )
383     print("Q4 joint global search complete", flush=True)
384     local_results = []
385     local_settings = config["local_refinement"]
386     for start in _deduplicate(candidates)[:int(local_settings["starts"])]:
387         local = minimize(
388             lambda values: _objective(values, gravity, config, local_settings, roots=True),
389             start,
390             method="Nelder-Mead",
391             bounds=_bounds(config),
392             options={
393                 "maxfev": int(local_settings["maxfev"]),
394                 "xatol": float(local_settings["xatol"]),
395                 "fatol": float(local_settings["fatol"]),
396             },

```

```

397     )
398     local_results.append(local)
399     candidates.append(local.x)
400     stage_log_path.write_text(
401         json.dumps(
402             {
403                 "stage": "local-refinement-in-progress",
404                 "completed_starts": len(local_results),
405                 "planned_starts": int(local_settings["starts"]),
406                 "local_results": [
407                     {
408                         "success": bool(item.success),
409                         "message": str(item.message),
410                         "evaluations": int(item.nfev),
411                         "duration_s": max(0.0, -float(item.fun)),
412                         "vector": np.asarray(item.x).tolist(),
413                     }
414                     for item in local_results
415                 ],
416             },
417             ensure_ascii=False,
418             indent=2,
419         )
420         + "\n",
421         encoding="utf-8",
422     )
423     print(f"Q4 local refinement {len(local_results)} complete", flush=True)
424     feasible = [
425         item
426         for item in _deduplicate(candidates)
427         if _feasibility_violation(_strategy_from_array(item), gravity, config) <= 1e-12
428     ]
429     ranked = sorted(
430         feasible,
431         key=lambda values: _objective(values, gravity, config, local_settings, roots=True),
432     )
433     count = min(int(config["formal_evaluation"]["candidate_count"]), len(ranked))
434     incumbent = np.concatenate(single_vectors)
435     formal_vectors = _deduplicate([*ranked[: max(0, count - 1)], incumbent][:count])
436     formal = []
437     for index, candidate in enumerate(formal_vectors, 1):
438         formal.append(_formal_evaluation(_strategy_from_array(candidate), config))
439         print(f"Q4 formal candidate {index}/{count} complete", flush=True)
440     formal.sort(key=lambda item: item.joint_duration_s, reverse=True)
441     stage_log_path.write_text(
442         json.dumps(
443             {
444                 "stage": "search-complete",
445                 "single_candidates": single_records,
446                 "global_success": bool(result.success),
447                 "global_message": str(result.message),
448                 "global_evaluations": int(result.nfev),
449                 "completed_local_starts": len(local_results),
450                 "formal_candidate_count": len(formal),
451                 "formal_durations_s": [item.joint_duration_s for item in formal],
452                 "single_concat_incumbent_retained": True,
453                 "sampled_time_grid": "absolute task time t=0",
454             },
455             ensure_ascii=False,
456             indent=2,
457         )
458         + "\n",

```

```

459         encoding="utf-8",
460     )
461     solver = {
462         "seed": int(config["seed"]),
463         "single_searches": single_records,
464         "joint_global": {
465             "success": bool(result.success),
466             "message": str(result.message),
467             "evaluations": int(result.nfev),
468             "sampled_duration_s": max(0.0, -float(result.fun)),
469             "settings": settings,
470         },
471         "local_refinements": [
472             {
473                 "success": bool(item.success),
474                 "message": str(item.message),
475                 "evaluations": int(item.nfev),
476                 "duration_s": max(0.0, -float(item.fun)),
477             }
478             for item in local_results
479         ],
480         "candidate_initialization": (
481             "per-UAV single candidates plus relay, overlap, heading/speed, "
482             "and edge-complement variants"
483         ),
484         "sampled_time_grid": "absolute task time t=0 plus cloud events",
485         "status": "converged" if result.success else "budget-exhausted-with-feasible-candidates",
486         "convergence_claimed": bool(result.success),
487         "formal_settings": config["formal_evaluation"],
488     }
489     return formal, solver
490
491
492 def _summary_row(evaluation: Q4Evaluation, rank: int) -> dict[str, Any]:
493     return {
494         "rank": rank,
495         "joint_duration_s": evaluation.joint_duration_s,
496         "conservative_duration_s": evaluation.conservative_duration_s,
497         "joint_synergy_s": evaluation.joint_duration_s - evaluation.conservative_duration_s,
498         "individual_durations_s": json.dumps(
499             [_duration(intervals) for intervals in evaluation.individual_intervals]
500         ),
501     }
502
503
504 def _uav_rows(evaluation: Q4Evaluation) -> list[dict[str, Any]]:
505     rows = []
506     for name, control, bomb, individual, deletion in zip(
507         UAV_NAMES,
508         evaluation.strategy.controls,
509         evaluation.bombs,
510         evaluation.individual_intervals,
511         evaluation.deletion_durations_s,
512         strict=True,
513     ):
514         rows.append(
515             {
516                 "schema_status": "provisional-schema; official result2.xlsx unavailable",
517                 "uav": name,
518                 "theta_rad": control.theta_rad,
519                 "theta_deg": float(np.degrees(control.theta_rad) % 360.0),
520                 "speed_m_s": control.speed_m_s,

```

```

521         "release_time_s": control.release_time_s,
522         "release_x_m": bomb.release_point[0],
523         "release_y_m": bomb.release_point[1],
524         "release_z_m": bomb.release_point[2],
525         "fuse_delay_s": control.fuse_delay_s,
526         "explosion_time_s": control.explosion_time_s,
527         "explosion_x_m": bomb.explosion_point[0],
528         "explosion_y_m": bomb.explosion_point[1],
529         "explosion_z_m": bomb.explosion_point[2],
530         "individual_strict_duration_s": _duration(individual),
531         "joint_deletion_loss_s": evaluation.joint_duration_s - deletion,
532         "minimum_cloud_center_height_m": bomb.explosion_point[2]
533         - CLOUD_DESCENT_SPEED * CLOUD_LIFETIME,
534     }
535 )
536 return rows
537
538
539 def _validation_record(evaluation: Q4Evaluation, config: dict[str, Any]) -> dict[str, Any]:
540     validation = config["validation"]
541     hit_time = missile_hit_time()
542     convergence = []
543     convergence_settings = validation["convergence_settings"]
544     for index, settings in enumerate(convergence_settings):
545         refine = index == len(convergence_settings) - 1
546         merged = {
547             **settings,
548             "continuous_refinement_starts": 1,
549             "continuous_refinement_maxfev": 50,
550             "adaptive_max_surface_angles": (
551                 config["formal_evaluation"]["adaptive_max_surface_angles"]
552                 if refine
553                 else settings["surface_angles"]
554             ),
555             "adaptive_max_surface_levels": (
556                 config["formal_evaluation"]["adaptive_max_surface_levels"]
557                 if refine
558                 else settings["surface_levels"]
559             ),
560         }
561         intervals = _find_joint_intervals(
562             _settings_margin(evaluation.bombs, merged, refine=refine),
563             evaluation.bombs,
564             scan_step=float(settings["time_scan_step_s"]),
565             root_tolerance=float(config["formal_evaluation"]["root_tolerance_s"]),
566         )
567         convergence.append(
568             {**settings, "continuous_refinement": refine, "joint_duration_s": _duration(intervals)}
569         )
570     boundary = []
571     direct = []
572     points = _surface_points(
573         int(validation["direct_surface_angles"]), int(validation["direct_surface_levels"])
574     )
575     probe = float(validation["boundary_probe_s"])
576     formal = config["formal_evaluation"]
577     for interval in evaluation.joint_intervals:
578         for location, time in (
579             ("before-start", interval.start - probe),
580             ("start", interval.start),
581             ("midpoint", 0.5 * (interval.start + interval.end)),
582             ("end", interval.end),

```

```

583         ("after-end", interval.end + probe),
584     ):
585         assessment = joint_coverage_assessment(
586             time,
587             evaluation.bombs,
588             surface_angles=int(formal["surface_angles"]),
589             surface_levels=int(formal["surface_levels"]),
590             continuous_refinement=True,
591             refinement_starts=int(formal["continuous_refinement_starts"]),
592             refinement_maxfev=int(formal["continuous_refinement_maxfev"]),
593             adaptive_max_surface_angles=int(formal["adaptive_max_surface_angles"]),
594             adaptive_max_surface_levels=int(formal["adaptive_max_surface_levels"]),
595         )
596         active = _active_bombs(time, evaluation.bombs)
597         covered = _independent_ray_intersections(
598             points,
599             missile_position(time),
600             [_cloud_center(time, bomb) for bomb in active],
601         )
602         boundary.append(
603             {
604                 "location": location,
605                 "time_s": time,
606                 "status": assessment.status,
607                 "margin_m": assessment.signed_margin_m,
608                 "lower_bound_m": assessment.sampled_lower_bound_m,
609                 "upper_bound_m": assessment.lipschitz_upper_bound_m,
610                 "surface_angles": assessment.surface_angles,
611                 "surface_levels": assessment.surface_levels,
612                 "trace": assessment.refinement_trace,
613             }
614         )
615         direct.append(
616             {
617                 "location": location,
618                 "time_s": time,
619                 "sampled_rays": len(points),
620                 "uncovered_rays": int(np.count_nonzero(~covered)),
621                 "all_covered": bool(np.all(covered)),
622             }
623         )
624         permutation_durations = []
625         medium = validation["convergence_settings"][1]
626         for order in permutations(evaluation.bombs):
627             intervals = _find_joint_intervals(
628                 _settings_margin(order, medium, refine=False),
629                 order,
630                 scan_step=float(medium["time_scan_step_s"]),
631                 root_tolerance=float(formal["root_tolerance_s"]),
632             )
633             permutation_durations.append(_duration(intervals))
634         q3_vector = np.array(
635             [
636                 3.1047922065753095,
637                 75.53740845287045,
638                 0.00008659727834377369,
639                 0.0001070796609000733,
640                 0.0000817521129715,
641                 2.7648584468708908,
642                 2.58163903013058,
643                 9.334353521045797,
644             ]

```

```

645 )
646 q3_bombs = q3_bomb_geometries(Q3Strategy(*q3_vector), float(config["gravity"]))
647 q3_intervals = _find_joint_intervals(
648     _settings_margin(q3_bombs, formal, refine=True),
649     q3_bombs,
650     scan_step=float(formal["time_scan_step_s"]),
651     root_tolerance=float(formal["root_tolerance_s"]),
652 )
653 complement_time = 4.768
654 center = np.array([17625.525268056128, 10.240444866222791, 1762.640057120904])
655 complement_bombs = tuple(
656     BombGeometry(
657         index,
658         complement_time,
659         0.0,
660         complement_time,
661         tuple(center + np.array([0.0, sign * 9.65, 0.0])),
662         tuple(center + np.array([0.0, sign * 9.65, 0.0])),
663     )
664     for index, sign in enumerate((-1.0, 1.0), 1)
665 )
666 individual_complement = [
667     joint_coverage_margin(
668         complement_time,
669         (bomb,),
670         surface_angles=360,
671         surface_levels=41,
672         continuous_refinement=True,
673         refinement_starts=1,
674         refinement_maxfev=50,
675     )
676     for bomb in complement_bombs
677 ]
678 joint_complement = joint_coverage_margin(
679     complement_time,
680     complement_bombs,
681     surface_angles=360,
682     surface_levels=41,
683     continuous_refinement=True,
684     refinement_starts=1,
685     refinement_maxfev=50,
686 )
687 return {
688     "feasibility": {
689         "violation": _feasibility_violation(
690             evaluation.strategy, float(config["gravity"]), config
691         ),
692         "speeds_m_s": [control.speed_m_s for control in evaluation.strategy.controls],
693         "release_times_s": [control.release_time_s for control in evaluation.strategy.controls],
694         "fuse_delays_s": [control.fuse_delay_s for control in evaluation.strategy.controls],
695         "explosion_times_s": [
696             control.explosion_time_s for control in evaluation.strategy.controls
697         ],
698         "explosion_heights_m": [bomb.explosion_point[2] for bomb in evaluation.bombs],
699         "cloud_minimum_heights_m": [
700             bomb.explosion_point[2] - CLOUD_DESCENT_SPEED * CLOUD_LIFETIME
701             for bomb in evaluation.bombs
702         ],
703         "all_speeds_within_bounds": all(
704             70.0 <= control.speed_m_s <= 140.0 for control in evaluation.strategy.controls
705         ),
706         "all_release_and_fuse_times_nonnegative": all(

```



```

707         control.release_time_s >= 0.0 and control.fuse_delay_s >= 0.0
708         for control in evaluation.strategy.controls
709     ),
710     "all_explosions_before_missile_hit": all(
711         control.explosion_time_s <= hit_time for control in evaluation.strategy.controls
712     ),
713     "all_explosions_before_ground": all(
714         bomb.explosion_point[2] >= 0.0 for bomb in evaluation.bombs
715     ),
716     "cloud_center_below_ground_triggered": any(
717         bomb.explosion_point[2] - CLOUD_DESCENT_SPEED * CLOUD_LIFETIME < 0.0
718         for bomb in evaluation.bombs
719     ),
720 },
721 "surface_time_convergence": convergence,
722 "boundary_checks": boundary,
723 "independent_unit_direction_ray_checks": direct,
724 "label_permutation_durations_s": permutation_durations,
725 "q3_geometry_regression": {
726     "computed_duration_s": _duration(q3_intervals),
727     "expected_duration_s": float(validation["q3_regression_expected_s"]),
728     "absolute_difference_s": abs(
729         _duration(q3_intervals) - float(validation["q3_regression_expected_s"])
730     ),
731     "tolerance_s": float(validation["q3_regression_tolerance_s"]),
732 },
733 "complementarity_construct": {
734     "individual_margins_m": individual_complement,
735     "joint_margin_m": joint_complement,
736     "each_individual_incomplete": all(value > 0.0 for value in individual_complement),
737     "joint_complete": joint_complement <= 0.0,
738 },
739 "joint_vs_conservative": {
740     "joint_duration_s": evaluation.joint_duration_s,
741     "conservative_duration_s": evaluation.conservative_duration_s,
742     "joint_not_less": evaluation.joint_duration_s + 1e-9
743     >= evaluation.conservative_duration_s,
744     "joint_not_less_than_each_individual": evaluation.joint_duration_s + 1e-9
745     >= max(_duration(intervals) for intervals in evaluation.individual_intervals),
746 },
747 "deletion": [
748     {
749         "uav": name,
750         "remaining_duration_s": duration,
751         "marginal_loss_s": evaluation.joint_duration_s - duration,
752     }
753     for name, duration in zip(UAV_NAMES, evaluation.deletion_durations_s, strict=True)
754 ],
755 }
756
757
758 def run_question_4(project_root: Path, config: dict[str, Any]) -> list[Path]:
759     """Run Question 4 and write strategy, workbook, solver, and validation artifacts."""
760     paths = prepare_output_paths(project_root / "outputs")
761     stage_path = paths.logs / "q4_search_stage.json"
762     evaluations, solver = optimize_question_4(config, stage_path)
763     if not evaluations:
764         raise RuntimeError("Question 4 optimization produced no formal candidate")
765     best = evaluations[0]
766     near_count = min(int(config["near_optimal_count"]), len(evaluations))
767     summaries = [_summary_row(item, rank) for rank, item in enumerate(evaluations[:near_count], 1)]
768     summary_path = save_table(

```

```

769     pd.DataFrame(summaries[:1]), paths.tables / "q4_summary.csv", overwrite=True
770 )
771 candidates_path = save_table(
772     pd.DataFrame(summaries), paths.tables / "q4_candidates.csv", overwrite=True
773 )
774 rows = pd.DataFrame(_uav_rows(best))
775 uavs_path = save_table(rows, paths.tables / "q4_uavs.csv", overwrite=True)
776 interval_rows = []
777 groups = [
778     ("joint-union-cover", best.joint_intervals),
779     ("conservative-single-cloud-union", best.conservative_intervals),
780     *(
781         (f"{name}-strict", intervals)
782         for name, intervals in zip(UAV_NAMES, best.individual_intervals, strict=True)
783     ),
784 ]
785 for criterion, intervals in groups:
786     for index, interval in enumerate(intervals, 1):
787         interval_rows.append(
788             {
789                 "criterion": criterion,
790                 "interval": index,
791                 "start_s": interval.start,
792                 "end_s": interval.end,
793                 "duration_s": interval.duration,
794             }
795         )
796 intervals_path = save_table(
797     pd.DataFrame(interval_rows), paths.tables / "q4_intervals.csv", overwrite=True
798 )
799 workbook_path = paths.tables / "result2.xlsx"
800 with pd.ExcelWriter(workbook_path, engine="openpyxl") as writer:
801     rows.to_excel(writer, sheet_name="provisional_result2", index=False)
802     pd.DataFrame(
803         [
804             {
805                 "schema_status": "provisional",
806                 "reason": "official result2.xlsx template is missing",
807                 "mapping_action": "map explicit fields when the official template is supplied",
808             }
809         ]
810     ).to_excel(writer, sheet_name="schema_note", index=False)
811 optimization_path = paths.logs / "q4_optimization.json"
812 optimization_path.write_text(
813     json.dumps(
814         {
815             "model": "q4-three-uav-joint-ray-cover-v1",
816             "best": {**summary_row(best, 1), "uavs": _uav_rows(best)},
817             "near_optimal_candidates": summaries,
818             "solver": solver,
819         },
820         ensure_ascii=False,
821         indent=2,
822     )
823     + "\n",
824     encoding="utf-8",
825 )
826 validation_path = paths.logs / "q4_validation.json"
827 validation_path.write_text(
828     json.dumps(_validation_record(best, config), ensure_ascii=False, indent=2) + "\n",
829     encoding="utf-8",
830 )

```

```

831     return [
832         summary_path,
833         candidates_path,
834         uavs_path,
835         intervals_path,
836         workbook_path,
837         stage_path,
838         optimization_path,
839         validation_path,
840     ]

```

B.6 问题五程序

```

1  """Question 5 hierarchical optimization for five UAVs and three missiles."""
2
3  from __future__ import annotations
4
5  import json
6  from collections.abc import Sequence
7  from dataclasses import asdict, dataclass
8  from itertools import product
9  from pathlib import Path
10 from typing import Any
11
12 import numpy as np
13 import pandas as pd
14 from scipy.optimize import brentq, differential_evolution, minimize
15
16 from mmkit.artifacts import prepare_output_paths, save_table
17
18 from .q1 import CLOUD_LIFETIME, MISSILE_SPEED, OcclusionInterval
19 from .q3 import (
20     BombGeometry,
21     _cloud_center,
22     _continuous_worst_margin,
23     _coverage_margins,
24     _duration,
25     _independent_ray_intersections,
26     _merge_intervals,
27     _surface_covering_radius,
28     _surface_points,
29 )
30 from .q4 import Q4Strategy, UAVControl
31 from .q4 import _bomb_geometries as q4_bomb_geometries
32
33 UAV_NAMES = ("FY1", "FY2", "FY3", "FY4", "FY5")
34 MISSILE_NAMES = ("M1", "M2", "M3")
35 TARGET_CENTER = np.array([0.0, 200.0, 5.0])
36
37
38 @dataclass(frozen=True, slots=True)
39 class UAVPlan:
40     """Shared flight controls and three automatically spaced bomb slots."""
41
42     theta_rad: float
43     speed_m_s: float
44     s0_s: float
45     s1_s: float
46     s2_s: float
47     fuse1_s: float
48     fuse2_s: float

```

```

49     fuse3_s: float
50
51     @property
52     def release_times_s(self) -> tuple[float, float, float]:
53         return (
54             self.s0_s,
55             self.s0_s + 1.0 + self.s1_s,
56             self.s0_s + 2.0 + self.s1_s + self.s2_s,
57         )
58
59     @property
60     def fuse_delays_s(self) -> tuple[float, float, float]:
61         return (self.fuse1_s, self.fuse2_s, self.fuse3_s)
62
63
64 @dataclass(frozen=True, slots=True)
65 class Q5Strategy:
66     """Forty-variable strategy represented as five eight-variable UAV blocks."""
67
68     plans: tuple[UAVPlan, UAVPlan, UAVPlan, UAVPlan, UAVPlan]
69
70
71 @dataclass(frozen=True, slots=True)
72 class MissileEvaluation:
73     """Formal joint coverage result for one missile."""
74
75     missile: str
76     intervals: tuple[OcclusionInterval, ...]
77
78     @property
79     def duration_s(self) -> float:
80         return _duration(self.intervals)
81
82
83 @dataclass(frozen=True, slots=True)
84 class Q5Evaluation:
85     """Formal three-missile evaluation and per-bomb deletion contributions."""
86
87     strategy: Q5Strategy
88     bombs: tuple[BombGeometry, ...]
89     missiles: tuple[MissileEvaluation, MissileEvaluation, MissileEvaluation]
90     deletion_losses_s: tuple[tuple[float, float, float], ...]
91
92     @property
93     def durations_s(self) -> tuple[float, float, float]:
94         return tuple(item.duration_s for item in self.missiles)
95
96     @property
97     def j_sum_s(self) -> float:
98         return sum(self.durations_s)
99
100     @property
101     def j_min_s(self) -> float:
102         return min(self.durations_s)
103
104
105 def missile_hit_time(initial: np.ndarray) -> float:
106     """Return time for an arbitrary missile to reach the false target at the origin."""
107     return float(np.linalg.norm(initial) / MISSILE_SPEED)
108
109
110 def missile_position(initial: np.ndarray, time: float) -> np.ndarray:

```

```

111     """Return an arbitrary missile position at absolute task time."""
112     return initial * (1.0 - time / missile_hit_time(initial))
113
114
115 def _strategy_from_array(values: np.ndarray) -> Q5Strategy:
116     array = np.asarray(values, dtype=float).reshape(5, 8)
117     return Q5Strategy(tuple(UAVPlan(*(float(value) for value in row)) for row in array))
118
119
120 def _strategy_vector(strategy: Q5Strategy) -> np.ndarray:
121     return np.array([value for plan in strategy.plans for value in asdict(plan).values()])
122
123
124 def _uav_initials(config: dict[str, Any]) -> tuple[np.ndarray, ...]:
125     return tuple(np.asarray(config["uavs"][name], dtype=float) for name in UAV_NAMES)
126
127
128 def _missile_initials(config: dict[str, Any]) -> tuple[np.ndarray, ...]:
129     return tuple(np.asarray(config["missiles"][name], dtype=float) for name in MISSILE_NAMES)
130
131
132 def _bomb_geometries(strategy: Q5Strategy, config: dict[str, Any]) -> tuple[BombGeometry, ...]:
133     gravity = float(config["gravity"])
134     bombs = []
135     number = 1
136     for plan, initial in zip(strategy.plans, _uav_initials(config), strict=True):
137         heading = np.array([np.cos(plan.theta_rad), np.sin(plan.theta_rad), 0.0])
138         for release, fuse in zip(plan.release_times_s, plan.fuse_delays_s, strict=True):
139             explosion_time = release + fuse
140             release_point = initial + plan.speed_m_s * release * heading
141             explosion_point = initial + plan.speed_m_s * explosion_time * heading
142             explosion_point += np.array([0.0, 0.0, -0.5 * gravity * fuse**2])
143             bombs.append(
144                 BombGeometry(
145                     number,
146                     release,
147                     fuse,
148                     explosion_time,
149                     tuple(float(value) for value in release_point),
150                     tuple(float(value) for value in explosion_point),
151                 )
152             )
153             number += 1
154     return tuple(bombs)
155
156
157 def _fuse_maximum(initial: np.ndarray, gravity: float) -> float:
158     return float(np.sqrt(2.0 * initial[2] / gravity))
159
160
161 def _block_bounds(initial: np.ndarray, config: dict[str, Any]) -> list[tuple[float, float]]:
162     bounds = config["bounds"]
163     fuse = (0.0, _fuse_maximum(initial, float(config["gravity"])))
164     return [
165         tuple(bounds["theta_rad"]),
166         tuple(bounds["speed_m_s"]),
167         tuple(bounds["s0_s"]),
168         tuple(bounds["s1_s"]),
169         tuple(bounds["s2_s"]),
170         fuse,
171         fuse,
172         fuse,

```

```

173 ]
174
175
176 def _feasibility_violation(strategy: Q5Strategy, config: dict[str, Any]) -> float:
177     violation = 0.0
178     latest_hit = max(missile_hit_time(item) for item in _missile_initials(config))
179     bombs = _bomb_geometries(strategy, config)
180     for uav_index, (plan, initial) in enumerate(
181         zip(strategy.plans, _uav_initials(config), strict=True)
182     ):
183         fuse_max = _fuse_maximum(initial, float(config["gravity"]))
184         violation += max(0.0, 70.0 - plan.speed_m_s) / 70.0
185         violation += max(0.0, plan.speed_m_s - 140.0) / 70.0
186         for value in asdict(plan).values():
187             violation += max(0.0, -value - 1e-12) / max(1.0, fuse_max)
188         for bomb in bombs[3 * uav_index : 3 * uav_index + 3]:
189             violation += max(0.0, bomb.explosion_time_s - latest_hit) / latest_hit
190             violation += max(0.0, -bomb.explosion_point[2]) / float(initial[2])
191     return violation
192
193
194 def _active_bombs(time: float, bombs: Sequence[BombGeometry]) -> tuple[BombGeometry, ...]:
195     return tuple(
196         bomb
197         for bomb in bombs
198         if bomb.explosion_time_s <= time <= bomb.explosion_time_s + CLOUD_LIFETIME
199     )
200
201
202 def _margin(
203     time: float,
204     bombs: Sequence[BombGeometry],
205     missile_initial: np.ndarray,
206     settings: dict[str, Any],
207     *,
208     refine: bool,
209 ) -> float:
210     active = _active_bombs(time, bombs)
211     if not active or time > missile_hit_time(missile_initial):
212         return float("inf")
213     missile = missile_position(missile_initial, time)
214     angles = int(settings["surface_angles"])
215     levels = int(settings["surface_levels"])
216     points = _surface_points(angles, levels)
217     margins = _coverage_margins(points, missile, active, time)
218     sampled = float(np.max(margins))
219     if not refine or sampled > 0.0:
220         return sampled
221     maximum_angles = int(settings.get("adaptive_max_surface_angles", angles))
222     maximum_levels = int(settings.get("adaptive_max_surface_levels", levels))
223     while sampled + _surface_covering_radius(angles, levels) > 0.0 and (
224         angles < maximum_angles or levels < maximum_levels
225     ):
226         angles = min(maximum_angles, 2 * angles)
227         levels = min(maximum_levels, 2 * (levels - 1) + 1)
228         points = _surface_points(angles, levels)
229         margins = _coverage_margins(points, missile, active, time)
230         sampled = float(np.max(margins))
231         if sampled > 0.0:
232             return sampled
233     upper = sampled + _surface_covering_radius(angles, levels)
234     if upper <= 0.0:

```

```

235         return upper
236     return _continuous_worst_margin(
237         missile,
238         active,
239         time,
240         points,
241         margins,
242         starts=int(settings.get("continuous_refinement_starts", 1)),
243         maxfev=int(settings.get("continuous_refinement_maxfev", 40)),
244     )
245
246
247 def _find_intervals(
248     bombs: Sequence[BombGeometry],
249     missile_initial: np.ndarray,
250     settings: dict[str, Any],
251     *,
252     refine: bool,
253 ) -> tuple[OcclusionInterval, ...]:
254     hit = missile_hit_time(missile_initial)
255     start = min(bomb.explosion_time_s for bomb in bombs)
256     end = min(hit, max(bomb.explosion_time_s + CLOUD_LIFETIME for bomb in bombs))
257     if start > end:
258         return ()
259     step = float(settings.get("time_scan_step_s", settings.get("time_step_s", 0.2)))
260     tolerance = float(settings.get("root_tolerance_s", 1e-8))
261     regular = np.arange(0.0, hit + 0.5 * step, step)
262     regular = regular[(regular >= start) & (regular <= end)]
263     events = np.array(
264         [
265             event
266             for bomb in bombs
267             for event in (bomb.explosion_time_s, bomb.explosion_time_s + CLOUD_LIFETIME)
268             if start <= event <= end
269         ]
270     )
271     probe = max(tolerance * 10.0, step * 1e-5)
272     probes = np.array(
273         [
274             value
275             for event in events
276             for value in (max(start, event - probe), min(end, event + probe))
277         ]
278     )
279     times = np.unique(np.concatenate(([start, end], regular, events, probes)))
280
281     def function(time: float) -> float:
282         return _margin(time, bombs, missile_initial, settings, refine=refine)
283
284     values = np.array([function(float(time)) for time in times])
285     inside = values <= 0.0
286     result = []
287     current = float(times[0]) if inside[0] else None
288     for index in np.flatnonzero(inside[1:] != inside[:-1]):
289         left, right = float(times[index]), float(times[index + 1])
290         midpoint = 0.5 * (left + right)
291         active = tuple(bomb.bomb for bomb in _active_bombs(left, bombs))
292         middle = tuple(bomb.bomb for bomb in _active_bombs(midpoint, bombs))
293         ending = tuple(bomb.bomb for bomb in _active_bombs(right, bombs))
294         if active != middle and middle == ending:
295             boundary = left
296         elif active == middle and middle != ending:

```

```

297         boundary = right
298     else:
299         boundary = float(brentq(function, left, right, xtol=tolerance, rtol=1e-14))
300     if inside[index + 1]:
301         current = boundary
302     elif current is not None:
303         result.append(OcclusionInterval(current, boundary))
304         current = None
305     if current is not None:
306         result.append(OcclusionInterval(current, float(times[-1])))
307     return _merge_intervals(result)
308
309
310 def _sampled_duration(
311     bombs: Sequence[BombGeometry], missile_initial: np.ndarray, settings: dict[str, Any]
312 ) -> tuple[float, float]:
313     hit = missile_hit_time(missile_initial)
314     step = float(settings["time_step_s"])
315     start = min(bomb.explosion_time_s for bomb in bombs)
316     end = min(hit, max(bomb.explosion_time_s + CLOUD_LIFETIME for bomb in bombs))
317     if start > end:
318         return 0.0, float("inf")
319     anchored = np.arange(0.0, hit + 0.5 * step, step)
320     anchored = anchored[(anchored >= start) & (anchored <= end)]
321     events = np.array(
322         [
323             event
324             for bomb in bombs
325             for event in (bomb.explosion_time_s, bomb.explosion_time_s + CLOUD_LIFETIME)
326             if start <= event <= end
327         ]
328     )
329     times = np.unique(np.concatenate(([start, end], anchored, events)))
330     values = np.array(
331         [_margin(float(time), bombs, missile_initial, settings, refine=False) for time in times]
332     )
333     return float(np.trapezoid((values <= 0.0).astype(float), times)), float(np.min(values))
334
335
336 def _single_bomb(control: np.ndarray, initial: np.ndarray, gravity: float) -> BombGeometry:
337     theta, speed, release, fuse = control
338     heading = np.array([np.cos(theta), np.sin(theta), 0.0])
339     explosion_time = release + fuse
340     release_point = initial + speed * release * heading
341     explosion_point = initial + speed * explosion_time * heading
342     explosion_point += np.array([0.0, 0.0, -0.5 * gravity * fuse**2])
343     return BombGeometry(
344         1,
345         float(release),
346         float(fuse),
347         float(explosion_time),
348         tuple(float(value) for value in release_point),
349         tuple(float(value) for value in explosion_point),
350     )
351
352
353 def _single_population(
354     uav: np.ndarray, missile: np.ndarray, fuse_max: float, settings: dict[str, Any], seed: int
355 ) -> np.ndarray:
356     count = 4 * int(settings["popsize"])
357     rng = np.random.default_rng(seed)
358     lower = np.array([0.0, 70.0, 0.0, 0.0])

```



```

359 upper = np.array([2.0 * np.pi, 140.0, 45.0, fuse_max])
360 population = rng.uniform(lower, upper, size=(count, 4))
361 structured = []
362 hit = missile_hit_time(missile)
363 for time in np.linspace(3.0, 0.85 * hit, 40):
364     point = missile_position(missile, float(time))
365     for fraction in (0.05, 0.15, 0.3, 0.5, 0.7):
366         desired = (1.0 - fraction) * point + fraction * TARGET_CENTER
367         displacement = desired[:2] - uav[:2]
368         speed = float(np.linalg.norm(displacement) / time)
369         height_drop = uav[2] - desired[2]
370         if 70.0 <= speed <= 140.0 and height_drop >= 0.0:
371             fuse = float(np.sqrt(2.0 * height_drop / 9.8))
372             release = float(time - fuse)
373             if 0.0 <= release <= 45.0 and fuse <= fuse_max:
374                 structured.append(
375                     np.array(
376                         [
377                             np.arctan2(displacement[1], displacement[0]) % (2 * np.pi),
378                             speed,
379                             release,
380                             fuse,
381                         ]
382                     )
383                 )
384     for index, item in enumerate(structured[:count]):
385         population[index] = item
386     return population
387
388
389 def _candidate_library(
390     config: dict[str, Any],
391 ) -> tuple[list[dict[str, Any]], dict[tuple[int, int], np.ndarray]]:
392     settings = config["single_library"]
393     gravity = float(config["gravity"])
394     records = []
395     best = {}
396     for uav_index, uav in enumerate(_uav_initials(config)):
397         fuse_max = _fuse_maximum(uav, gravity)
398         for missile_index, missile in enumerate(_missile_initials(config)):
399             hit = missile_hit_time(missile)
400
401             def objective(
402                 values: np.ndarray,
403                 fixed_uav: np.ndarray = uav,
404                 fixed_missile: np.ndarray = missile,
405                 fixed_hit: float = hit,
406             ) -> float:
407                 bomb = _single_bomb(values, fixed_uav, gravity)
408                 if bomb.explosion_time_s > fixed_hit or bomb.explosion_point[2] < 0.0:
409                     return 100.0 + max(0.0, bomb.explosion_time_s - fixed_hit)
410                 duration, minimum = _sampled_duration((bomb,), fixed_missile, settings)
411                 return -duration if duration > 0.0 else max(0.0, minimum) / 100.0
412
413             seed = int(config["seed"]) + 10 * uav_index + missile_index
414             result = differential_evolution(
415                 objective,
416                 [(0.0, 2 * np.pi), (70.0, 140.0), (0.0, 45.0), (0.0, fuse_max)],
417                 init=_single_population(uav, missile, fuse_max, settings, seed),
418                 seed=seed,
419                 maxiter=int(settings["maxiter"]),
420                 popsize=int(settings["popsize"]),

```

```

421         tol=float(settings["tol"]),
422         polish=False,
423         workers=1,
424     )
425     vector = np.asarray(result.x, dtype=float)
426     best[(uav_index, missile_index)] = vector
427     records.append(
428         {
429             "uav": UAV_NAMES[uav_index],
430             "missile": MISSILE_NAMES[missile_index],
431             "sampled_duration_s": max(0.0, -float(result.fun)),
432             "physical_feasible": bool(
433                 _single_bomb(vector, uav, gravity).explosion_time_s <= hit
434                 and _single_bomb(vector, uav, gravity).explosion_point[2] >= 0.0
435             ),
436             "positive_coverage_found": bool(float(result.fun) < 0.0),
437             "evaluations": int(result.nfev),
438             "success": bool(result.success),
439             "message": str(result.message),
440             "vector": vector.tolist(),
441         }
442     )
443     return records, best
444
445
446 def _assembled_strategy(
447     assignment: Sequence[int], library: dict[tuple[int, int], np.ndarray]
448 ) -> Q5Strategy:
449     plans = []
450     for uav_index, missile_index in enumerate(assignment):
451         theta, speed, release, fuse = library[(uav_index, missile_index)]
452         base = max(0.0, float(release) - 1.0)
453         plans.append(
454             UAVPlan(
455                 float(theta), float(speed), base, 0.0, 0.0, float(fuse), float(fuse), float(fuse)
456             )
457         )
458     return Q5Strategy(tuple(plans))
459
460
461 def _q4_anchor_strategy(
462     library: dict[tuple[int, int], np.ndarray], config: dict[str, Any]
463 ) -> Q5Strategy:
464     """Embed the three formally verified Q4 bombs as a monotone M1 incumbent."""
465     plans = []
466     for row in config["validation"]["q4_strategy"]:
467         theta, speed, release, fuse = (float(value) for value in row)
468         plans.append(UAVPlan(theta, speed, release, 0.0, 0.0, fuse, fuse, fuse))
469     for uav_index, missile_index in ((3, 1), (4, 2)):
470         theta, speed, release, fuse = library[(uav_index, missile_index)]
471         plans.append(
472             UAVPlan(
473                 float(theta),
474                 float(speed),
475                 float(release),
476                 0.0,
477                 0.0,
478                 float(fuse),
479                 float(fuse),
480                 float(fuse),
481             )
482         )

```

```

483     return Q5Strategy(tuple(plans))
484
485
486 def _proxy_scores(
487     strategy: Q5Strategy, config: dict[str, Any], settings: dict[str, Any]
488 ) -> tuple[float, tuple[float, float, float]]:
489     violation = _feasibility_violation(strategy, config)
490     if violation > 0.0:
491         return -100.0 - 100.0 * violation, (0.0, 0.0, 0.0)
492     bombs = _bomb_geometries(strategy, config)
493     durations = tuple(
494         _sampled_duration(bombs, missile, settings)[0] for missile in _missile_initials(config)
495     )
496     score = sum(durations) - 2.0 * sum(max(0.0, 0.05 - value) for value in durations)
497     return score, durations
498
499
500 def _block_refine(
501     strategy: Q5Strategy, config: dict[str, Any], stage_path: Path
502 ) -> tuple[Q5Strategy, list[dict[str, Any]]]:
503     settings = config["block_refinement"]
504     current = _strategy_vector(strategy)
505     records = []
506     for round_index in range(int(settings["rounds"])):
507         for uav_index, initial in enumerate(_uav_initials(config)):
508             offset = 8 * uav_index
509
510             def objective(block: np.ndarray, fixed_offset: int = offset) -> float:
511                 candidate = current.copy()
512                 candidate[fixed_offset : fixed_offset + 8] = block
513                 return _proxy_scores(_strategy_from_array(candidate), config, settings)[0]
514
515             result = minimize(
516                 objective,
517                 current[offset : offset + 8],
518                 method="Nelder-Mead",
519                 bounds=_block_bounds(initial, config),
520                 options={
521                     "maxfev": int(settings["maxfev_per_block"]),
522                     "xatol": float(settings["xatol"]),
523                     "fatol": float(settings["fatol"]),
524                 },
525             )
526             if float(result.fun) < objective(current[offset : offset + 8]):
527                 current[offset : offset + 8] = result.x
528             score, durations = _proxy_scores(_strategy_from_array(current), config, settings)
529             records.append(
530                 {
531                     "round": round_index + 1,
532                     "uav": UAV_NAMES[uav_index],
533                     "evaluations": int(result.nfev),
534                     "success": bool(result.success),
535                     "message": str(result.message),
536                     "proxy_score_s": score,
537                     "durations_s": durations,
538                 }
539             )
540             stage_path.write_text(
541                 json.dumps({"stage": "block-refinement-in-progress", "records": records}, indent=2)
542                 + "\n",
543                 encoding="utf-8",
544             )

```

```

545     return _strategy_from_array(current), records
546
547
548 def _marginal_fill(
549     strategy: Q5Strategy, config: dict[str, Any], stage_path: Path
550 ) -> tuple[Q5Strategy, list[dict[str, Any]]]:
551     """Search otherwise unused second/third slots while preserving each anchor bomb."""
552     settings = config["marginal_fill"]
553     current = _strategy_vector(strategy)
554     records = []
555     rng = np.random.default_rng(int(config["seed"]) + 900)
556     for name in settings["uav_order"]:
557         uav_index = UAV_NAMES.index(name)
558         offset = 8 * uav_index
559         initial = _uav_initials(config)[uav_index]
560         fuse_max = _fuse_maximum(initial, float(config["gravity"]))
561         lower = np.array([0.0, 0.0, 0.0, 0.0])
562         upper = np.array([20.0, 20.0, fuse_max, fuse_max])
563         population = rng.uniform(lower, upper, size=(int(settings["population_size"]), 4))
564         plan = _strategy_from_array(current).plans[uav_index]
565         anchor = np.array([plan.s1_s, plan.s2_s, plan.fuse2_s, plan.fuse3_s])
566         population[0] = anchor
567         structured = (
568             (0.0, 0.0, plan.fuse1_s, plan.fuse1_s),
569             (1.0, 1.0, plan.fuse1_s, plan.fuse1_s),
570             (3.0, 3.0, plan.fuse1_s, plan.fuse1_s),
571             (6.0, 6.0, plan.fuse1_s, plan.fuse1_s),
572             (0.0, 2.0, 0.8 * plan.fuse1_s, 1.2 * plan.fuse1_s),
573             (2.0, 5.0, 1.2 * plan.fuse1_s, 0.8 * plan.fuse1_s),
574         )
575         for index, values in enumerate(structured, 1):
576             if index < len(population):
577                 population[index] = np.clip(values, lower, upper)
578
579         base_vector = current.copy()
580
581         def full_vector(
582             values: np.ndarray,
583             base: np.ndarray = base_vector,
584             fixed_offset: int = offset,
585         ) -> np.ndarray:
586             candidate = base.copy()
587             candidate[fixed_offset + 3 : fixed_offset + 5] = values[:2]
588             candidate[fixed_offset + 6 : fixed_offset + 8] = values[2:]
589             return candidate
590
591         def objective(values: np.ndarray) -> float:
592             candidate = _strategy_from_array(full_vector(values))
593             return -_proxy_scores(candidate, config, settings)[0]
594
595         before_score, before_durations = _proxy_scores(
596             _strategy_from_array(current), config, settings
597         )
598         result = differential_evolution(
599             objective,
600             [(0.0, 20.0), (0.0, 20.0), (0.0, fuse_max), (0.0, fuse_max)],
601             init=population,
602             seed=int(config["seed"]) + 900 + uav_index,
603             maxiter=int(settings["maxiter"]),
604             popsize=2,
605             polish=False,
606             workers=1,

```

```

607     )
608     candidate = full_vector(np.asarray(result.x, dtype=float))
609     after_score, after_durations = _proxy_scores(
610         _strategy_from_array(candidate), config, settings
611     )
612     accepted = after_score > before_score + float(settings["improvement_threshold_s"])
613     if accepted:
614         current = candidate
615     records.append(
616         {
617             "uav": name,
618             "evaluations": int(result.nfev),
619             "success": bool(result.success),
620             "message": str(result.message),
621             "before_score_s": before_score,
622             "after_score_s": after_score,
623             "before_durations_s": before_durations,
624             "after_durations_s": after_durations,
625             "accepted": accepted,
626         }
627     )
628     stage_path.write_text(
629         json.dumps({"stage": "marginal-fill-in-progress", "records": records}, indent=2) + "\n",
630         encoding="utf-8",
631     )
632     return _strategy_from_array(current), records
633
634
635 def _formal_evaluation(
636     strategy: Q5Strategy, config: dict[str, Any], *, compute_deletion: bool
637 ) -> Q5Evaluation:
638     bombs = _bomb_geometries(strategy, config)
639     settings = config["formal_evaluation"]
640     missiles = tuple(
641         MissileEvaluation(name, _find_intervals(bombs, initial, settings, refine=True))
642         for name, initial in zip(MISSILE_NAMES, _missile_initials(config), strict=True)
643     )
644     deletion = []
645     if compute_deletion:
646         deletion_settings = config["deletion_evaluation"]
647         for removed in range(15):
648             remaining = tuple(bomb for index, bomb in enumerate(bombs) if index != removed)
649             losses = []
650             for missile, baseline in zip(_missile_initials(config), missiles, strict=True):
651                 intervals = _find_intervals(remaining, missile, deletion_settings, refine=False)
652                 losses.append(
653                     max(
654                         0.0,
655                         baseline.duration_s - min(_duration(intervals), baseline.duration_s),
656                     )
657                 )
658             deletion.append(tuple(losses))
659     else:
660         deletion = [(0.0, 0.0, 0.0)] * 15
661     return Q5Evaluation(strategy, bombs, missiles, tuple(deletion))
662
663
664 def optimize_question_5(
665     config: dict[str, Any], stage_path: Path
666 ) -> tuple[list[Q5Evaluation], dict[str, Any], list[dict[str, Any]]]:
667     """Build the 15-pair library, assemble incumbents, and refine five UAV blocks."""
668     library_records, library = _candidate_library(config)

```

```

669     stage_path.write_text(
670         json.dumps(
671             {"stage": "candidate-library-complete", "candidate_library": library_records}, indent=2
672         )
673         + "\n",
674         encoding="utf-8",
675     )
676     print("Q5 15-pair candidate library complete", flush=True)
677     library_scores = {
678         (UAV_NAMES.index(record["uav"]), MISSILE_NAMES.index(record["missile"])): record[
679             "sampled_duration_s"
680         ]
681         for record in library_records
682     }
683     assignments = []
684     for assignment in product(range(3), repeat=5):
685         if len(set(assignment)) < 3:
686             continue
687         score = sum(library_scores[(uav, missile)] for uav, missile in enumerate(assignment))
688         assignments.append((score, assignment))
689     assignments.sort(reverse=True)
690     assembly_count = int(config["assembly"]["candidate_assignments"])
691     assembled = []
692     for _, assignment in assignments[:assembly_count]:
693         strategy = _assembled_strategy(assignment, library)
694         score, durations = _proxy_scores(strategy, config, config["assembly"])
695         assembled.append((score, min(durations), strategy, assignment, durations))
696     assembled.sort(key=lambda item: (item[0], item[1]), reverse=True)
697     incumbent = _q4_anchor_strategy(library, config)
698     incumbent_score, incumbent_durations = _proxy_scores(incumbent, config, config["assembly"])
699     stage_path.write_text(
700         json.dumps(
701             {
702                 "stage": "assembly-complete",
703                 "q4_anchor_incumbent": {
704                     "proxy_sum_s": incumbent_score,
705                     "durations_s": incumbent_durations,
706                     "retained_for_formal_evaluation": True,
707                 },
708                 "assignments": [
709                     {
710                         "assignment": [MISSILE_NAMES[index] for index in item[3]],
711                         "proxy_sum_s": item[0],
712                         "durations_s": item[4],
713                     }
714                     for item in assembled
715                 ],
716             },
717             indent=2,
718         )
719         + "\n",
720         encoding="utf-8",
721     )
722     print("Q5 feasible three-missile incumbent assembled", flush=True)
723     refined, block_records = _block_refine(incumbent, config, stage_path)
724     filled, fill_records = _marginal_fill(refined, config, stage_path)
725     candidates = [incumbent, refined, filled, *(item[2] for item in assembled[1:])]
726     proxy_ranked = sorted(
727         candidates,
728         key=lambda item: _proxy_scores(item, config, config["block_refinement"])[0],
729         reverse=True,
730     )

```

```

731 count = min(int(config["formal_evaluation"]["candidate_count"]), len(proxy_ranked))
732 formal_strategies = [proxy_ranked[0]]
733 if count > 1:
734     formal_strategies.append(incumbent)
735 formal = []
736 for index, strategy in enumerate(formal_strategies[:count], 1):
737     formal.append(_formal_evaluation(strategy, config, compute_deletion=False))
738     print(f"Q5 formal candidate {index}/{count} complete", flush=True)
739 formal.sort(key=lambda item: (item.j_sum_s, item.j_min_s), reverse=True)
740 formal[0] = _formal_evaluation(formal[0].strategy, config, compute_deletion=True)
741 stage_path.write_text(
742     json.dumps(
743         {
744             "stage": "search-complete",
745             "library_size": len(library_records),
746             "block_records": block_records,
747             "marginal_fill_records": fill_records,
748             "formal_scores": [
749                 {
750                     "j_sum_s": item.j_sum_s,
751                     "j_min_s": item.j_min_s,
752                     "durations_s": item.durations_s,
753                 }
754                 for item in formal
755             ],
756             "q4_anchor_incumbent_retained": True,
757             "sampled_time_grid": "absolute task time t=0",
758         },
759         indent=2,
760     )
761     + "\n",
762     encoding="utf-8",
763 )
764 solver = {
765     "seed": int(config["seed"]),
766     "method": "15 single-pair DE searches, assignment assembly, and five bounded 8D blocks",
767     "candidate_library_size": len(library_records),
768     "block_refinements": block_records,
769     "marginal_fill": fill_records,
770     "formal_settings": config["formal_evaluation"],
771     "status": "budget-limited-hierarchical-search-with-feasible-incumbent",
772     "convergence_claimed": False,
773     "sampled_time_grid": "absolute task time t=0 plus cloud events",
774     "q4_anchor_incumbent_retained": True,
775 }
776 return formal, solver, library_records
777
778
779 def _summary(evaluation: Q5Evaluation, rank: int) -> dict[str, Any]:
780     return {
781         "rank": rank,
782         "D1_s": evaluation.durations_s[0],
783         "D2_s": evaluation.durations_s[1],
784         "D3_s": evaluation.durations_s[2],
785         "J_sum_s": evaluation.j_sum_s,
786         "J_min_s": evaluation.j_min_s,
787     }
788
789
790 def _bomb_rows(evaluation: Q5Evaluation) -> list[dict[str, Any]]:
791     rows = []
792     for index, bomb in enumerate(evaluation.bombs):

```

```

793     uav_index, slot = divmod(index, 3)
794     plan = evaluation.strategy.plans[uav_index]
795     losses = evaluation.deletion_losses_s[index]
796     rows.append(
797         {
798             "schema_status": "provisional-schema; official result3.xlsx unavailable",
799             "uav": UAV_NAMES[uav_index],
800             "bomb_slot": slot + 1,
801             "used": sum(losses) > 1e-8,
802             "theta_rad": plan.theta_rad,
803             "theta_deg": float(np.degrees(plan.theta_rad) % 360.0),
804             "speed_m_s": plan.speed_m_s,
805             "release_time_s": bomb.release_time_s,
806             "release_x_m": bomb.release_point[0],
807             "release_y_m": bomb.release_point[1],
808             "release_z_m": bomb.release_point[2],
809             "fuse_delay_s": bomb.fuse_delay_s,
810             "explosion_time_s": bomb.explosion_time_s,
811             "explosion_x_m": bomb.explosion_point[0],
812             "explosion_y_m": bomb.explosion_point[1],
813             "explosion_z_m": bomb.explosion_point[2],
814             "M1_deletion_loss_s": losses[0],
815             "M2_deletion_loss_s": losses[1],
816             "M3_deletion_loss_s": losses[2],
817             "J_sum_deletion_loss_s": sum(losses),
818         }
819     )
820     return rows
821
822
823 def _validation(evaluation: Q5Evaluation, config: dict[str, Any]) -> dict[str, Any]:
824     bombs = evaluation.bombs
825     formal = config["formal_evaluation"]
826     validation = config["validation"]
827     convergence = {}
828     boundaries = {}
829     direct = {}
830     points = _surface_points(
831         int(validation["direct_surface_angles"]), int(validation["direct_surface_levels"])
832     )
833     for name, missile, baseline in zip(
834         MISSILE_NAMES, _missile_initials(config), evaluation.missiles, strict=True
835     ):
836         convergence[name] = []
837         for index, settings in enumerate(validation["convergence_settings"]):
838             merged = {
839                 **settings,
840                 **(
841                     {
842                         "adaptive_max_surface_angles": formal["adaptive_max_surface_angles"],
843                         "adaptive_max_surface_levels": formal["adaptive_max_surface_levels"],
844                         "continuous_refinement_starts": 1,
845                         "continuous_refinement_maxfev": 40,
846                     }
847                     if index == 2
848                     else {}
849                 ),
850             }
851             intervals = _find_intervals(bombs, missile, merged, refine=index == 2)
852             convergence[name].append({**settings, "duration_s": _duration(intervals)})
853             boundaries[name], direct[name] = [], []
854         for interval in baseline.intervals:

```



```

855     for location, time in (
856         ("before-start", interval.start - validation["boundary_probe_s"]),
857         ("start", interval.start),
858         ("midpoint", 0.5 * (interval.start + interval.end)),
859         ("end", interval.end),
860         ("after-end", interval.end + validation["boundary_probe_s"]),
861     ):
862         margin = _margin(time, bombs, missile, formal, refine=True)
863         active = _active_bombs(time, bombs)
864         covered = _independent_ray_intersections(
865             points,
866             missile_position(missile, time),
867             [_cloud_center(time, bomb) for bomb in active],
868         )
869         boundaries[name].append({"location": location, "time_s": time, "margin_m": margin})
870         direct[name].append(
871             {
872                 "location": location,
873                 "time_s": time,
874                 "sampled_rays": len(points),
875                 "uncovered_rays": int(np.count_nonzero(~covered)),
876                 "all_covered": bool(np.all(covered)),
877             }
878         )
879     rng = np.random.default_rng(int(config["seed"]) + 500)
880     permutations = []
881     medium = validation["convergence_settings"][1]
882     for _ in range(int(validation["permutation_samples"])):
883         order = tuple(bombs[index] for index in rng.permutation(len(bombs)))
884         permutations.append(
885             [
886                 _duration(_find_intervals(order, missile, medium, refine=False))
887                 for missile in _missile_initials(config)
888             ]
889         )
890     q4_strategy = Q4Strategy(
891         tuple(UAVControl(*(float(value) for value in row)) for row in validation["q4_strategy"])
892     )
893     q4_config = {
894         "gravity": config["gravity"],
895         "uavs": {name: config["uavs"][name] for name in UAV_NAMES[:3]},
896     }
897     q4_bombs = q4_bomb_geometries(q4_strategy, float(config["gravity"]), q4_config)
898     q4_duration = _duration(
899         _find_intervals(q4_bombs, _missile_initials(config)[0], formal, refine=True)
900     )
901     gaps = [np.diff(plan.release_times_s).tolist() for plan in evaluation.strategy.plans]
902     return {
903         "feasibility": {
904             "violation": _feasibility_violation(evaluation.strategy, config),
905             "release_gaps_s": gaps,
906             "all_release_gaps_at_least_1s": all(min(item) >= 1.0 for item in gaps),
907             "all_durations_positive": all(value > 0.0 for value in evaluation.durations_s),
908             "explosion_heights_m": [bomb.explosion_point[2] for bomb in bombs],
909         },
910         "surface_time_convergence": convergence,
911         "boundary_checks": boundaries,
912         "independent_unit_direction_ray_checks": direct,
913         "label_permutation_duration_samples_s": permutations,
914         "label_permutation_max_deviation_s": max(
915             (
916                 abs(sample[index] - permutations[0][index])

```

```

917         for sample in permutations[1:]
918         for index in range(3)
919     ),
920     default=0.0,
921 ),
922 "q4_m1_geometry_regression": {
923     "computed_duration_s": q4_duration,
924     "expected_duration_s": float(validation["q4_regression_expected_s"]),
925     "absolute_difference_s": abs(
926         q4_duration - float(validation["q4_regression_expected_s"])
927     ),
928     "tolerance_s": float(validation["q4_regression_tolerance_s"]),
929     "within_tolerance": abs(q4_duration - float(validation["q4_regression_expected_s"]))
930     <= float(validation["q4_regression_tolerance_s"]),
931 },
932 "deletion_contributions_s": [list(item) for item in evaluation.deletion_losses_s],
933 "arithmetic": {
934     "durations_s": evaluation.durations_s,
935     "reported_J_sum_s": evaluation.j_sum_s,
936     "recomputed_J_sum_s": sum(evaluation.durations_s),
937     "consistent": evaluation.j_sum_s == sum(evaluation.durations_s),
938 },
939 "q4_monotone_lower_bound": {
940     "q5_D1_s": evaluation.durations_s[0],
941     "q4_reference_s": float(validation["q4_regression_expected_s"]),
942     "satisfied": evaluation.durations_s[0] + float(validation["q4_regression_tolerance_s"])
943     >= float(validation["q4_regression_expected_s"]),
944 },
945 }
946
947
948 def run_question_5(project_root: Path, config: dict[str, Any]) -> list[Path]:
949     """Run Question 5 and save provisional tables, workbook, logs, and stage state."""
950     paths = prepare_output_paths(project_root / "outputs")
951     stage_path = paths.logs / "q5_search_stage.json"
952     if bool(config.get("formal_only_persistent_rerun", False)):
953         previous_stage = (
954             json.loads(stage_path.read_text(encoding="utf-8")) if stage_path.exists() else None
955         )
956         persistent = _strategy_from_array(
957             np.asarray(config["persistent_incumbent"]["vector"], dtype=float)
958         )
959         evaluations = [_formal_evaluation(persistent, config, compute_deletion=True)]
960         library_path_existing = paths.tables / "q5_candidate_library.csv"
961         library = (
962             pd.read_csv(library_path_existing).to_dict(orient="records")
963             if library_path_existing.exists()
964             else []
965         )
966         solver = {
967             "seed": int(config["seed"]),
968             "status": "persistent-incumbent-formal-rerun",
969             "convergence_claimed": False,
970             "persistent_incumbent": config["persistent_incumbent"],
971             "marginal_fill_comparison": {
972                 "candidate_J_sum_s": 13.869304210239239,
973                 "retained_incumbent_J_sum_s": evaluations[0].j_sum_s,
974                 "candidate_improved_incumbent": False,
975             },
976             "previous_search_stage": previous_stage,
977             "formal_settings": config["formal_evaluation"],
978             "deletion_settings": config["deletion_evaluation"],

```

```

979     }
980     stage_path.write_text(
981         json.dumps(
982             {
983                 "stage": "search-complete",
984                 "mode": "persistent-incumbent-formal-rerun",
985                 "persistent_J_sum_s": evaluations[0].j_sum_s,
986                 "marginal_fill_candidate_J_sum_s": 13.869304210239239,
987                 "retained_persistent_incumbent": True,
988                 "previous_search_stage": previous_stage,
989             },
990             indent=2,
991         )
992         + "\n",
993         encoding="utf-8",
994     )
995 else:
996     evaluations, solver, library = optimize_question_5(config, stage_path)
997 if not evaluations or evaluations[0].j_min_s <= 0.0:
998     raise RuntimeError("Question 5 produced no strategy covering all three missiles")
999 best = evaluations[0]
1000 summaries = [
1001     _summary(item, rank)
1002     for rank, item in enumerate(evaluations[: int(config["near_optimal_count"])], 1)
1003 ]
1004 summary_path = save_table(
1005     pd.DataFrame(summaries[:1]), paths.tables / "q5_summary.csv", overwrite=True
1006 )
1007 candidates_path = save_table(
1008     pd.DataFrame(summaries), paths.tables / "q5_candidates.csv", overwrite=True
1009 )
1010 library_path = save_table(
1011     pd.DataFrame(library), paths.tables / "q5_candidate_library.csv", overwrite=True
1012 )
1013 rows = pd.DataFrame(_bomb_rows(best))
1014 bombs_path = save_table(rows, paths.tables / "q5_bombs.csv", overwrite=True)
1015 interval_rows = [
1016     {
1017         "missile": item.missile,
1018         "interval": number,
1019         "start_s": interval.start,
1020         "end_s": interval.end,
1021         "duration_s": interval.duration,
1022     }
1023     for item in best.missiles
1024     for number, interval in enumerate(item.intervals, 1)
1025 ]
1026 intervals_path = save_table(
1027     pd.DataFrame(interval_rows), paths.tables / "q5_intervals.csv", overwrite=True
1028 )
1029 workbook_path = paths.tables / "result3.xlsx"
1030 with pd.ExcelWriter(workbook_path, engine="openpyxl") as writer:
1031     rows.to_excel(writer, sheet_name="provisional_result3", index=False)
1032     pd.DataFrame(
1033         [
1034             {
1035                 "schema_status": "provisional",
1036                 "reason": "official result3.xlsx template is missing",
1037                 "mapping_action": "map explicit fields when official template is supplied",
1038             }
1039         ]
1040     ).to_excel(writer, sheet_name="schema_note", index=False)

```

```

1041 optimization_path = paths.logs / "q5_optimization.json"
1042 optimization_path.write_text(
1043     json.dumps(
1044         {
1045             "model": "q5-hierarchical-three-missile-joint-cover-v1",
1046             "objective": "J_sum=D1+D2+D3; J_min reported",
1047             "best": {**_summary(best, 1), "bombs": _bomb_rows(best)},
1048             "near_optimal_candidates": summaries,
1049             "solver": solver,
1050         },
1051         ensure_ascii=False,
1052         indent=2,
1053     )
1054     + "\n",
1055     encoding="utf-8",
1056 )
1057 validation_path = paths.logs / "q5_validation.json"
1058 validation_path.write_text(
1059     json.dumps(_validation(best, config), ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
1060 )
1061 return [
1062     summary_path,
1063     candidates_path,
1064     library_path,
1065     bombs_path,
1066     intervals_path,
1067     workbook_path,
1068     stage_path,
1069     optimization_path,
1070     validation_path,
1071 ]

```